

1. УПРАВЛЕНИЕ БАЗАМИ ДАННЫХ

1.1. Файловые системы

Определение. Файловые системы – это набор программ, которые выполняют для пользователей некоторые операции (например, создание отчета). Каждая программа определяет свои собственные данные и управляет ими.

Файловые системы были первой попыткой компьютеризировать ручные картотеки. Ручные картотеки позволяют успешно справиться с поставленными задачами, если количество хранимых объектов невелико. Также они пригодны и для работы с большим количеством объектов, которые нужно только хранить и извлекать. Однако они совершенно не подходят для тех случаев, когда требуется установить перекрестные связи или выполнить обработку сведений. Вместо организации централизованного хранилища всех данных предприятия, был использован децентрализованный подход, при котором сотрудники каждого отдела при помощи специалистов по обработке данных работали со своими собственными данными и хранили их с своим отделом.

Существует ряд ограничений, присущих файловым системам:

- разделение и изоляция данных;
- дублирование данных;
- зависимость от данных. Физическая структура и способ хранения записей файлов данных жестко зафиксированы в коде программ приложений. Это означает, что изменить существующую структуру данных достаточно сложно. Такая особенность файловых систем называется **зависимостью от программ и данных** (program-data dependence);
- несовместимость файлов;
- фиксированные запросы и быстрое увеличение количества приложений.

1.2. Системы баз данных

Для повышения эффективности работы необходимо использовать новый подход, а именно **базу данных** (database) и **систему управления базами данных** (Database Management System – DBMS).

Определение. База данных – это совместно используемый набор логически связанных данных (и описание этих данных), предназначенный для удовлетворения информационных потребностей организации.

Рассмотрим данное определение более подробно. База данных – это единое большое хранилище данных, которое создается однократно, а затем используется одновременно многими пользователями разных подразделений. Вместо разрозненных данных с избыточными данными, здесь все данные собраны вместе с минимальной долей избыточности. База данных уже не принадлежит какому-либо единственному отделу, а является общим корпоративным ресурсом. При этом база данных хранит не только рабочие данные этой организации, но и их описания. По этой причине базу данных называют *набором интегрированных записей с самоописанием*. В совокупности описание данных называется **системным каталогом** (system catalog), или **словарем данных** (data dictionary), а сами элементы описания принято называть **метаданными** (meta-data), т. е. «данными о данных». Именно наличие самоописания данных в базе данных обеспечивает в ней **независимость между программами и данными** (program-data independence).

Подход, основанный на применении баз данных, где определение данных отделено от приложений, очень похож на подход, используемый при разработке современного программного обеспечения, когда наряду с внутренним определением объекта существует его внешнее определение. Пользователи объекта видят только его внешнее определение и не заботятся о том, как он определяется и как функционирует. Одно из преимуществ такого подхода, а именно **абстрагирование данных** (data abstraction), заключается в том, что можно изменить внутреннее определение объекта без каких-либо последствий для его пользовате-

лей при условии неизменности внешнего определения. Аналогичным образом в подходе с использованием баз данных структура данных отделена от приложений и хранится в базе данных. Добавление новых структур данных или изменение существующих никак не влияет на приложения при условии, что они не зависят непосредственно от изменяемых компонентов. Т.е., добавление нового поля в запись или создание нового файла никак не повлияет на работу имеющихся приложений. Однако удаление поля из используемого приложением файла повлияет на это приложение, а потому его также потребуется соответствующим образом модифицировать.

Под понятием «логически связанных» данных из определения базы данных понимается процесс выделения сущностей, атрибутов и связей. **Сущностью** (entity) называется отдельный тип объекта организации (человек, место или вещь, понятие или событие), который необходимо представить в базе данных. **Атрибутом** (attribute) называется свойство, которое описывает некоторую характеристику описываемого объекта. **Связь** (relationship) – это объединение нескольких сущностей.

На рис. 1.1 приведен один из вариантов нотации диаграммы «сущность – связь», демонстрирующая отношения между объектами банковской системы. Согласно этой диаграмме каждый БАНК ИМЕЕТ один или более БАНКОВСКИХ СЧЕТОВ. Кроме того, каждый КЛИЕНТ МОЖЕТ ВЛАДЕТЬ (одновременно) одной или более КРЕДИТНОЙ КАРТОЙ и одним или более БАНКОВСКИМ СЧЕТОМ, каждый из которых ОПРЕДЕЛЯЕТ в точности одну КРЕДИТНУЮ КАРТУ (отметим, что у клиента может и не быть ни счета, ни кредитной карты). КАЖДАЯ КРЕДИТНАЯ КАРТА ИМЕЕТ ровно один зависимый от нее ПАРОЛЬ КАРТЫ, а каждый КЛИЕНТ ЗНАЕТ (но может и забыть) ПАРОЛЬ КАРТЫ.

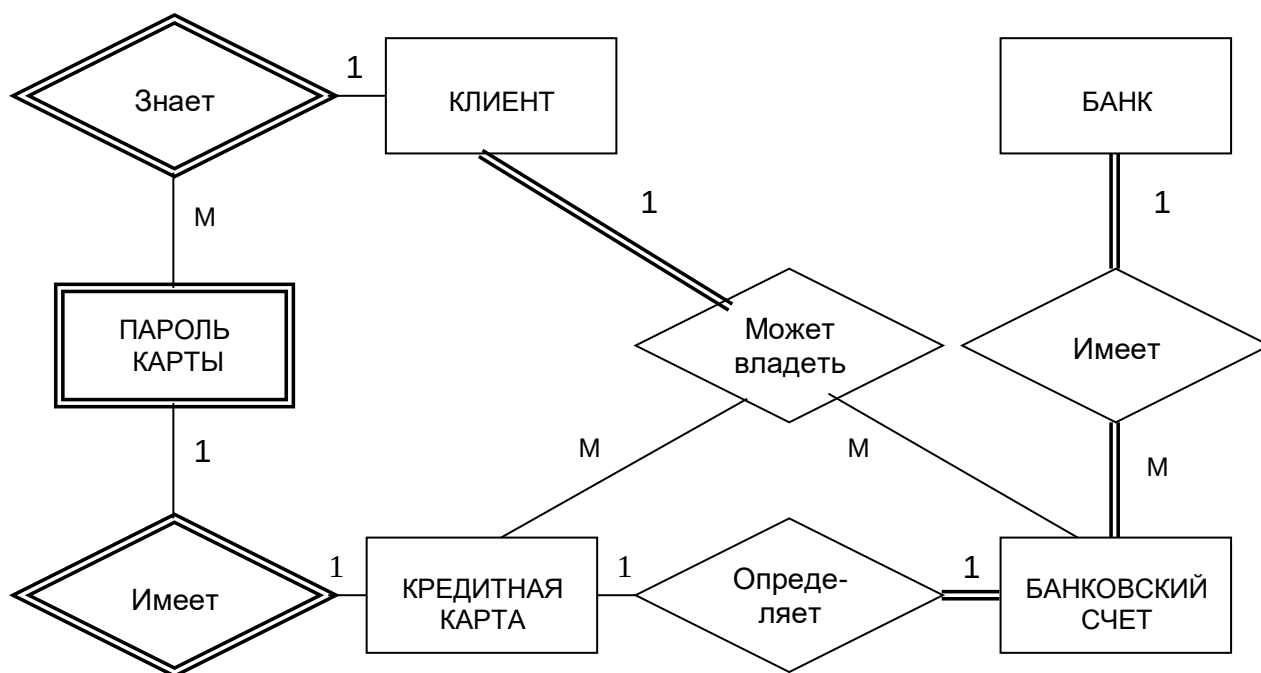


Рис. 1.1. ER-диаграмма в нотации Чена

СУБД является программным обеспечением, которое взаимодействует с прикладными программами пользователей и базой данных и обладает приведенными ниже возможностями:

- позволяет определять базу данных с помощью **языка определения данных** (DDL – Data Definition Language). Язык DDL предоставляет пользователям средства указания типа данных и их структуры, а также средства задания ограничений для информации, хранимой в базе данных;

– позволяет вставлять, обновлять, удалять и извлекать информацию из базы данных с помощью **языка управления данными** (DML – Data Manipulation Language). Наличие централизованного хранилища всех данных и их описаний позволяет использовать язык DML как общий инструмент организации запросов, который иногда называют **языком запросов** (query language). Наличие языка запросов позволяет устранить присущие файловым системам ограничения, при которых пользователям приходится иметь дело только с фиксированным набором запросов или постоянно возрастающим количеством программ. Существует два разновидности языков DML – **процедурные** (procedural) и **непроцедурные** (non-procedural) языки, которые отличаются способом извлечения данных. Основное отличие между ними заключается в том, что процедурные языки обычно обрабатывают информацию в базе данных последовательно, запись за записью, а непроцедурные языки оперируют сразу целыми наборами записей. Поэтому с помощью процедурных языков DML обычно указывается, *как* можно получить желаемый результат, тогда как непроцедурные языки DML используются для описания того, *что* следует получить. Наиболее распространенным типом непроцедурного языка является язык структурированных запросов (Structured Query Language - SQL), который в настоящее время определяется специальным стандартом и *фактически* является обязательным языком для любых реляционных СУБД;

– предоставляет контролируемый доступ к базе данных с помощью следующих средств: системы обеспечения безопасности, предотвращающей несанкционированный доступ к базе данных со стороны пользователей; системы поддержки целостности данных, обеспечивающей непротиворечивое состояние хранимых данных; системы управления параллельной работой приложений, контролирующей процессы их совместного доступа к базе данных; системы восстановления, позволяющей восстановить базу данных до предыдущего непротиворечивого состояния, нарушенного в результате сбоя аппаратного или программного обеспечения; доступного пользователям каталога, содержащего описание хранимой в базе данных информации.

Обладание указанными выше функциональными возможностями превращает СУБД в мощный инструмент обработки информации. Однако поскольку для конечных пользователей неважно, насколько проста или сложна внутренняя организация системы, существует мнение, что СУБД усложняет работу, предоставляя пользователям гораздо большее количество данных, чем им действительно требуется. Для решения этой проблемы в СУБД имеется механизм создания **представления** (view), который позволяет любому пользователю иметь собственный взгляд на базу данных. Язык DDL включает средства определения представлений, каждое из которых является некоторым подмножеством базы данных. Помимо упрощения работы за счет предоставления пользователям только действительно необходимых им данных, представления обладают некоторыми другими достоинствами:

– обеспечивают дополнительный уровень безопасности. Представления могут создаваться с целью исключения тех данных, которые не должны видеть некоторые пользователи. Например, можно создать некоторое представление, которое позволит руководителям отделений и сотрудникам расчетного отдела бухгалтерии просматривать все данные о персонале, включая сведения об их зарплате. В то же время для организации доступа к данным других пользователей можно создать еще одно представление, из которого все сведения о зарплате будут исключены;

– предоставляют механизм настройки внешнего интерфейса базы данных;

– позволяют сохранять внешний интерфейс базы данных непротиворечивым и неизменным даже при внесении изменений в ее структуру (добавление или удаление полей, изменений связей, разбиение файлов, их реорганизация или переименование). Если в файл добавляются или из него удаляются поля, не используемые в некотором представлении, то такие изменения никак не отразятся на данном представлении. Таким образом, представление обеспечивает полную независимость программ от реальной структуры данных, что позволяет устранить важнейший недостаток файловых систем.

Однако реальный объем функциональных возможностей, предлагаемых в конкретной СУБД, отличается от продукта к продукту. Например, в СУБД для персонального компьютера может не поддерживаться параллельный совместный доступ, а управление режимом безопасности, поддержанием целостности данных и восстановлением будет присутствовать только в очень ограниченной степени. В то же время современные мощные многопользовательские СУБД, представляющие собой чрезвычайно сложное программное обеспечение, состоящее из миллионов строк кода и многих томов документации, предлагают все перечисленные выше функциональные возможности. Программное обеспечение СУБД постоянно совершенствуется для удовлетворения новых требований пользователей (мультимедийные базы данных) и поэтому функциональная часть СУБД никогда не будет статичной.

Как показано на рис. 1.2, в среде СУБД можно выделить пять основных компонентов: аппаратное обеспечение, программное обеспечение, данные, процедуры и пользователи.

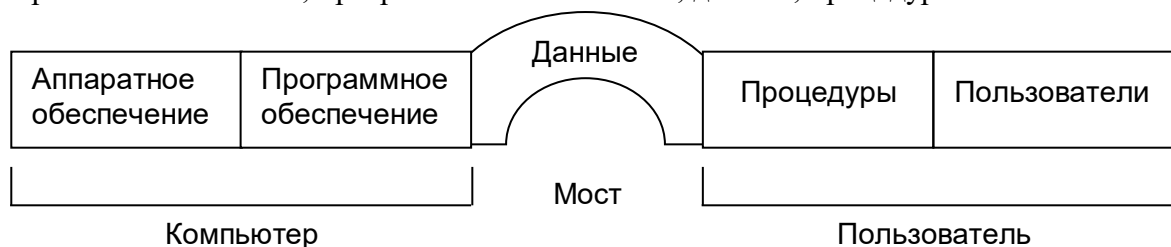


Рис. 1.2. Среда СУБД

Аппаратное обеспечение

Для работы СУБД и приложений необходимо аппаратное обеспечение, которое варьируется в очень широких пределах – от единственного персонального компьютера или одного мейнфрейма до сети из множества компьютеров. Используемое аппаратное обеспечение зависит от требований организации и используемой СУБД. Одни СУБД предназначены для работы только с конкретными типами операционных систем или оборудования, другие работают с широким кругом аппаратного обеспечения и различными операционными системами. Для работы СУБД обычно требуется некоторый минимум оперативной и дисковой памяти, но такой минимальной конфигурации может оказаться совершенно недостаточно для достижения приемлемой производительности системы. В настоящее время наиболее распространенной архитектурой является архитектура **клиент/сервер** (client-server), когда сервером является компьютер с **серверной частью** СУБД (backend), а клиентами – компьютеры с **клиентскими частями** СУБД (frontend).

Программное обеспечение

Этот компонент включает программное обеспечение самой СУБД и прикладных программ, операционную систему, сетевое программное обеспечение, если СУБД используется в сети. Приложения создаются на языках третьего поколения (С, COBOL, Fortran, Ada, Pascal) или на языках четвертого поколения (SQL, операторы которого внедряются в программы на языках третьего поколения). Однако СУБД может иметь свои собственные инструменты четвертого поколения, предназначенные для быстрой разработки приложений с использованием встроенных непроецедурных языков запросов, генераторов отчетов, форм, графических изображений и полномасштабных приложений. Использование инструментов четвертого поколения существенно повышает производительность системы и способствует созданию более удобных для обслуживания программ.

Данные

С точки зрения конечных пользователей самым важным компонентом среды СУБД являются данные. На рис. 1.2 показано, что данные являются связующим звеном между ком-

пьютером и пользователем. База данных содержит как рабочие данные, так и метаданные, т. е. «данные о данных». Структура базы данных называется **схемой** (schema). В системном каталоге базы данных содержатся следующие сведения: имена, типы и размеры элементов данных; имена связей; ограничения целостности данных; имена зарегистрированных пользователей, которым предоставлены определенные права доступа к данным; используемые индексы и структуры хранения.

Процедуры

К процедурам относятся инструкции и правила, которые должны учитываться при проектировании и использовании базы данных. Пользователям и обслуживающему персоналу базы данных необходимо предоставить документацию, содержащую подробное описание процедур использования и сопровождения данной системы, включая инструкции о правилах выполнения следующих действий:

- регистрация в СУБД;
- использование отдельного инструмента СУБД или приложения;
- запуск и останов СУБД;
- создание резервных копий СУБД;
- обработка сбоев аппаратного и программного обеспечения, включая процедуры идентификации вышедшего из строя компонента, исправления отказавшего компонента, а также восстановления базы данных после устранения неисправности;
- изменение структуры таблиц, реорганизация базы данных, размещенной на нескольких дисках, способы улучшения производительности и методы архивирования данных на вторичных устройствах хранения.

Пользователи

Распределение обязанностей в системах баз данных происходит между четырьмя различными группами пользователей: администраторы данных и баз данных; разработчики баз данных; прикладные программисты и конечные пользователи.

База данных и СУБД являются корпоративными ресурсами, управление ими предусматривает управление и контроль за СУБД и помещенными в нее данными. **Администратор данных** или АД (Data Administrator – DA) отвечает за управление данными, включая планирование базы данных, разработку и сопровождение стандартов, бизнес-правил и деловых процедур, а также за концептуальное и логическое проектирование базы данных. АД консультирует и дает свои рекомендации руководству высшего звена, контролируя соответствие общего направления развития базы данных установленным корпоративными целями. **Администраторы баз данных** или АБД (Database Administrator – DBA) отвечает за физическую реализацию базы данных, включая физическое проектирование и воплощение проекта, за обеспечение безопасности и целостности данных, за сопровождение операционной системы, а также за обеспечение максимальной производительности приложений и пользователей. По сравнению с АД обязанности АБД носят более технический характер, и для него необходимо знание конкретной СУБД и системного окружения. В одних организациях между этими ролями не делается различий, а в других выделяются отдельные группы персонала с указанием круга обязанностей.

В проектировании больших баз данных участвуют два разных типа разработчиков: разработчики логической базы данных и разработчики физической базы данных. **Разработчик логической базы данных** занимается идентификацией данных (сущностей и их атрибутов), связей между данными и устанавливает ограничения, накладываемые на хранимые данные. Разработчик логической базы данных должен обладать всесторонним и полным пониманием структуры данных организации и ее **бизнес-правил**. Бизнес-правила описывают основные характеристики данных с точки зрения организации. Для эффективной работы разработчик логической базы данных должен как можно раньше вовлечь всех предполагаемых пользователей базы данных в процесс создания модели данных. **Разработчик физической базы дан-**

ных получает готовую логическую модель данных, занимается ее физической реализацией, в том числе: преобразованием логической модели данных в набор таблиц и ограничений целостности данных; выбором конкретных структур хранения и методов доступа к данным, обеспечивающих необходимый уровень производительности при работе с базой данных; проектированием любых требуемых мер защиты.

Многие этапы физического проектирования базы данных в значительной степени зависят от выбранной целевой СУБД, и поэтому существуют различные способы воплощения требуемой схемы. Следовательно, разработчик физической базы данных должен разбираться в функциональных возможностях целевой СУБД и понимать достоинства и недостатки каждого возможного варианта воплощения. Разработчик физической базы данных должен уметь выбрать наиболее подходящую стратегию хранения данных с учетом всех существующих особенностей их использования. Если концептуальное и логическое проектирование базы данных отвечает на вопрос «что?», то физическое проектирование отвечает на вопрос «как?».

После создания базы данных следует приступить к разработке приложений, предоставляющих пользователям необходимые им функциональные возможности. Именно эту работу и выполняют **прикладные программисты**. Обычно прикладные программисты работают на основе спецификаций, созданных системными аналитиками. Как правило, каждая программа содержит некоторые операторы, требующие от СУБД выполнения определенных действий с базой данных (извлечение, вставка, обновление, удаление данных).

Конечные пользователи являются клиентами базы данных. Она проектируется, создается и поддерживается для того, чтобы обслуживать их информационные потребности. Конечных пользователей можно классифицировать по способу использования ими системы: **наивные пользователи**, иногда даже не подозревающие о наличии СУБД, инициирующие выполнение операций базы данных, вводя простейшие команды или выбирая команды меню, и **опытные пользователи**, которые знакомы со структурой базы данных, возможностями СУБД и могут создавать собственные прикладные программы.

1.3. История развития СУБД

Считается, что развитие СУБД началось в 60-е годы XX века, когда разрабатывался проект полета корабля Apollo на Луну. В то время не существовало никаких систем, способных обрабатывать и управлять тем огромным количеством данных, которое было необходимо для реализации этого проекта. В результате специалисты основного подрядчика – фирмы North American Aviation (теперь эта фирма называется Rockwell International) разработали программное обеспечение под названием **GUAM** (Generalized Update Access Method). Основная идея GUAM была построена на том, что малые компоненты объединяются вместе как части более крупных компонентов до тех пор, пока не будет собран воедино весь проект. Эта соответствующая инвертированному дереву структура называется **иерархической структурой** (hierarchical structure). В середине 60-х годов корпорация IBM присоединилась к фирме NAA для совместной работы над GUAM, в результате чего была создана система IMS (Information Management System). Причина, по которой корпорация IBM ограничила функциональные возможности IMS только управлением иерархиями записей, заключалась в том, что необходимо было обеспечить работу с устройствами хранения с последовательным доступом, а именно с магнитными лентами, которые были основным типом носителя в то время. Через некоторое время это ограничение удалось преодолеть. Несмотря на то, что IMS является самой первой из всех коммерческих СУБД, она до сих пор остается основной иерархической СУБД, используемой на большинстве крупных мейнфреймов.

Другим заметным достижением середины 60-х годов было появление системы IDS (Integrated Data Store) фирмы General Electric. Работу над ней возглавлял один из пионеров исследований в области систем управления базами данных – Чарльз Бачман (Charles Bachmann). Развитие этой системы привело к созданию нового типа систем управления базами данных – **сетевых** (network) СУБД, что оказало существенное влияние на информационные системы того поколения. Сетевая СУБД создавалась для представления более слож-

ных взаимосвязей между данными, чем те, которые можно было моделировать с помощью иерархических структур, а также для формирования стандарта баз данных. Для создания таких стандартов в 1965 году на конференции организации **CODASYL** (Conference on Data Systems Languages), проходившей при участии представителей правительства США и бизнесменов, была сформирована рабочая группа List Processing Task Force, переименованная в 1967 году в группу **Database Task Group** (DBTG). В компетенцию группы DBTG входило определение спецификаций среды, которая допускала бы разработку баз данных и управление данными. Предварительный вариант отчета этой группы был опубликован в 1969 году, а первый полный вариант – в 1971 году. Предложения группы DBTG содержали три компонента:

- сетевая **схема** – это логическая организация всей базы данных в целом (с точки зрения АБД), которая включает определение имени базы данных, типа каждой записи и компонентов записей каждого типа;
- **подсхема** – это часть базы данных, как она видится пользователям и приложениям;
- язык управления данными – инструмент для определения характеристик и структуры данных, а также для управления ими.

Группа DBTG также предложила стандартизировать три различных языка:

- язык определения данных (Data Definition Language – **DDL**) для схемы, который позволяет АБД описывать ее;
- язык определения данных (также **DDL**) для подсхемы, который позволяет определять в приложениях те части базы данных, доступ к которым будет необходим;
- язык манипулирования данными (Data Manipulation Language – **DML**), предназначенный для управления данными.

Несмотря на то, что этот отчет официально не был одобрен Национальным институтом стандартизации США (American National Standards Institute – ANSI), большое количество систем было разработано в полном соответствии с предложениями группы DBTG. Теперь они называются CODASYL-системами или DBTG-системами. CODASYL-системы и системы на основе иерархических подходов представляют собой СУБД первого поколения. Однако этим двум моделям присущи приведенные ниже недостатки:

- даже для выполнения простых запросов с использованием переходов и доступом к определенным записям необходимо создавать достаточно сложные программы;
- независимость от данных существует лишь в минимальной степени;
- отсутствие общепризнанных теоретических основ.

В 1970 году Э. Ф. Кодд (E. F. Codd), работавший в исследовательской лаборатории корпорации IBM, опубликовал очень важную статью о реляционной модели данных, позволявшей устранить недостатки прежних моделей. Вслед за этим появилось множество экспериментальных реляционных СУБД, а первые коммерческие продукты появились в конце 70-х – начале 80-х годов. Следует особенно отметить проект System R, разработанный в исследовательской лаборатории корпорации IBM (штат Калифорния). Данный проект был задуман с целью доказать практичность реляционной модели, что достигалось посредством реализации предусмотренных ею структур данных и требуемых функциональных возможностей. На основе этого проекта были получены важнейшие результаты:

- был разработан структурированный язык запросов SQL, который с тех пор стал стандартным языком реляционных СУБД;
- в 80-х годы были созданы различные коммерческие реляционные СУБД (DB2, SQL/DS корпорации IBM, Oracle корпорации Oracle Corporation).

В настоящее время существует несколько сотен различных реляционных СУБД для мейнфреймов и персональных компьютеров. В качестве примеров многопользовательских СУБД могут служить системы Informix фирмы Informix Software, Inc., Oracle фирмы Oracle Corporation, Sybase фирмы Sybase Corp., InterBase фирмы Microsoft и другие. Примерами реляционных СУБД для персональных компьютеров являются Access и Fox Pro фирмы Mi-

Microsoft, Paradox и Visual dBase фирмы Borland, R:Base фирмы Microgrm и другие. Реляционные СУБД относятся к СУБД **второго поколения**.

Однако реляционная модель также обладает некоторыми недостатками – в частности, ограниченными возможностями моделирования. Для решения этой проблемы был выполнен большой объем исследовательской работы. В 1976 году Чен предложил модель «сущность-связь» (Entity-Relationship model – ER-модель), которая в настоящее время стала самой распространенной технологией проектирования баз данных. В 1979 году Кодд сделал попытку устранить недостатки своей основополагающей работы и опубликовал расширенную версию реляционной модели – RM/T (1979), затем еще одну версию – RM/V2 (1990). Попытки создания модели данных, позволяющей более точно описывать реальный мир, нестрого называют **семантическим моделированием данных** (semantic data modelling).

В ответ на все возрастающую сложность приложений баз данных появились две новые системы: **объектно-ориентированные СУБД** или **ОО СУБД** (Object-Oriented DBMS – OODBMS) и **объектно-реляционные СУБД** или **ОР СУБД** (Object-Relational DBMS – ORDBMS). Однако в отличие от предыдущих моделей действительная структура этих моделей пока не совсем ясна. Попытки реализации подобных моделей представляют собой СУБД **третьего поколения**.

1.4. Преимущества и недостатки СУБД

СУБД обладает как многообещающими потенциальными преимуществами, так и недостатками. К преимуществам СУБД относятся:

- *контроль за избыточностью данных*. Если традиционные файловые системы неэкономно расходуют внешнюю память, сохраняя одни и те же данные в нескольких файлах, то при использовании базы данных, наоборот, предпринимается попытка исключить избыточность данных за счет интеграции файлов. Однако полностью избыточность информации в базах данных не исключается, а лишь контролируется степень ее избыточности. В одних случаях ключевые элементы данных необходимо дублировать для моделирования связей, в других случаях некоторые данные потребуется дублировать для повышения производительности системы;

- *непротиворечивость данных*. Устранение избыточности данных или контроль над ними позволяет сократить риск возникновения противоречивых состояний. Если элемент данных хранится в базе только в одном экземпляре, то для изменения его значения потребуется выполнить только одну операцию обновления, причем новое значение станет доступным сразу всем пользователям базы данных. Если же элемент данных с ведома системы хранится в базе данных в нескольких экземплярах, то такая система должна следить за тем, чтобы копии не противоречили друг другу. Однако во многих современных СУБД такой тип непротиворечивости данных автоматически не поддерживается;

- *совместное использование данных*. Файлы обычно принадлежат отдельным лицам или отделам, которые используют их в своей работе. В то же время база данных принадлежит всей организации в целом и может совместно использоваться всеми зарегистрированными пользователями. При такой организации работы большое количество пользователей может работать с большим объемом данных. Более того, можно создавать новые приложения на основе уже существующей в базе данных информации и добавлять в нее только те данные, которые еще не хранятся в ней, а не определять заново требования ко всем данным, необходимым новому приложению. Новые приложения также могут использовать такие предоставляемые типичными СУБД функциональные возможности, как определение структур данных и управление доступом к данным, организация параллельной обработки и обеспечение средств копирования/восстановления;

- *поддержка целостности данных*. Целостность данных означает корректность и непротиворечивость хранимых в ней данных. Целостность обычно описывается с помощью **ограничений**, т. е. правил поддержки непротиворечивости, которые не должны нарушаться в базе данных. Ограничения можно применять к элементам данных внутри одной записи или

к связям между записями. Интеграция данных позволяет АБД задавать требования по поддержке целостности данных, а СУБД – применять их;

- *повышенная безопасность*. Безопасность базы данных заключается в защите базы данных от несанкционированного доступа со стороны пользователей. Без привлечения соответствующих мер безопасности интегрированные данные становятся более уязвимыми, чем данные в файловой системе. Однако интеграция позволяет АБД определить требуемую систему безопасности базы данных, а СУБД – привести ее в действие. Система обеспечения безопасности может быть выражена в форме учетных имен и паролей для идентификации пользователей, которые зарегистрированы в базе данных. Доступ к данным со стороны зарегистрированного пользователя может быть ограничен только некоторыми операциями (извлечение, вставка, обновление, удаление данных);

- *применение стандартов*. Интеграция позволяет АБД определять и применять необходимые стандарты. Например, стандарты отдела и организации, государственные и международные стандарты могут регламентировать формат данных при обмене ими между системами, соглашения об именах, форму представления документации, процедуры обновления и правила доступа;

- *повышение доступности данных*. Данные по различным отделам в результате интеграции становятся непосредственно доступными конечным пользователям. Потенциально это повышает функциональность системы, что приводит к более качественному обслуживанию конечных пользователей или клиентов организации. Во многих СУБД предусмотрены языки запросов, инструменты для создания отчетов, которые позволяют пользователям задавать непредусмотренные заранее вопросы и почти немедленно получать требуемую информацию на своих терминалах, не прибегая к помощи программиста;

- *улучшение показателей производительности*. На базовом уровне СУБД обеспечивает все низкоуровневые процедуры работы с файлами, которые обычно выполняют приложения. Наличие этих процедур позволяет программисту сконцентрироваться на разработке более специальных, необходимых пользователям функций, не заботясь о подробностях их воплощения на более низком уровне. Во многих СУБД также предусмотрена среда разработки четвертого поколения с инструментами, упрощающими создание приложений баз данных. Результатом является повышение производительности работы программистов и сокращение времени разработки новых приложений с соответствующей экономией средств;

- *упрощение сопровождения системы за счет независимости от данных*. В файловых системах описания данных и логика доступа к данным встроены в каждое приложение, что делает программы зависимыми от данных. В СУБД реализован другой подход: описания данных отделены от приложений, поэтому приложения защищены от изменений в описаниях данных (принцип независимости от данных). Наличие независимости программ от данных значительно упрощает обслуживание и сопровождение работающих с базой данных;

- *улучшенное управление параллельность*. В некоторых файловых системах при одновременном доступе к одному и тому же файлу двух пользователей может возникнуть конфликт двух запросов, результатом которого будет потеря информации или утрата ее целостности. В свою очередь, во многих СУБД предусмотрена возможность параллельного доступа к базе данных и гарантируется отсутствие подобных проблем;

- *развитые службы резервного копирования и восстановления*. Ответственность за обеспечение защиты данных от сбоев аппаратного и программного обеспечения в файловых системах возлагается на пользователя (например, ежесуточное резервное копирование данных). При этом в случае сбоя может быть восстановлена резервная копия, но результаты работы, выполненной после резервного копирования, будут утрачены, и данную работу потребует выполнить заново. В современных СУБД предусмотрены средства сокращения объема потерь информации от возникновения различных сбоев.

Однако СУБД имеют и ряд недостатков:

- *сложность*. Обеспечение функциональности, которой должна обладать каждая хорошая СУБД, сопровождается значительным усложнением программного обеспечения

СУБД. Чтобы воспользоваться всеми преимуществами СУБД, проектировщики и разработчики баз данных, администраторы данных и администраторы баз данных, а также конечные пользователи должны хорошо понимать функциональные возможности СУБД. Непонимание принципов работы системы может привести к неудачным результатам проектирования;

- *размер*. Сложность и широта функциональных возможностей приводит к тому, что СУБД становится чрезвычайно сложным программным продуктом, который может занимать много места на диске и требовать большого объема оперативной памяти для эффективной работы;

- *стоимость СУБД*. В зависимости от имеющейся вычислительной среды и требуемых функциональных возможностей, стоимость СУБД может варьироваться в очень широких пределах. Например, однопользовательская СУБД для персонального компьютера может стоить около 100 фунтов стерлингов. Однако большая многопользовательская СУБД для мейнфрейма, обслуживающая сотни пользователей, может быть чрезвычайно дорогой: от 100 000 до 500 000 фунтов стерлингов. Кроме того, следует учитывать ежегодные расходы на сопровождение системы, которые составляют некоторый процент от ее общей стоимости;

- *дополнительные затраты на аппаратное обеспечение*. Для удовлетворения требований, предъявляемых к дисковым накопителям со стороны СУБД и базы данных, может понадобиться приобрести дополнительные устройства хранения информации. Более того, для достижения требуемой производительности может понадобиться более мощный компьютер, который, возможно, будет работать только с СУБД;

- *затраты на преобразование*. В некоторых ситуациях стоимость СУБД и дополнительного аппаратного обеспечения может оказаться несущественной по сравнению со стоимостью преобразования существующих приложений для работы с новой СУБД и новым аппаратным обеспечением. Эти затраты также включают стоимость подготовки персонала для работы с новой системой, а также оплату услуг специалистов, которые будут оказывать помощь в преобразовании и запуске новой системы. Эти факторы являются одними из основных причин, по которым некоторые организации остаются сторонниками прежних систем и не хотят переходить к более современным технологиям управления базами данных. Термин **традиционная система** иногда используется для обозначения устаревших и, как правило, не самых лучших систем;

- *производительность*. Обычно файловая система создается для некоторых специализированных приложений, например, для оформления счетов, и потому ее производительность может быть весьма высока. Однако СУБД предназначены для решения более общих задач и обслуживания сразу нескольких приложений. В результате многие приложения в новой среде будут работать не так быстро, как прежде;

- *более серьезные последствия при выходе системы из строя*. Централизация ресурсов повышает уязвимость системы. Поскольку работа всех пользователей и приложений зависит от готовности к работе СУБД, выход из строя одного из ее компонентов может привести к полному прекращению всей работы организации.

ГЛАВА 2. СРЕДА БАЗЫ ДАННЫХ

2.1. Трехуровневая архитектура ANSI/SPARC

Первая попытка создания стандартной терминологии и общей архитектуры СУБД была предпринята в 1971 году группой DBTG. Она была создана после конференции CODASYL (Conference on Data Systems and Languages – Конференция по языкам и системам баз данных). Группа DBTG признала необходимость использования двухуровневого подхода, построенного на основе использования системного представления или **схемы** (schema), и пользовательских представлений или **подсхем** (subschema). Подобные терминология и архитектура были предложены в 1975 году. Комитетом планирования стандартов и норм SPARC (Standards Planning and Requirements Committee) Национального института стандартизации США (American National Standard Institute – ANSI), ANSI/X3/SPARC. Однако комитет AN-

SI/SPARC признал необходимость использования трехуровневого подхода, реализующего необходимость поддержки независимого уровня изоляции программ от особенностей представления данных на более низком уровне. Несмотря на то, что модель ANSI/SPARC не стала стандартом, она представляет собой основу для понимания функциональных особенностей СУБД. Трехуровневая архитектура ANSI/SPARC имеет внешний, концептуальный и внутренний уровни, как показано на рис. 2.1. Цель трехуровневой архитектуры заключается в отделении пользовательского представления базы данных от ее физического представления, что обусловлено рядом причин:

- каждый пользователь должен иметь возможность обращаться к одним и тем же данным, используя свое представление о них, и изменять свое представление о данных, не оказывая при этом влияние на других пользователей;
- пользователи не должны знать особенности физического хранения данных в базе;
- АБД должен иметь возможность изменять структуру хранения данных в базе, не оказывая влияние на пользовательские представления;
- АБД должен иметь возможность изменять концептуальную или глобальную структуру базы данных без влияния на всех пользователей.

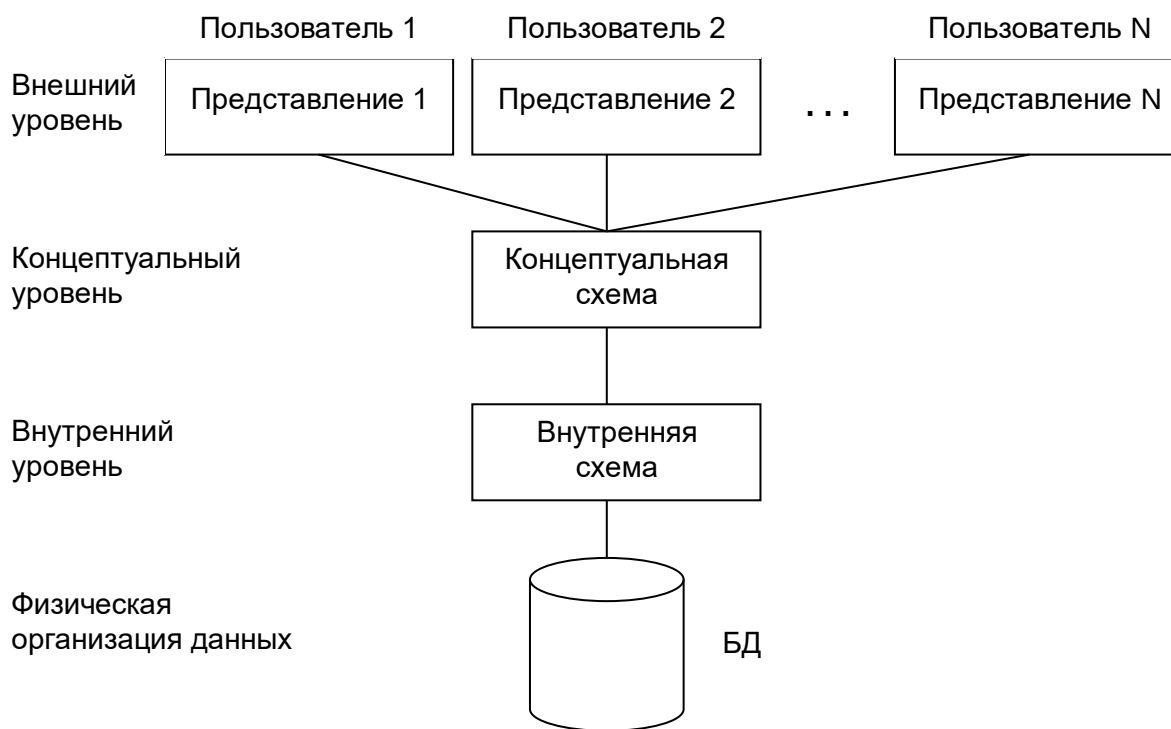


Рис. 2.1. Трехуровневая архитектура ANSI/SPARC

Уровень, на котором воспринимают данные конечные пользователи, называется **внешним уровнем** (external level). СУБД и операционная система воспринимают данные на **внутреннем уровне** (internal level). **Концептуальный уровень** (conceptual level) представления данных предназначен для отображения внешнего уровня на внутренний и обеспечения необходимой **независимости** друг от друга.

Определение. Внешний уровень – это представление базы данных с точки зрения пользователей. Этот уровень описывает ту часть базы данных, которая относится к каждому пользователю.

Внешний уровень состоит из нескольких различных внешних представлений базы данных. Каждый пользователь имеет дело с представлением «реального мира», выраженным в наиболее удобной для него форме. Пользователь может даже не подозревать о существова-

нии сущностей, атрибутов и связей «реального мира», необходимых для других пользователей.

Определение. Концептуальный уровень – это обобщающее представление базы данных. Этот уровень описывает то, *какие* данные хранятся в базе данных, а также связи, существующие между ними.

Промежуточным уровнем в трехуровневой архитектуре является концептуальный уровень. Этот уровень содержит логическую структуру всей базы данных (с точки зрения АБД). Фактически, это полное представление требований к данным со стороны организации, которое не зависит от способа их хранения. На концептуальном уровне представлены следующие компоненты:

- все сущности, атрибуты и связи;
- накладываемые на данные ограничения;
- семантическая информация о данных;
- информация о мерах обеспечения безопасности и поддержки целостности данных.

Описание сущности на концептуальном уровне должно содержать сведения о типах данных атрибутов (целочисленный, действительный или символьный) и их длине (количестве значащих цифр или максимальном количестве символов), но не должно включать сведений об организации хранения данных, например, об объеме занятого пространства в байтах.

Определение. Внутренний уровень – это физическое представление базы данных в компьютере. Этот уровень описывает, *как* информация хранится в базе данных.

Внутренний уровень описывает физическую реализацию базы данных и предназначен для достижения оптимальной производительности и обеспечения экономичности использования дискового пространства. Он содержит описание структур данных и организации отдельных файлов, используемых для хранения данных в запоминающих устройствах. На этом уровне осуществляется взаимодействие СУБД с методами доступа операционной системы с целью размещения данных на запоминающих устройствах, создания индексов, извлечения данных и т. д. На внутреннем уровне хранится следующая информация:

- распределение дискового пространства для хранения данных и индексов;
- описание подробностей сохранения записей (с указанием реальных размеров сохраняемых элементов данных);
- сведения о размещении записей;
- сведения о сжатии данных и выбранных методах их шифрования.

Ниже внутреннего уровня находится **физический уровень** (physical level), который контролируется операционной системой, но под руководством СУБД. Однако функции СУБД и операционной системы на физическом уровне не вполне четко разделены и могут варьироваться от системы к системе. В одних СУБД используются многие предусмотренные в данной операционной системе методы доступа, тогда как в других применяются только самые основные и реализована собственная файловая организация. Физический уровень доступа к данным ниже СУБД состоит только из известных операционной системе элементов (например, указателей, реализация последовательности распределения внутренних записей на диске).

Общее описание базы данных называется **схемой базы данных**. Существует три различных типа схем базы данных, которые определяются в соответствии с уровнями абстракции трехуровневой архитектуры. На самом высоком уровне имеется несколько **внешних схем** или **подсхем**, которые соответствуют различным представлениям данных конечных пользователей. На концептуальном уровне описание базы данных называют **концептуальной схемой**, а на самом низком уровне абстракции – **внутренней схемой**.

Концептуальная схема описывает все элементы данных и связи между ними с указанием необходимых ограничений поддержки целостности данных. Для каждой базы данных имеется только одна концептуальная схема. На нижнем уровне находится внутренняя схема, которая является полным описанием внутренней модели данных. Она содержит определения

хранимых записей, методы представления, описания полей данных, сведения об индексах и выбранных схемах хеширования. Для каждой базы данных существует только одна внутренняя схема.

СУБД отвечает за установление соответствия между этими тремя типами схем, а также за проверку их непротиворечивости. Иными словами СУБД должна убедиться в том, что каждую внешнюю схему можно вывести на основе концептуальной схемы. Для установления соответствия между любыми внешней и внутренней схемами СУБД должна использовать информацию из концептуальной схемы. Концептуальная схема связана с внутренней схемой посредством **концептуально внутреннего отображения**. Оно позволяет СУБД найти фактическую запись или набор записей на физическом устройстве хранения, которые образуют **логическую запись** в концептуальной схеме с учетом любых ограничений, установленных для выполняемых над данной логической записью операций. Оно также позволяет обнаружить любые различия в именах объектов, именах атрибутов, порядке следования атрибутов, их типах данных и т. д. Наконец, каждая внешняя схема связана с концептуальной схемой с помощью **внешнего концептуального отображения**. С его помощью СУБД может отображать имена пользовательского представления на соответствующую часть концептуальной схемы.

Важно различать описание базы данных и саму базу данных. Описанием базы является **схема базы данных**. Схема создается в процессе проектирования базы данных и изменяется достаточно редко. Однако содержащаяся в базе данных информация может меняться часто. Совокупность информации, хранящейся в базе данных в любой определенный момент времени, называется **состоянием базы данных**. Следовательно, одной и той же схеме базы данных может соответствовать множество ее различных состояний. Схема базы данных иногда называется **содержанием** базы данных, а ее состояние – **детализацией**.

Основным назначением трехуровневой архитектуры является обеспечение **независимости от данных**, которая означает, что изменения на нижних уровнях никак не влияют на верхние уровни. Различают два типа независимости от данных: **логическую** и **физическую**.

Определение. Логическая независимость от данных означает полную защищенность внешних схем от изменений, вносимых в концептуальную схему.

Такие изменения концептуальной схемы, как добавление или удаление новых сущностей, атрибутов или связей, должны осуществляться без необходимости внесения изменений в уже существующие внешние схемы или переписывания прикладных программ. При этом пользователи, для которых эти изменения предназначены, должны знать о них, но важно, чтобы другие пользователи о них не подозревали.

Определение. Физическая независимость от данных означает защищенность концептуальной схемы от изменений, вносимых во внутреннюю схему.

Такие изменения внутренней схемы, как использование различных файловых систем или структур хранения, разных устройств хранения, модификация индексов или хеширование, должны осуществляться без необходимости внесения изменений в концептуальную или внешнюю схемы. Пользователями могут быть замечены изменения только в общей производительности системы. На самом деле наиболее распространенной причиной внесения изменений во внутреннюю схему является именно недостаточная производительность выполнения операций.

Принятое в архитектуре ANSI/SPARC двухэтапное отображение может сказываться на эффективности работы, но при этом оно обеспечивает более высокую независимость от данных. Для повышения эффективности в модели ANSI/SPARC допускается использование прямого отображения внешних схем на внутренние без обращения к концептуальной схеме. Однако это снижает уровень независимости от данных, поскольку при каждом изменении внутренней схемы потребуется внесение определенных изменений во внешнюю схему и во все зависящие от нее прикладные программы.

2.2. Языки баз данных

Внутренний язык СУБД для работы с данными состоит из двух частей: **языка определения данных** (Data Definition Language – DDL) и **языка управления данными** (Data Manipulation Language – DML). Язык DDL используется для определения схемы базы данных, а язык DML – для чтения и обновления данных, хранимых в базе. Эти языки называются **подъязыками данных**, поскольку в них отсутствуют конструкции для выполнения всех вычислительных операций, обычно используемых в языках программирования высокого уровня. Во многих СУБД предусмотрена возможность внедрения операторов подъязыка данных в программы, написанные на языках программирования высокого уровня (COBOL, Fortran, Pascal, C, Ada). В этом случае язык высокого уровня принято называть **базовым языком** (host language). Перед компиляцией файла программы на базовом языке, содержащей внедренные операторы подъязыка данных, потребуется предварительно удалить эти операторы, заменив их вызовами соответствующих функций СУБД. Затем этот предварительно обработанный файл обычным образом компилируется в объектный модуль, который компонуется с библиотекой, содержащей вызываемые в программе функции СУБД. После этого полученный программный код будет готов к выполнению. Помимо механизма внедрения для большинства подъязыков данных также предоставляются средства интерактивного выполнения их операторов, вводимых пользователем непосредственно со своего терминала. Приведем краткую характеристику языков баз данных.

Определение. Язык DDL – это описательный язык, который позволяет АБД или пользователю описать и поименовать сущности, необходимые для работы некоторого приложения, а также связи между различными сущностями.

Схема базы данных состоит из набора определений, выраженных на специальном языке DDL, который используется как для определения новой схемы, так и для модификации уже существующей. Этот язык нельзя использовать для управления данными. Результатом компиляции DDL-операторов является набор таблиц, хранимый в особых файлах, называемых **системным каталогом**. В системном каталоге интегрированы **метаданные**, т. е. данные, которые описывают объекты базы данных, а также позволяют упростить способ доступа к ним и управления ими. Метаданные включают определения записей, элементов данных, а также другие объекты, представляющие интерес для пользователей или необходимые для работы СУБД. Перед доступом к реальным данным СУБД обычно обращается к системному каталогу. Для обозначения системного каталога также используются термины **словарь данных** (обычно относится к программному обеспечению более общего типа) и **каталог данных**. Теоретически для каждой схемы в трехуровневой архитектуре можно было бы выделить несколько различных языков DDL: язык DDL внешних схем, язык DDL концептуальной схемы и язык DDL внутренней схемы. Однако на практике существует один общий язык DDL, который позволяет задавать спецификации, как минимум, для внешней и внутренней схем.

Определение. Язык DML – это язык, содержащий набор операторов для поддержки основных операций манипулирования содержащимися в базе данными.

К операциям управления данными относятся следующие:

- вставка в базу данных новых сведений;
- модификация сведений, хранимых в базе данных;
- извлечение сведений, содержащихся в базе данных;
- удаление сведений из базы данных.

Таким образом, одна из основных функций СУБД заключается в поддержке языка манипулирования данными, с помощью которого пользователь может задавать выражения для выполнения перечисленных выше операций с данными. Понятие манипулирования данными применимо как к внешнему и концептуальному уровням, так и к внутреннему уровню. Однако на внутреннем уровне для этого необходимо определить очень сложные процедуры низкого уровня, которые позволяют выполнять доступ к данным весьма эффективно. На более высоких уровнях, наоборот, акцент переносится в сторону большей простоты использования и основные усилия направляются на обеспечение эффективного взаимодействия пользовате-

ля с системой. Языки DML отличаются базовыми конструкциями извлечения данных. Следует различать два типа языков DML: **процедурный** и **непроцедурный**. Основное отличие между ними заключается в том, что процедурные языки указывают то, *как* можно получить результат оператора языка DML, тогда как непроцедурные языки описывают то, *какой* результат будет получен. Как правило, в процедурных языках записи рассматриваются по отдельности, тогда как непроцедурные языки оперируют с наборами записей.

Определение. Процедурный язык DML – это язык, который позволяет сообщить системе о том, какие данные необходимы, и точно указать, как их можно извлечь.

С помощью процедурного языка DML программист указывает на то, какие данные ему необходимы и как их можно получить. Это значит, что программист должен определить все операции доступа к данным посредством вызова соответствующих процедур. Обычно процедурный язык DML позволяет извлечь запись, обработать ее и, в зависимости от полученных результатов, извлечь другую запись, которая должны быть подвергнута аналогичной обработке, и т. д. Подобный процесс извлечения данных продолжается до тех пор, пока не будут извлечены все запрашиваемые данные. Языки DML сетевых и иерархических СУБД обычно являются процедурными.

Определение. Непроцедурный язык DML – это язык, который позволяет указать лишь то, какие данные требуются, но не то, как их следует извлекать.

Непроцедурные языки DML позволяют определить весь набор требуемых данных с помощью одного оператора извлечения или обновления. С помощью непроцедурных языков DML пользователь указывает, какие данные ему нужны, не определяя способ их получения. СУБД транслирует выражение на языке DML в процедуру (набор процедур), которая обеспечивает манипулирование набором данных. Такой подход освобождает пользователя от необходимости знать детали внутренней реализации структур данных и особенности алгоритмов, используемых для извлечения и возможного преобразования данных. В результате работа пользователя получает определенную степень независимости от данных. Непроцедурные языки часто также называют *декларативными языками*. Реляционные СУБД обычно включают поддержку непроцедурных языков манипулирования данными – язык структурированных запросов SQL (Structured Query Language) или язык запросов по образцу QBE (Query by Example). Непроцедурные языки обычно проще в понимании и использовании, т. к. большая часть работы при этом выполняется СУБД, а не пользователем. Часть процедурного языка DML, которая отвечает за извлечение данных, называется *языком запросов*. Язык запросов можно определить как высокоуровневый узкоспециализированный язык, предназначенный для удовлетворения различных требований по выборке информации из базы данных.

В то время как языки третьего поколения являются процедурными, языки 4GL (Fourth-Generation Language) выступают как непроцедурные, поскольку пользователь определяет, *что* должно быть сделано, а не *как* именно желаемый результат должен быть достигнут. Реализация языков четвертого поколения в значительной мере основана на использовании компонентов высокого уровня, которые часто называют «инструментами четвертого поколения». Пользователю нет необходимости определять все этапы выполнения программы: достаточно определить необходимые параметры, на основании которых инструменты четвертого поколения автоматически осуществляют генерацию прикладного приложения. Ожидается, что языки четвертого поколения позволят повысить производительность работы на порядок, но за счет ограничения типов задач, которые можно будет решать с их помощью. Выделяют следующие типы языков четвертого поколения:

- языки представления информации, например, языки запросов или генераторы отчетов;
- специализированные языки, например, языки электронных таблиц;
- генераторы приложений, которые при создании приложений обеспечивают определение, вставку, обновление или извлечение сведений из базы данных;
- языки очень высокого уровня, предназначенные для генерации кода приложений.

2.3. Модели данных

Как упоминалось выше, схема создается с помощью некоторого языка определения данных для конкретной СУБД. Вследствие того, что это язык относительно низкого уровня, с его помощью трудно описать требования к данным в масштабе всей организации так, чтобы созданная схема была понятна пользователям разных категорий. Таким образом, необходимо описание схемы на более высоком уровне, называемом моделью данных.

Определение. Модель данных – это интегрированный набор понятий для описания данных, связей между ними и ограничений, накладываемых на данные в некоторой организации.

Модель является представлением «реального мира» объектов и событий, а также существующих между ними связей. При этом акцент делается на самых важных аспектах деятельности организации (можно сказать, что модель представляет саму организацию). Модель данных рассматривают как сочетание трех компонентов:

- структурная часть, т. е. набор правил, по которым создается база данных;
- управляющая часть, которая определяет типы допустимых операций с данными (операции обновления и извлечения данных, операции изменения структуры базы данных);
- набор ограничений поддержки целостности данных (необязательно), гарантирующих корректность используемых данных.

Цель построения модели данных заключается в представлении данных в понятном виде. Для архитектуры ANSI/SPARC можно идентифицировать следующие три связанные модели данных:

- внешнюю модель данных, отображающую представления каждого существующего в организации типа пользователей, которую иногда называют **предметной областью** (Universe of Discourse – UoD);
- концептуальную модель данных, отображающую логическое (или обобщенное) представление о данных, независимое от типа выбранной СУБД;
- внутреннюю модель данных, отображающую концептуальную схему определенным образом, понятным выбранной целевой СУБД.

Иерархическая модель

Определение. Под иерархической моделью данных понимается модель, объединяющая **записи**, хранимые в общей древовидной структуре с одним корневым типом записи, который имеет несколько подчиненных типов записи или не имеет совсем. Каждый подчиненный тип записи также может иметь несколько подчиненных типов или не иметь их совсем.

Основной структурой, поддерживающей иерархическое представление информации, является **дерево**. Для моделирования информации с помощью древовидной структуры зачастую используется обобщенное дерево (general tree), которое схематично изображено на рис. 2.3. На рисунке показано абстрактное представление данной структуры, состоящей из узлов (nodes), соединенных связями, которые называются дугами (arcs) или ребрами (edges). Самый верхний узел называется **корневым узлом** (root node). Он может иметь несколько **дочерних узлов** (child nodes) или не иметь их совсем. Дочерние узлы, в свою очередь, также могут иметь несколько дочерних узлов или не иметь их. В результате подобная структура может быть определена рекурсивно. Все узлы дерева, за исключением корня, должны иметь родительский узел. Любая часть дерева, исходящая из одного узла (помимо корня дерева), называется **поддеревом** (subtree). С практической точки зрения, каждый узел может быть представлен в виде некоторого **типа записи**, где каждая связь является встроенным **указателем** (или **адресом**), или с помощью некоторого физического упорядочения записей.

Узлы представляют объекты предметной области, а связи между ними определяются расположением узлов и ребер, которые, соединяя узлы, образуют эту древовидную структуру. Объекты могут иметь как одинаковый тип, так и являться разными сущностями. Иерархическая структура естественным образом поддерживает связи типа «один ко многим» (1: M)

и типа «один к одному» (1:1). Можно сделать вывод, что между двумя узлами может быть только одна связь, а связи в конкретной структуре могут иметь только определенное упорядочение. Однако на самом деле между узлами может быть задано сразу несколько ссылок, например, от родительского к дочернему узлу и обратно. Упорядочение указателей может быть таким, что все однотипные экземпляры могут быть связаны вместе. В основном на способ организации ссылок влияет характер алгоритма обработки, который должна поддерживать данная структура. В обобщенной структуре дерева типы записей обычно упорядочены внутри структуры, причем, как правило, слева направо. Экземпляры записей также обычно упорядочиваются, что повышает эффективность операций извлечения данных.

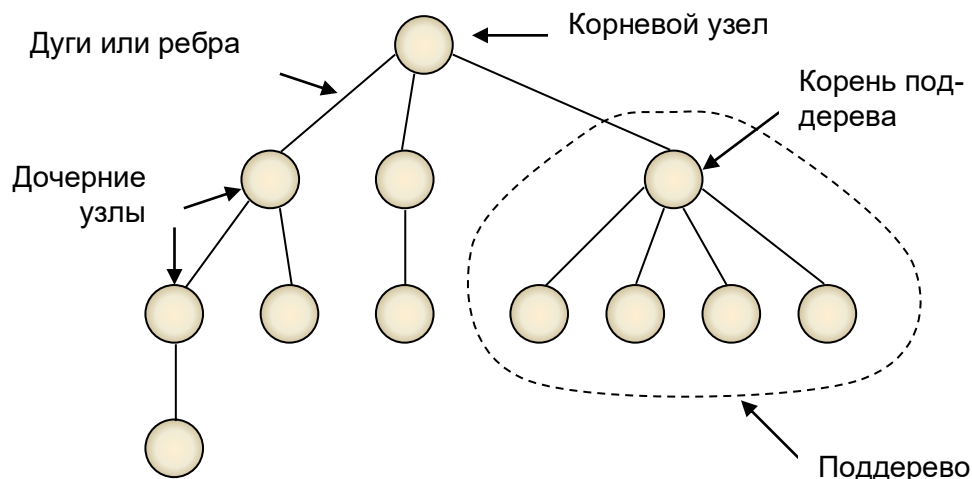


Рис. 2.3. Обобщенное представление древовидной структуры

При работе с обобщенной древовидной структурой используются два метода доступа ко всем узлам (типам записей) внутри дерева. Один метод начинается с доступа к корню с последующей обработкой всего дерева с доступом к поддеревьям в порядке слева направо. Это так называемый **прямой порядок обхода** дерева (pre-order traversal) или нисходящий порядок обхода. Другой метод начинается с доступа к самым нижним узлам с постепенным нисходящим переходом от одного поддерева к другому слева направо и с завершением обработки в корне. Этот метод называют **обратным порядком обхода** дерева (post-order traversal) или восходящим порядком обхода.

В информационных структурах чаще всего используется нисходящий порядок обхода, поскольку самые важные данные, как правило, располагаются на самых высоких уровнях древовидной структуры. Данные на более низких уровнях обычно логически связаны с данными на более высоких уровнях, а потому в случае необходимости они могут быть извлечены вместе с этими высокоуровневыми данными. Выборочный доступ к данным можно организовать путем изменения общего прямого порядка обхода с доступом только к интересующим нас данным, а не ко всей имеющейся информации. Набор всех экземпляров записей одного корня называется **единым экземпляром дерева**. В некоторых особых реализациях структуры данных единый экземпляр дерева эквивалентен записи в базе данных.

Важной особенностью иерархической структуры является то, что в ней дочерние экземпляры не могут существовать без наличия родительских экземпляров. Это вызвано присутствием в данной структуре такой встроенной зависимости, при которой при удалении корневого экземпляра удаляется все дерево (или поддерево). Такой подход обладает как преимуществами, так и недостатками. Одним из преимуществ является то, что целостность на уровне ссылок автоматически обеспечивается там, где экземпляры записей полностью зависят от других экземпляров записей. Недостатки такой структуры связаны с невозможностью хранения экземпляров записей, которые не имеют никаких родительских записей. Существо-

ет несколько методов разрешения этой проблемы (например, путем введения пустой подстановочной записи), но основное затруднение заключается в том, что подобная структура не позволяет достаточно просто моделировать характер экземпляров данных из реального мира.

Другим важным недостатком иерархической модели данных является трудность моделирования связей типа «многие ко многим» (M:N) или других, более сложных неиерархических связей, в которых тип записи имеет несколько разных связей с несколькими другими типами записей. Для хранения таких связей необходимо иметь иерархические структуры, которые дополняют основную структуру. Эти факторы приводят к необходимости дублирования данных и повышению уровня накладных расходов на их сопровождение.

Сетевая модель

Как правило, объекты предметной области, представленные набором древовидных структур, допускают связывания между поддеревьями, что неприемлемо с точки зрения иерархической модели. Этим обусловлено появление сетевой модели данных, когда каждый порожденный элемент может иметь более одного исходного и каждый элемент может быть связан с любым другим без каких-либо ограничений.

Определение. Под **сетевой моделью данных** понимается модель, состоящая из записей, элементов данных и связей типа «один ко многим» (1:M), установленных между записями. При этом связи типа «многие ко многим» (M:N) и рекурсивные связи поддерживаются с помощью декомпозиции.

При этом понятие «сетевая» не основано на использовании графов. Возможно, термин «сетевая» использовался разработчиками просто для подчеркивания того факта, что компоненты этой модели данных могут иметь установленные между ними связи. Сетевая база данных состоит из набора записей, соответствующих каждому экземпляру объекта предметной области, и набора связей между ними. Так например, информация об участии сотрудников в проектах организации может быть представлена в сетевой базе данных. В данном примере сетевая модель хорошо отражает то, что в проекте могут участвовать разные сотрудники и в то же время сотрудник может участвовать в различных проектах.

В качестве базовой физической структуры данных выступает сеть, в которой записи связаны друг с другом в один набор с помощью указателей. Записи могут содержать встроенные в них указатели. Под логической структурой данных понимается набор, в котором один тип записи-родителя может быть связан со многими типами записей-потомков. Сложные сети создаются с помощью типов наборов. Поддержка целостности на уровне ссылок для связей типа «родитель-потомок» обеспечивается средствами СУБД с использованием правил вставки и сохранения для структуры наборов. Если записи-потомки закреплены за записью-родителем, то удаление записи-родителя приводит к каскадному удалению записей-потомков. Если записи-потомки не являются частью набора, то удаление записи-родителя эквивалентно заданию неопределенной связи (NULL). Можно запрограммировать и другие варианты действий. Обычно внешние ключи в типах записей не требуются.

В качестве недостатков сетевой модели можно отметить следующие:

- если концептуальная схема должна быть изменена в соответствии с изменениями структуры хранения записей, то это может вызвать необходимость изменения прикладных программ. Следовательно, реализация макета базы данных может существенно усложниться;
- новые или измененные приложения могут не обладать достаточной производительностью, т. к. база данных структурирована для существовавших ранее приложений. Может оказаться невозможной эффективная поддержка всех требуемых приложений;
- при проектировании базы данных потребуется квалифицированная помощь, особенно при определении необходимых структур данных и при разработке связанных с ними прикладных программ.

Реляционная модель

В реляционных базах данных вся информация представляется в виде двумерных таблиц. С ее созданием начинается новый этап в эволюции СУБД. Простота и гибкость модели привлекли к ней внимание разработчиков и снискали ей множество сторонников. Несмотря на некоторые недостатки, реляционная модель данных стала доминирующей, а реляционные СУБД стали промышленным стандартом «де-факто».

Реляционная модель опирается на систему понятий реляционной алгебры, важнейшими из которых являются «таблица», «отношение», «строка», «первичный ключ». Все операции над реляционной базой данных сводятся к манипуляциям с таблицами. Таблица состоит из строк и столбцов и имеет имя, уникальное внутри базы данных. Таблица отражает тип объекта реального мира (сущность), а каждая ее строка (кортеж) – конкретный объект (рис. 2.4). Например, таблица «Сотрудники отдела» содержит сведения обо всех сотрудниках отдела, каждая ее строка – набор значений атрибутов конкретного сотрудника. Значения конкретного атрибута выбираются из домена (domain) – множества всех возможных значений атрибута объекта. Имя столбца должно быть уникальным в таблице. Столбцы расположены в таблице в соответствии с порядком следования их имен при ее создании. Любая таблица должна иметь по крайней мере один столбец. В отличие от столбцов строки не имеют имен. Порядок их следования в таблице не определен, а количество логически не ограничено.

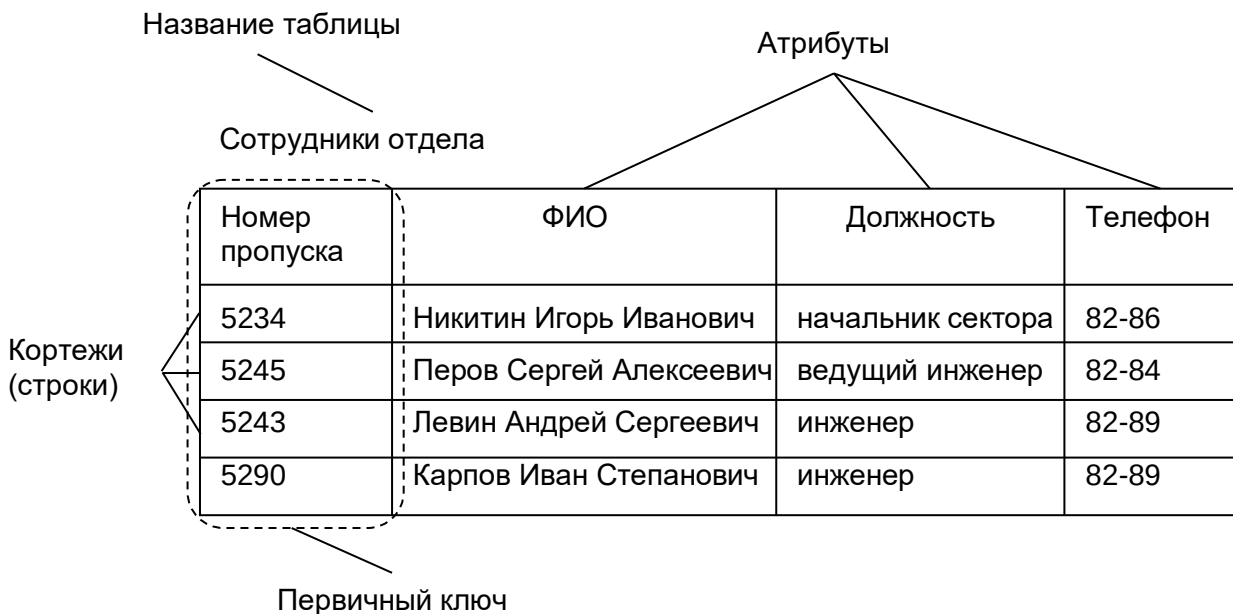


Рис. 2.4. Отношение реляционной базы данных

Любая таблица имеет один или несколько столбцов, значения в которых однозначно идентифицируют каждую ее строку. Такой столбец (или набор столбцов) называется первичным ключом. В таблице «Сотрудники отдела» первичным ключом служит столбец «Номер пропуска». В таблице не должно быть строк, имеющих одно и то же значение первичного ключа. Если таблица удовлетворяет этому требованию, она называется отношением.

Взаимосвязь таблиц в реляционной модели поддерживается внешними ключами. Внешний ключ – это столбец, значения которого однозначно характеризуют сущности, представленные строками некоторого другого отношения, т. е. задают значения их первичного ключа. Говорят, что отношение, в котором определен внешний ключ, ссылается на соответствующее отношение, в котором такой же атрибут является первичным ключом.

Таблицы невозможно хранить и обрабатывать, если в базе данных отсутствуют «данные о данных» (метаданные), например, описатели таблиц, столбцов и т. д. Метаданные также представлены в табличной форме и хранятся в словаре данных. Помимо таблиц в базе

данных могут храниться и другие объекты, такие как экранные формы, шаблоны отчетов и прикладные программы, работающие с информацией базы данных.

Реляционная модель поддерживает связи типа «один к одному» и «один ко многим». Связи типа «многие ко многим» и рекурсивные связи поддерживаются с помощью декомпозиции.

Для пользователей информационной системы важно, чтобы база данных отражала предметную область однозначно и непротиворечиво. Если она обладает такими свойствами, то говорят, что база данных удовлетворяет условию целостности. Чтобы добиться выполнения условия целостности, на базу данных накладываются некоторые ограничения, которые называются ограничениями целостности. *Выделяют два основных типа ограничений целостности: целостность сущностей и целостность ссылок.* Ограничение первого типа состоит в том, что любой кортеж отношения должен быть отличим от любого другого его кортежа, т. е. другими словами, любое отношение должно обладать первичным ключом. Это требование удовлетворяется автоматически, если в системе не нарушаются базовые свойства отношений. Ограничение целостности по ссылкам заключается в том, что внешний ключ не может быть указателем на несуществующую строку в таблице. Поддержка целостности на уровне ссылок варьируется от разработки процедур, правил или триггеров, используемых непосредственно средствами СУБД, до процедур, реализованных в прикладных программах. Стандарт SQL-92 требует реализации этой функции механизмами самой СУБД.

Реляционная модель обладает гибкостью по отношению к изменению требований к данным и методам доступа. Настройка для работы с новыми или измененными приложениями может быть выполнена без особых затруднений. В СУБД, поддерживающей выбор структуры файлов, для разных таблиц могут быть выбраны самые подходящие структуры данных. Доступ к типам записей осуществляется с помощью команд реляционной алгебры или реляционного исчисления. Эти команды могут быть вложенными, что позволяет создавать сложные запросы.

Большинство пользователей, включая конечных пользователей, легко понимают логическую структуру данных, и поэтому в состоянии спроектировать базу данных и отобразить ее на физические структуры. К сожалению, недостаточное знание методов проектирования баз данных может привести к созданию слабых, неэффективных и негибких макетов.

Доминирование реляционной модели в современных СУБД обусловлено рядом причин, в числе которых:

- наличие развитой теории реляционной модели данных, которая поддерживается теоретическими исследованиями в большей степени по сравнению с другими моделями;
- наличие аппарата приведения к реляционной других моделей данных;
- поддержка реляционной моделью специальных средств ускоренного доступа к информации;
- возможность манипулирования данными без необходимости знания конкретной физической организации базы данных во внешней памяти;
- наличие стандартизованного высокоуровневого языка запросов к базе данных.

Иногда дополнительно к иерархической, сетевой и реляционной моделям данных выделяют модель данных на основе инвертированных списков. Базы данных, организованные с помощью инвертированных списков, похожи на реляционные базы данных. Отличия заключаются в следующем:

- хранимые таблицы и индексы для ключей поиска напрямую доступны пользователям;
- строки таблиц упорядочены системой в некоторой физической последовательности.

2.4. Функции СУБД

Приведем краткое описание функций СУБД, которые должны быть реализованы в любой полномасштабной СУБД:

1. *Хранение, извлечение и обновление данных.* СУБД должна предоставлять пользователям возможность сохранять, извлекать и обновлять данные в базе данных. Это самая фунда-

ментальная функция СУБД. Причем, способ реализации этой функции должен быть скрыт от конечного пользователя.

2. *Каталог, доступный конечным пользователям.* СУБД должна иметь доступный конечным пользователям каталог, в котором хранится описание элементов данных. Ключевой особенностью архитектуры ANSI/SPARC является наличие интегрированного **системного каталога** с данными о схемах, пользователях, приложениях и т. д. Предполагается, что каталог доступен как пользователям, так и функциям СУБД. Системный каталог или словарь данных является хранилищем информации, описывающей данные в базе данных (метаданные). В зависимости от типа используемой СУБД количество информации и способ ее применения могут варьироваться. Обычно в системном каталоге хранятся следующие сведения:

- имена, типы и размеры элементов данных;
- имена связей;
- накладываемые на данные ограничения поддержки целостности;
- имена санкционированных пользователей;
- внешняя, концептуальная и внутренняя схемы и отображения между ними;
- статистические данные (частота транзакций, счетчик обращения к объектам базы данных).

Системный каталог позволяет достичь определенных преимуществ:

- информация о данных может быть централизованно собрана и сохранена, что позволит контролировать доступ к этим данным, как и к любому другому ресурсу;
- можно определить смысл данных, что поможет другим пользователям понять их назначение;
- упрощается сообщение, так как сохраняются точные определения смысла данных. В системном каталоге также могут быть указаны один или несколько пользователей, которые являются владельцами данных или обладают правом доступа к ним;
- благодаря централизованному хранению избыточность и противоречивость описания отдельных элементов данных могут быть легко обнаружены;
- внесенные в базу данных изменения могут быть зафиксированы;
- последствия любых изменений могут быть определены еще до их внесения, поскольку в системном каталоге зафиксированы все существующие элементы данных, установленные между ними связи, а также все их пользователи;
- меры обеспечения безопасности могут быть дополнительно усилены;
- появляются новые возможности организации поддержки целостности данных;
- может выполняться аудит сохраняемой информации.

3. *Поддержка транзакций.* СУБД должна иметь механизм, который гарантирует выполнение либо всех операций обновления данной транзакции, либо ни одной из них. Транзакция представляет собой набор действий, выполняемых отдельным пользователем или прикладной программой с целью доступа или изменения содержимого базы данных (например, удаление сведений о сотруднике из базы данных и передача ответственности за всю курируемую им работу другому сотруднику). Если во время выполнения транзакции произойдет сбой, например, из-за выхода из строя компьютера, база данных попадет в противоречивое состояние, поскольку некоторые изменения уже будут внесены, а остальные – нет. В этом случае все частичные изменения должны быть отменены для возвращения базы данных в исходное непротиворечивое состояние.

4. *Сервисы управления параллельностью.* СУБД должны иметь механизм, который гарантирует корректное обновление базы данных при параллельном выполнении операций обновления многими пользователями. Одна из основных целей создания и использования СУБД заключается в том, чтобы множество пользователей могло осуществлять доступ к совместно обрабатываемым данным. Параллельный доступ сравнительно просто организовать, если все пользователи выполняют только чтение данных. Конфликтные ситуации с нежелательными последствиями легко могут возникнуть, когда два и более пользователей пы-

таются обновить данные. СУБД должна гарантировать, что при одновременном доступе к базе данных многих пользователей таких конфликтов не произойдет.

5. *Сервисы восстановления.* СУБД должна предоставлять средства восстановления базы данных на случай какого-либо ее повреждения или разрушения. Сбой может произойти в результате выхода из строя системы или запоминающего устройства, возможны ошибки аппаратного и программного обеспечения, которые могут привести к останову СУБД. К тому же пользователь может потребовать отмены операции. Во всех подобных случаях СУБД должна предоставить механизм восстановления базы данных и возврата к ее непротиворечивому состоянию. Сервисы восстановления тесно связаны с управлением транзакциями.

6. *Сервисы контроля доступа к данным.* СУБД должна иметь механизм, гарантирующий возможность только санкционированного доступа к базе данных. Иногда требуется скрыть некоторые хранимые в базе данных сведения от других пользователей. Например, менеджерам отделений компании можно было бы предоставить всю информацию, связанную с зарплатой сотрудников, но желательно скрыть ее от других пользователей. Кроме того, базу данных следует защитить от любого несанкционированного доступа. Термин **безопасность** относится к защите базы данных от преднамеренного или случайного несанкционированного доступа.

7. *Поддержка обмена данными.* СУБД должна обладать способностью к интеграции с коммуникационным программным обеспечением. СУБД для персональных компьютеров должны поддерживать работу в локальной сети, чтобы вместо нескольких баз данных для каждого пользователя можно было установить одну централизованную базу данных и использовать ее как общий ресурс для всех пользователей. При этом предполагается, что не база данных должна быть распределена в сети, а удаленные пользователи должны иметь возможность доступа к централизованной базе данных. Такая топология называется **распределенной обработкой**.

8. *Службы поддержки целостности данных.* СУБД должна обладать инструментами контроля за тем, чтобы данные и их изменения соответствовали заданным правилам. Целостность базы данных означает корректность и непротиворечивость хранимых данных. Она может рассматриваться как еще один тип защиты базы данных, но в более широком смысле целостность связана с качеством самих данных. Целостность обычно выражается в виде ограничений или правил сохранения непротиворечивости данных (например, сотрудник не имеет права работать больше, чем на полторы ставки в данной организации).

9. *Службы поддержки независимости от данных.* СУБД должна обладать инструментами поддержки независимости программ от фактической структуры базы данных. Обычно она достигается за счет реализации механизма поддержки представлений или подсхем. Как правило, система легко адаптируется к добавлению нового объекта, атрибута или связи, но не к их удалению. В некоторых системах вообще запрещается вносить любые изменения в уже существующие компоненты логической схемы.

10. *Вспомогательные службы.* СУБД должна предоставлять некоторый набор различных вспомогательных служб. Вспомогательные утилиты обычно предназначены для оказания помощи АБД в эффективном администрировании базы данных. Приведем примеры таких утилит:

- утилиты импортирования, предназначенные для загрузки данных из плоских файлов или других СУБД, а также утилиты экспортирования, которые служат для выгрузки базы данных в плоские файлы или другие СУБД;
- средства мониторинга, предназначенные для отслеживания характеристик функционирования и использования базы данных;
- программы статистического анализа, позволяющие оценить производительность или степень использования базы данных;
- инструменты реорганизации индексов, предназначенные для перестройки индексов и обработки случаев их переполнения;

– инструменты сборки мусора и перераспределения памяти для физического устранения удаленных записей с запоминающих устройств, объединения освобожденного пространства и перераспределения памяти в случае необходимости.

2.5. Компоненты СУБД

Структуру компонентов СУБД практически невозможно обобщить, поскольку она сильно различается в разных системах.

СУБД состоит из нескольких программных компонентов (модулей), каждый из которых предназначен для выполнения специфической операции. При этом некоторые функции СУБД могут поддерживаться операционной системой, предоставляющей базовые службы, а сама СУБД всегда представляет надстройку над ними. Таким образом, при проектировании СУБД следует учитывать особенности интерфейса между создаваемой СУБД и конкретной операционной системой.

Основные программные компоненты среды СУБД представлены на рис.

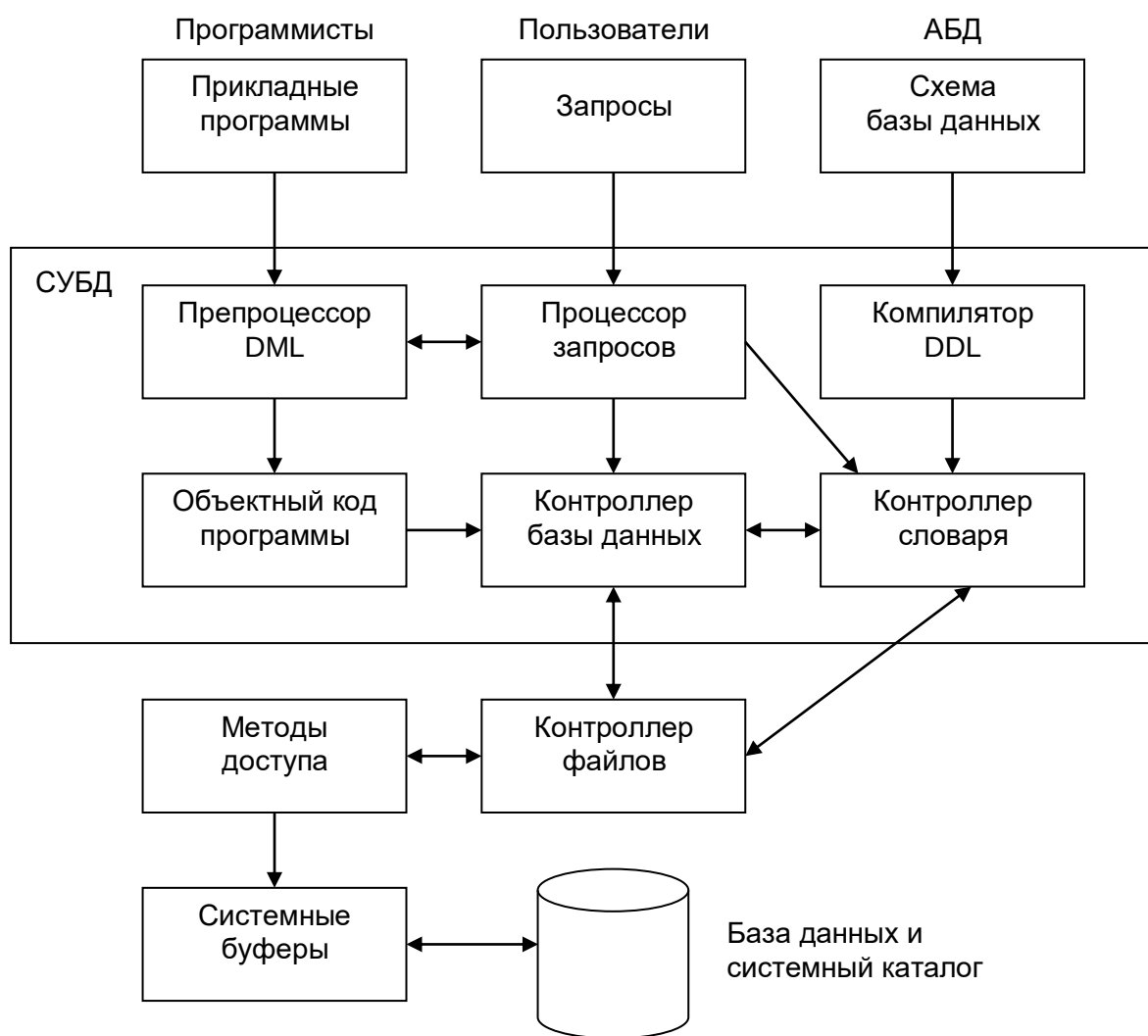


Рис. 2.5. Основные компоненты типичной системы управления базами данных

Приведем краткую характеристику программных компонентов среды СУБД:

- *процессор запросов*. Это основной компонент СУБД, который преобразует запросы в последовательность низкоуровневых инструкций для контроллера базы данных;
- *контроллер базы данных*. Этот компонент взаимодействует с запущенными пользователями прикладными программами и запросами. Контроллер базы данных принимает запро-

сы и проверяет внешние и концептуальные схемы для определения тех концептуальных записей, которые необходимы для удовлетворения требований запросов.

– *контроллер файлов*. Манипулирует предназначенными для хранения данных файлами и отвечает за распределение доступного дискового пространства. Создает и поддерживает список структур и индексов, определенных во внутренней схеме. Если используются хешированные файлы, то в его обязанности входит и вызов функций хеширования для генерации адресов записей;

– *препроцессор языка DML*. Этот модуль преобразует внедренные в прикладные программы DML-операторы в вызовы стандартных процедур базового языка. Для генерации соответствующего кода препроцессор языка DML должен взаимодействовать с процессором запросов;

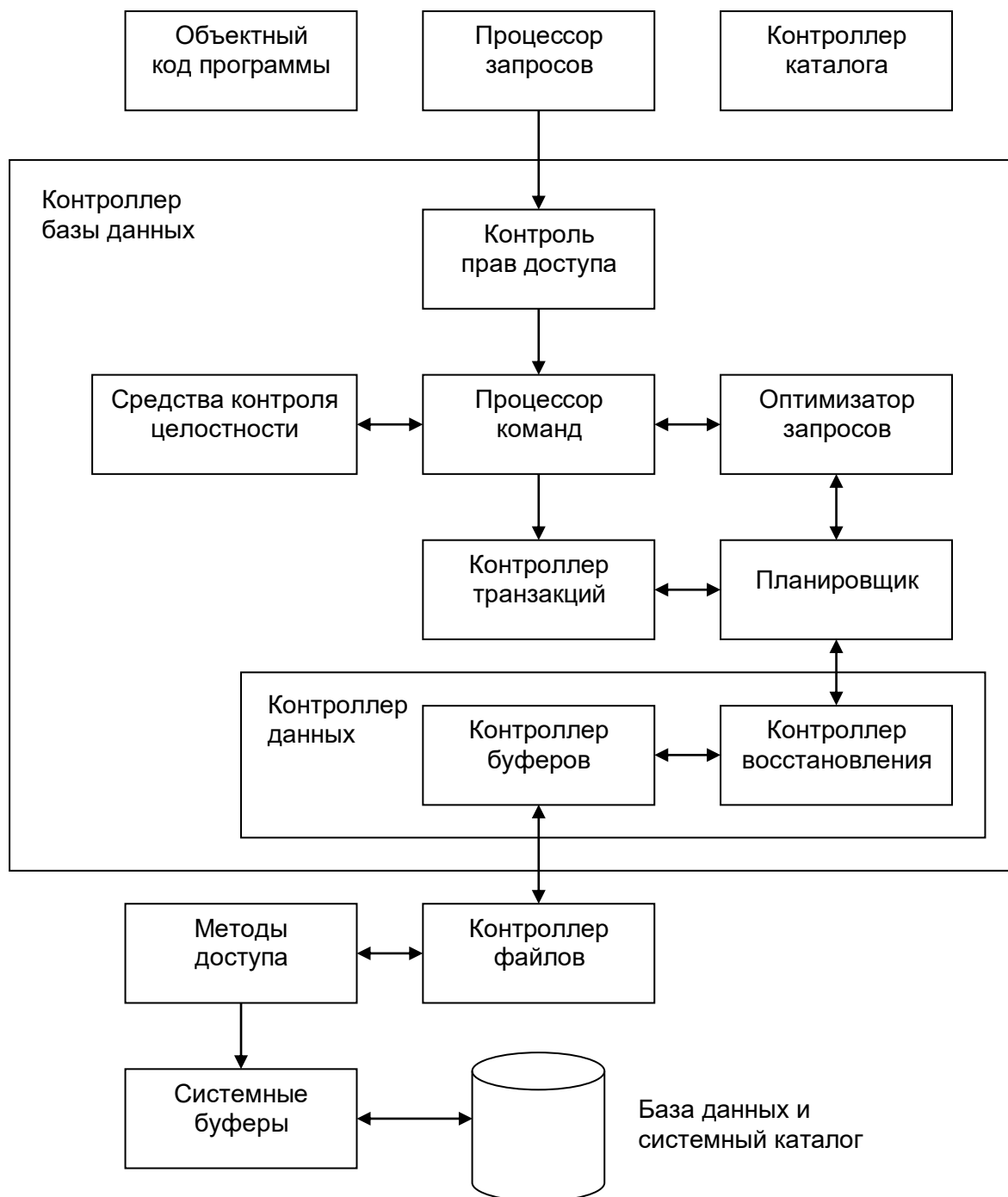


Рис. 2.6. Компоненты контроллера базы данных

– *компилятор языка DDL*. Компилятор языка DDL преобразует DDL-команды в набор таблиц, содержащих метаданные. Затем эти таблицы сохраняются в системном каталоге, а управляющая информация – в заголовках файлов с данными;

– *контроллер словаря*. Контроллер словаря управляет доступом к системному каталогу и обеспечивает работу с ним. Системный каталог доступен большинству компонентов СУБД.

Кратко рассмотрим основные программные компоненты, входящие в состав контроллера базы данных:

– *контроль прав доступа*. Этот модуль проверяет наличие у данного пользователя полномочий для выполнения затребованной операции;

– *процессор команд*. После проверки полномочий пользователя для выполнения затребованной операции управление передается процессору команд;

– *средства контроля целостности*. В случае операций, которые изменяют содержимое базы данных, средства контроля целостности выполняют проверку того, удовлетворяет ли затребованная операция всем установленным ограничениям поддержки целостности данных;

– *оптимизатор запросов*. Этот модуль определяет оптимальную стратегию выполнения запросов;

– *контроллер транзакций*. Этот модуль осуществляет требуемую обработку операций с базой данных. Он управляет относительным порядком выполнения операций, затребованных в отдельных транзакциях;

– *планировщик*. Этот модуль отвечает за бесконфликтное выполнение параллельных операций с базой данных. Он управляет относительным порядком выполнения операций, затребованных в отдельных транзакциях;

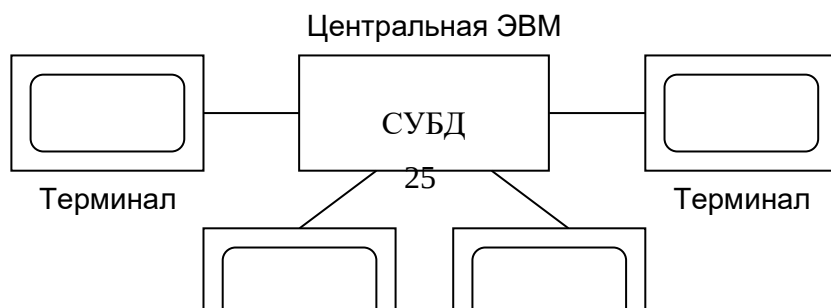
– *контроллер восстановления*. Этот модуль гарантирует восстановление базы данных до непротиворечивого состояния при возникновении сбоев. В частности, он отвечает за фиксацию и отмену результатов выполнения транзакций;

– *контроллер буферов*. Этот модуль отвечает за перенос данных между оперативной памятью и вторичным запоминающим устройством (жестким диском, магнитной лентой). Контроллер восстановления и контроллер буферов иногда называют *контроллером данных*.

2.6. Архитектура многопользовательских СУБД

Телеобработка

Традиционной архитектурой многопользовательских систем раньше считалась схема, получившая название «телеобработка», при которой один из компьютеров с единственным процессором был соединен с несколькими терминалами так, как показано на рис. 2.7. При этом вся обработка выполнялась в рамках единственного компьютера, а присоединенные к нему пользовательские терминалы были типичными «неинтеллектуальными» устройствами, не способными функционировать самостоятельно. С центрального процессора терминалы были связаны с помощью кабелей, по которым они посылали сообщения пользовательским приложениям (через подсистему управления обменом данными операционной системы). В свою очередь, пользовательские приложения обращались к необходимым службам СУБД. Таким же образом сообщения возвращались назад на пользовательский терминал. К сожалению, при такой архитектуре основная и чрезвычайно большая нагрузка возлагалась на центральный компьютер, который должен был выполнять не только действия прикладных программ и СУБД, но и значительную работу по обслуживанию терминалов (например, форматирование данных, выводимых на экран терминала).



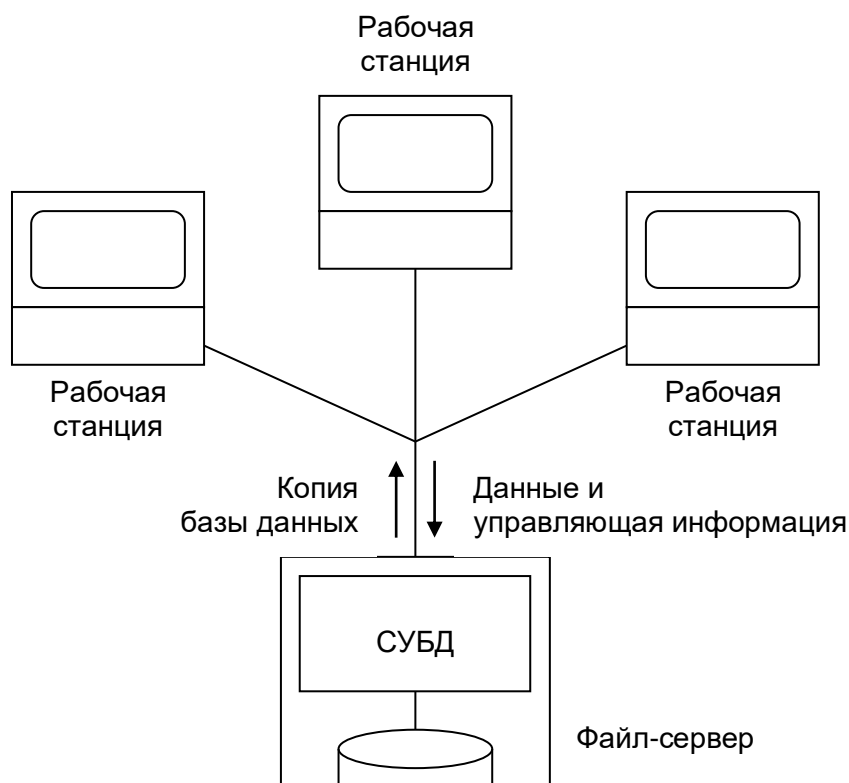
В последние годы был достигнут существенный прогресс в разработке высокопроизводительных персональных компьютеров и составленных из них сетей. При этом во всей индустрии наблюдается заметная тенденция к **децентрализации** (downsizing), т. е. замене дорогих мейнфреймов более эффективными с точки зрения эксплуатационных затрат, сетями персональных компьютеров, позволяющими получить аналогичные и даже лучшие результаты. Эта технология привела к появлению двух типов архитектуры СУБД: технологии файлового сервера и технологии клиент/сервер.

Файловый сервер

В среде файлового сервера обработка данных распределена в сети, обычно представляющей собой локальную вычислительную сеть (ЛВС). Файловый сервер содержит файлы, необходимые для работы приложений и самой СУБД. Однако пользовательские приложения и сама СУБД размещены и функционируют на отдельных рабочих станциях, и обращаются к файловому серверу только по мере необходимости получения доступа к нужным им файлам (рис. 2.8). Таким образом, файловый сервер функционирует просто как совместно используемый жесткий диск. СУБД на каждой рабочей станции посылает запросы файловому серверу по всем необходимым ей данным, которые хранятся на диске файл-сервера. Такой подход характеризуется значительным сетевым трафиком, что может привести к снижению производительности всей системы в целом.

Таким образом, архитектура с использованием файлового сервера обладает следующими основными недостатками:

- большой объем сетевого трафика;
- на каждой рабочей станции должна находиться полная копия СУБД;
- управление параллельностью, восстановлением и целостностью усложняется, поскольку доступ к одним и тем же файлам могут осуществлять сразу несколько экземпляров СУБД.



Технология клиент/сервер

Технология клиент/сервер разработана с целью устранения недостатков, имеющих в первых двух подходах. Термин «клиент/сервер» означает такой способ взаимодействия программных компонентов, при котором они образуют единую систему: существует **клиентский процесс**, требующий определенных ресурсов, а также **серверный процесс**, который эти ресурсы предоставляет. Как правило, принято размещать сервер на одном узле локальной сети, а клиентов – на других узлах. На рис. 2.9 показана общая архитектура типа клиент/сервер, хотя возможны и альтернативные варианты топологии: один клиент – один сервер; несколько клиентов – один сервер; несколько клиентов – несколько серверов.

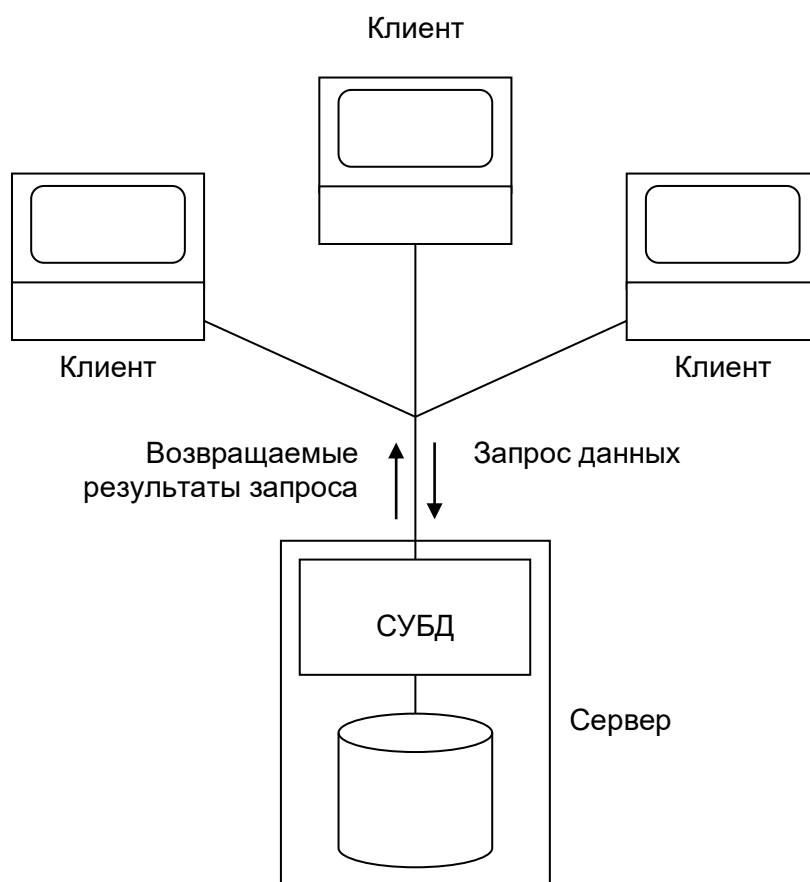


Рис. 2.9. Общая схема построения систем с архитектурой «клиент/сервер»

В контексте базы данных клиент управляет пользовательским интерфейсом и логикой приложения, действуя как сложная рабочая станция, на которой выполняются приложения баз данных. Клиент принимает от пользователя запрос, проверяет синтаксис и генерирует запрос к базе данных на языке SQL или другом языке базы данных, который соответствует

логике приложения. Затем он передает сообщение серверу, ожидает поступления ответа и формирует полученные данные для представления их пользователю. Сервер принимает и обрабатывает запросы к базе данных, а затем передает полученные результаты обратно клиенту. Такая обработка включает проверку полномочий клиента, обеспечение требований целостности, поддержку системного каталога, а также выполнение запроса и обновление данных. Помимо этого, поддерживается управление параллельностью и восстановлением. Выполняемые клиентом и сервером операции приведены в табл. 2.1.

Таблица 2.1

**Функции, выполняемые участниками взаимодействия в среде
клиент/сервер**

Клиент	Сервер
Управляет пользовательским интерфейсом	Принимает и обрабатывает запросы к базе данных со стороны клиента
Принимает и проверяет синтаксис введенного пользователем запроса	Проверяет полномочия пользователей
Выполняет приложение	Гарантирует соблюдение ограничений целостности
Генерирует запрос к базе данных и передает его серверу	Выполняет запросы/обновления и возвращает результаты клиенту
Отображает полученные данные пользователю	Поддерживает системный каталог
	Обеспечивает параллельный доступ к базе данных
	Обеспечивает управление восстановлением

Такой тип архитектуры обладает приводимыми ниже преимуществами:

- обеспечивается более широкий доступ к существующим базам данных;
- повышается общая производительность системы. Поскольку клиенты и сервер находятся на разных компьютерах, их процессоры способны выполнять приложения параллельно. При этом настройка производительности компьютера с сервером упрощается, если на нем выполняется только работа с базой данных;
- стоимость аппаратного обеспечения снижается. Достаточно мощный компьютер с большим устройством хранения необходим только серверу;
- сокращаются коммуникационные расходы. Приложения выполняют часть операций на клиентских компьютерах и посылают через сеть только запросы к базе данных, что позволяет существенно сократить объем пересылаемых по сети данных;
- повышается уровень непротиворечивости данных. Сервер может самостоятельно управлять проверкой целостности данных, поскольку все ограничения определяются и проверяются только в одном месте. При этом каждому приложению не придется выполнять собственную проверку;
- такая архитектура естественным образом отображается на архитектуру открытых систем.

Некоторые разработчики баз данных использовали архитектуру клиент/сервер для организации средств работы с распределенными базами данных, т. е. набором нескольких баз данных, логически связанных и распределенных в компьютерной сети. Однако несмотря на то, что архитектура клиент/сервер вполне может быть использована для организации распределенной СУБД, сама по себе она не образует распределенную СУБД.

Дальнейшим расширением двухуровневой архитектуры клиент/сервер является трехуровневая архитектура, при которой функциональная часть прежнего *толстого* (интеллектуального) клиента разделяется на две части: *тонкий* (неинтеллектуальный) клиент на рабочей станции управляет только пользовательским интерфейсом, тогда как средний уровень обработки данных управляет всей остальной логикой приложения. Третьим уровнем здесь является сервер базы данных. Эта трехуровневая архитектура оказалась более подходящей для некоторых сред, например, для сетей Internet и Intranet, где в качестве клиента может использоваться обычный Web-браузер.

2.7. Системные каталоги

Определение. Системный каталог – это хранилище данных, которые описывают сохраняемую в базе данных информацию, т. е. метаданные или «данные о данных».

Системный каталог СУБД является одним из фундаментальных компонентов системы. Многие программные компоненты СУБД строятся на использовании данных, хранящихся в системном каталоге. Например, модуль контроля прав доступа использует системный каталог для проверки наличия у пользователя полномочий, необходимых для выполнения запрошенных им операций. Для проведения подобной проверки системный каталог должен включать следующие компоненты:

- имена пользователей, для которых разрешен доступ к базе данных;
- имена элементов данных в базе данных;
- элементы данных, к которым каждый пользователь имеет право доступа, и разрешенные типы доступа к ним – для вставки, обновления, удаления или чтения.

Другим примером могут служить средства проверки целостности данных, которые используют системный каталог для проверки того, удовлетворяет ли запрошенная операция всем установленным ограничениям поддержки целостности данных. Для выполнения этой проверки в системном каталоге должны храниться такие сведения:

- имена элементов данных из базы данных;
- типы и размеры элементов данных;
- ограничения, установленные для каждого из элементов данных.

Термин «словарь данных» часто используется для программного обеспечения более общего типа, чем просто каталог СУБД. Система словаря данных может быть либо пассивной, либо активной. **Активная** система всегда согласуется со структурой базы данных, поскольку она автоматически поддерживается этой системой. **Пассивная** система может противоречить состоянию базы данных из-за иницилируемых пользователями изменений. Если словарь данных является частью базы данных, то он называется **интегрированным** словарем данных. **Изолированный** словарь данных обладает своей собственной специализированной СУБД. Его предпочтительно использовать на начальных этапах проектирования базы данных для некоторой организации, когда требуется отложить на какое-то время привязку к конкретной СУБД. Однако недостаток этого подхода заключается в том, что после выбора СУБД и воплощения базы данных изолированный словарь данных значительно труднее поддерживать в состоянии согласования с базой данных. Эту проблему можно было бы свести к минимуму, если преобразовать использовавшийся при проектировании словарь данных непосредственно в каталог СУБД. До недавнего времени это было невозможно, но по мере развития стандартов словарей данных эта идея становится все более реальной.

Во многих системах словарь данных является внутренним компонентом СУБД, в котором хранится информация, непосредственно связанная с этой базой данных. Однако хранимые с помощью СУБД данные обычно обеспечивают только часть всех информационных

потребностей организации. Как правило, имеется дополнительная информация, которая хранится с помощью других инструментов – например, таких как CASE-инструменты, инструменты ведения документации, инструменты настройки и управления проектами. Каждое из упомянутых выше средств обычно имеет свой внутренний словарь данных, доступный и другим внешним инструментам. К сожалению, не существует никакого универсального способа совместного использования разных наборов информации различными группами пользователей или приложений.

Относительно недавно была предпринята попытка стандартизировать интерфейс словарей данных для достижения большей доступности и упрощения их совместного использования. Ее результатом стала разработка службы словаря информационных ресурсов (Information Resource Dictionary System – IRDS). Служба IRDS представляет собой программный инструмент, предназначенный для управления информационными ресурсами организации, а также для их документирования. Она включает определение таблиц, содержащих словарь данных, и операций, которые могут быть использованы для доступа к этим таблицам. Операции обеспечивают непротиворечивый метод доступа к словарю данных и способ преобразования определений данных из словаря одного типа в определения другого типа. Например, с помощью службы IRDS информация, хранимая в IRDS-совместимом словаре данных СУБД DB2, может быть передана в IRDS-совместимый словарь данных СУБД Oracle или направлена некоторому приложению системы DB2.

Одним из достоинств службы IRDS является способность расширения словаря данных. Так, если пользователь захочет сохранить определения нового типа информации в каком-то инструменте (например, определения отчетов об управлении проектом в некоторой СУБД), то система IRDS в данной СУБД может быть расширена для включения этой информации. Определения IRDS были приняты в качестве стандарта ISO (1990, 1993). Стандарты IRDS определяют набор правил хранения информации в словаре данных и доступа к ней, преследуя при этом три следующих цели:

- расширяемость данных;
- целостность данных;
- контролируемый доступ к данным.

Служба IRDS построена на использовании интерфейса сервисов, состоящего из набора функций, доступного для вызова с целью получения доступа к словарю данных. Интерфейс сервисов может вызываться со стороны таких типов пользовательских интерфейсов, как:

- панельный интерфейс;
- командный язык;
- файлы экспорта/импорта;
- прикладные программы.

Панельный интерфейс состоит из набора панелей или экранов, каждый из которых предоставляет доступ к заранее описанному набору сервисов. Он несколько напоминает язык QBE и позволяет пользователям просматривать и изменять словарь данных. Интерфейс командного языка состоит из набора команд или операторов, которые позволяют выполнять манипуляции со словарем данных. Интерфейс командного языка может вызываться интерактивно (с помощью терминала) или посредством внедрения операторов в программы на языках программирования высокого уровня. Интерфейс экспорта/импорта генерирует файл, который можно передавать между различными IRDS-совместимыми системами. Стандарт IRDS определяет и универсальный формат обмена информацией. Причем он не требует, чтобы база данных, лежащая в основе словаря данных, соответствовала какой-то одной модели данных. Именно поэтому интерфейс IRDS-сервисов способен объединять гетерогенные СУБД.

ГЛАВА 3. ВВЕДЕНИЕ В РЕЛЯЦИОННЫЕ БАЗЫ ДАННЫХ

3.2. Используемая терминология

Структура реляционных данных

Определение. Отношение – это плоская таблица, состоящая из столбцов и строк.

В любой реляционной СУБД предполагается, что пользователь воспринимает базу данных как набор таблиц. Однако такое восприятие относится только к логической структуре базы данных, т. е. ко внешнему и концептуальному уровням архитектуры ANSI/SPARC. Подобное восприятие не относится к физической структуре базы данных, которая может быть реализована с помощью различных структур хранения.

Определение. Атрибут – это поименованный столбец отношения.

В реляционной модели отношения используются для хранения информации об объектах, представленных в базе данных. Отношение обычно имеет вид двумерной таблицы, в которой строки соответствуют отдельным записям, а столбцы – атрибутам. При этом атрибуты могут располагаться в любом порядке: независимо от их переупорядочивания отношение будет оставаться одним и тем же, а потому иметь тот же смысл.

Определение. Домен – это набор допустимых значений для одного или нескольких атрибутов.

. Каждый атрибут реляционной базы данных определяется на некотором домене. Домены могут отличаться для каждого из атрибутов, но два и более атрибутов могут определяться на одном и том же домене.

Понятие домена имеет большое значение, поскольку благодаря ему пользователь может централизованно определить смысл и источник значений, которые могут получать атрибуты. В результате при выполнении реляционной операции системе доступно больше информации, что позволяет ей избежать семантически некорректных операций.

Определение. Кортеж – это строка отношения.

Элементами отношения являются кортежи или строки таблицы. Кортежи могут располагаться в любом порядке, при этом отношение будет оставаться тем же самым, а значит, иметь тот же смысл.

Описание структуры отношения вместе со спецификацией доменов и любыми другими ограничениями возможных значений атрибутов иногда называют его заголовком или содержанием (intension). Обычно оно является фиксированным до тех пор, пока смысл отношения не изменяется за счет добавления в него дополнительных атрибутов. Кортежи называются **расширением** (extension), **состоянием** (state) или **телом** отношения, которое постоянно меняется.

Определение. Степень отношения – это количество атрибутов, которое оно содержит.

Отношение с одним атрибутом имеет степень 1 и называется **унарным** (unary) отношением или 1-арным кортежем. Отношение с двумя атрибутами называется **бинарным** (binary), отношение с тремя атрибутами – **тернарным** (ternary), а для отношений с большим количеством атрибутов используется термин **n-арный** (n-ary). Определение степени отношения является частью заголовка отношения.

Определение. Кардинальность – это количество кортежей, которое содержит отношение.

Количество содержащихся в отношении кортежей называется **кардинальностью** отношения. Эта характеристика меняется при каждом добавлении или удалении кортежей. Кардинальность является свойством тела отношения и определяется текущим состоянием отношения в произвольно взятый момент.

Определение. Реляционная база данных – это набор нормализованных отношений.

Реляционная база данных состоит из отношений, структура которых определяется с помощью особых методов, называемых нормализацией (normalization).

Отметим, что помимо двух предложенных наборов терминов в реляционной модели, существует еще один вариант, в котором отношение называется файлом (file), кортежи – записями (records), а атрибуты – полями (fields). Эта терминология основана на том факте, что физически РСУБД может хранить каждое отношение в отдельном файле В табл. 3.1 показаны соответствия, существующие между тремя группами терминов.

Таблица 3.1

Альтернативные варианты терминов в реляционной модели

Официальные термины	Альтернативный вариант 1	Альтернативный вариант 2
Отношение	Таблица	Файл
Кортеж	Строка	Запись
Атрибут	Столбец	Поле

Свойства отношений

Отношение обладает следующими характеристиками:

- отношение имеет имя, которое отличается от имен всех других отношений;
- каждая ячейка отношения содержит только атомарное (неделимое) значение;
- каждый атрибут имеет уникальное имя;
- значения атрибута берутся из одного и того же домена;
- порядок следования атрибутов не имеет никакого значения;
- каждый кортеж является уникальным, т. е. дубликатов кортежей быть не может;
- теоретически порядок следования кортежей в отношении не имеет никакого значения. Однако на практике этот порядок может существенно повлиять на эффективность доступа к ним.

Реляционные ключи

Необходимо иметь возможность уникальной идентификации каждого отдельного кортежа отношения по значениям его атрибутов.

Определение. Суперключ (superkey) – атрибут или множество атрибутов, которое единственным образом идентифицирует кортеж данного отношения.

Поскольку суперключ может содержать дополнительные атрибуты, которые необязательны для уникальной идентификации кортежа, наиболее интересными являются суперключи, состоящие только из тех атрибутов, которые действительно необходимы для уникальной идентификации кортежей.

Определение. Потенциальный ключ – это суперключ, который не содержит подмножества, также являющегося суперключом данного отношения.

Потенциальный ключ К для данного отношения R обладает двумя свойствами:

- **уникальность.** В каждом кортеже отношения R значение ключа К единственным образом идентифицируют этот кортеж;
- **неприводимость.** Никакое допустимое подмножество ключа К не обладает свойством уникальности.

Отношение может иметь несколько потенциальных ключей. Если ключ состоит из нескольких атрибутов, то он называется **составным ключом**.

Отметим, что любой конкретный набор кортежей отношения нельзя использовать для доказательства того, что некий атрибут или комбинация атрибутов являются потенциальным ключом. Тот факт, что в некоторый момент времени не существует значений-дубликатов, совсем не означает, что их не может быть вообще. Для идентификации потенциального ключа требуется знать *смысл* используемых атрибутов в «реальном мире», только это позволит обоснованно принять решение о возможности существования значений-дубликатов. Только

исходя из подобной семантической информации можно гарантировать, что некоторая комбинация атрибутов является потенциальным ключом отношения.

Определение. Первичный ключ – это потенциальный ключ, который выбран для уникальной идентификации кортежей внутри отношения.

Поскольку отношение не содержит кортежей-дубликатов, всегда можно уникальным образом идентифицировать каждую его строку. Это значит, что отношение всегда имеет первичный ключ. В худшем случае все множество атрибутов можно использовать как первичный ключ, но обычно, чтобы различить кортежи, достаточно использовать несколько меньшее подмножество атрибутов. Потенциальные ключи, которые не выбраны в качестве первичного ключа, называются **альтернативными ключами**.

Определение. Внешний ключ - это атрибут или множество атрибутов внутри отношения, которое соответствует потенциальному ключу некоторого (может быть, того же самого) отношения.

Если некий атрибут присутствует в нескольких отношениях, то его наличие обычно отражает определенную связь между кортежами этих отношений.

3.3. Базовые таблицы и представления

Исходные таблицы базы данных называются **базовыми** таблицами, а таблицы, полученные из них путем выполнения реляционных выражений, называются **производными** таблицами. Базовые таблицы *существуют независимо*, в то время как производные таблицы зависят от базовых. Базовые таблицы являются *именованными* (их имена указываются в операторах создания), а большинство производных таблиц – наоборот, *неименованными*. Однако реляционные системы поддерживают один определенный вид производных таблиц, называемый *представлением* (view), которое имеет имя. Таким образом, **представление** – это именованная таблица, которая (в отличие от базовой) не может существовать сама по себе, а определяется в терминах одной или нескольких именованных таблиц (базовых таблиц или других представлений).

Определение. Базовое отношение – это поименованное отношение, соответствующее сущности в концептуальной схеме, кортежи которого физически хранятся в базе данных.

Определение. Представление – это динамический результат одной или нескольких реляционных операций над базовыми отношениями с целью создания некоторого иного отношения. Представление является **виртуальным отношением**, которого реально в базе данных не существует, но которое создается по требованию отдельного пользователя в момент поступления этого требования.

С точки зрения пользователя представление является отношением, которое постоянно существует и с которым можно работать точно так же, как с базовым отношением. Однако представление не хранится в базе данных так, как базовые отношения (хотя его определение хранится в системном каталоге). Содержимое представления определяется как результат выполнения запроса к одному или нескольким базовым отношениям. Любые операции над представлением автоматически транслируются в операции над отношениями, на основании которых оно было создано. Таким образом, представления имеют динамическую природу.

Механизм представлений может использоваться по нескольким причинам:

- он предоставляет мощный и гибкий механизм защиты за счет сокрытия некоторых частей базы данных от определенных пользователей. Пользователь не будет иметь никаких сведений о существовании каких-либо атрибутов или кортежей, отсутствующих в доступных ему представлениях;

- он позволяет организовать доступ пользователей к данным наиболее удобным для них образом, поэтому одни и те же данные в одно и то же время могут рассматриваться разными пользователями совершенно отличными способами;

- он позволяет упрощать сложные операции с базовыми отношениями. Например, если представление будет определено на основе соединения двух отношений, то пользователь сможет выполнять над ним простые унарные операции выборки и проекции, которые будут

автоматически преобразованы средствами СУБД в эквивалентные операции с выполнением соединения базовых отношений.

Все обновления данных в базовом отношении должны быть немедленно отражены во всех представлениях, связанных с этим базовым отношением. Аналогично, при обновлении данных в представлении внесенные изменения должны быть отражены в его базовом отношении. Однако на типы изменений, которые могут быть выполнены с помощью представлений, накладываются определенные ограничения:

- обновления допускаются через представление, которое определено на основе простого запроса к единственному базовому отношению и содержит первичный или потенциальный ключ этого базового отношения;
- обновления не допускаются в любых представлениях, определенных на базе нескольких базовых отношений;
- обновления не допускаются в любых представлениях, включающих выполнение операций обобщения или группирования.

Представления принято делить на следующие классы: **теоретически необновляемые**, **теоретически обновляемые** и **частично обновляемые**.

Различия между базовой таблицей и представлением характеризуются следующим образом:

- базовые таблицы «реально существуют» в том смысле, что они представляют данные, которые действительно хранятся в базе данных;
- представления, наоборот, «реально не существуют», а просто предоставляют различные способы просмотра «реальных» данных.

Отметим следующее, базовую таблицу в общем случае нельзя рассматривать как физически хранимую таблицу. Дело в том, что базовые таблицы представляют *абстракцию* некоторого множества хранимых данных, в которой скрыты все детали уровня хранения. В принципе базовая таблица и ее хранимый дубликат могут отличаться в разной степени.

3.4. История развития языка SQL

Язык SQL предназначен для доступа к информации и управления реляционной базой данных. Начиная с 1986 года комитеты ISO (International Organization for Standardization) и ANSI (American National Standards Institute) приступили к созданию ряда стандартов языка SQL, которые впоследствии были приняты и получили следующие названия: SQL86, SQL89, SQL92 и SQL99.

Различают версии: SQL86, 87, 89, 92, 99, 2003.

В настоящее время действует стандарт, принятый в 2003 году (SQL:2003) с небольшими модификациями, внесёнными позже.

SQL 86-87 – минимальный стандарт синтаксиса языка

SQL 89 – появился механизм ссылочной целостности

SQL 92 – расширяет возможности SQL путем введения процедур и функций, поддерживает практически все СУБД.

SQL 97 – характеризуется характерным расширением языка, поддержка регулярных выражений

SQL 2003 – введены расширения для работы с XML-данными, оконные функции.

SQL 2006 – расширена функциональность работы с XML-данными. Появилась возможность совместно использовать в запросах SQL и XQuery.

SQL 2008 – Улучшены возможности оконных функций, устранены некоторые неоднозначности стандарта SQL-2003

Процедурные расширения

Поскольку SQL не является языком программирования (то есть не предоставляет средств для автоматизации операций с данными). Практически в каждой СУБД применяется свой процедурный язык. Стандарт для процедурных расширений представлен спецификацией [SQL/PSM](#). Расширение SQL/PSM закреплено стандартом ISO/IEC 9075-4:2003. SQL/PSM стандартизирует процедурное расширение для SQL, включая управление потоком выполнения, обработку условий, обработку флагов состояний, курсоры и локальные переменные, а также присваивание выражений переменным и параметрам. Более того, SQL/PSM формализует объявление и поддержку постоянных подпрограмм языков баз данных (например, «хранимых процедур»).

Перечень процедурных расширений для самых популярных СУБД приведён в следующей таблице:

СУБД	Краткое название	Расшифровка
InterBase/Firebird	PSQL	Procedural SQL
IBM DB2	SQL PL (англ.)	SQL Procedural Language (расширяет SQL/PSM); также в DB2 хранимые процедуры могут писаться на обычных языках программирования: Си , Java и т. д.
MS SQL Server/Sybase ASE	Transact-SQL	Transact-SQL
MySQL	SQL/PSM	SQL/Persistent Stored Module
Oracle	PL/SQL	Procedural Language/SQL (основан на языке Ada)
PostgreSQL	PL/pgSQL	Procedural Language/PostgreSQL Structured Query Language (очень похож на Oracle PL/SQL)

SQL

Операторы

Операторы SQL делятся на:

- операторы определения данных (*Data Definition Language, DDL*)
 - CREATE создает объект БД (саму базу, таблицу, представление, пользователя и т. д.)
 - ALTER изменяет объект
 - DROP удаляет объект
- операторы манипуляции данными (*Data Manipulation Language, DML*)
 - SELECT считывает данные, удовлетворяющие заданным условиям
 - INSERT добавляет новые данные
 - UPDATE изменяет существующие данные
 - DELETE удаляет данные
- операторы определения доступа к данным (*Data Control Language, DCL*)
 - GRANT предоставляет пользователю (группе) разрешения на определенные операции с объектом
 - REVOKE отзывает ранее выданные разрешения
 - DENY задает запрет, имеющий приоритет над разрешением
- операторы управления транзакциями (*Transaction Control Language, TCL*)
 - COMMIT применяет транзакцию.
 - ROLLBACK откатывает все изменения, сделанные в контексте текущей транзакции.
 - SAVEPOINT делит транзакцию на более мелкие участки.

Create (SQL)

Наиболее общие команды (поддерживаются большинством СУБД): CREATE TABLE и CREATE VIEW

```
CREATE TABLE Student (  
    Code INTEGER NOT NULL,  
    Name CHAR (30) NOT NULL ,  
    Address CHAR (50),  
    Mark DECIMAL  
);
```

```
CREATE VIEW London_view AS SELECT * FROM Salespeople WHERE city = 'London';
```

Alter (SQL)

ALTER TABLE - данный запрос используется для добавления, удаления или модификации существующих таблиц.

```
ALTER TABLE table_name  
ADD column_name datatype
```

Для удаления колонки в таблице, используйте следующий синтаксис (не все базы данных позволяют удалять одну колонку):

```
ALTER TABLE table_name  
DROP COLUMN column_name
```

Для изменения типа данных колонки, используйте следующий синтаксис:

```
ALTER TABLE table_name
ALTER COLUMN column_name datatype
```

Drop (SQL)

Индексы, таблицы и базы данных могут быть легко удалены с помощью запроса DROP

```
DROP INDEX index_name
DROP TABLE table_name
DROP DATABASE database_name
```

Select (SQL)

Формат запроса с использованием данного оператора:

SELECT *список полей* **FROM** *список таблиц* **WHERE** *условия...*

Основные ключевые слова, относящиеся к запросу SELECT:

- **WHERE** — используется для определения, какие строки должны быть выбраны или включены в GROUP BY.
- **GROUP BY** — используется для объединения строк с общими значениями в элементы меньшего набора строк.
- **HAVING** — используется для определения, какие строки после GROUP BY должны быть выбраны.
- **ORDER BY** — используется для определения, какие столбцы используются для сортировки результирующего набора данных.

```
SELECT LastName,FirstName FROM Persons
SELECT * FROM Persons
```

Возвращает список идентификаторов отделов, продажи которых превысили 1000 долларов за 1 января 2000 года, вместе с суммами продаж за этот день:

```
SELECT DeptID, SUM(SaleAmount) FROM Sales
WHERE SaleDate = '01-Jan-2000'
GROUP BY DeptID
HAVING SUM(SaleAmount) > 1000
```

Insert (SQL)

INSERT INTO - используется для добавления новых строк в таблицу.

```
INSERT INTO Persons (P_Id, LastName, FirstName)
VALUES (5, 'Tjessem', 'Jakob')
```

Update (SQL)

UPDATE используется для обновления записей в таблице.

```
UPDATE Persons
SET Address='Nissestien 67', City='Sandnes'
WHERE LastName='Tjessem' AND FirstName='Jakob'
```

Что будет если пропустить WHERE?

Delete (SQL)

DELETE используется для удаление записей в таблице.

```
DELETE FROM Persons
WHERE LastName='Tjessem' AND FirstName='Jakob'
```

Как удалить все строки?

Grant/ Revoke (SQL)

привилегии даются и отменяются двум командами: GRANT (ДОПУСК) и REVOKE (ОТМЕНА).

```
GRANT privilege_name
ON object_name
TO {user_name |PUBLIC |role_name}
[WITH GRANT OPTION];

GRANT SELECT, INSERT ON Orders TO Adrian, Diane;
GRANT UPDATE (city, comm) ON Salespeople TO Diane;
REVOKE INSERT, DELETE ON Customers FROM Adrian, Stephen;

BEGIN TRANSACTION;
INSERT INTO MyTable VALUES ('50', 'some string');
COMMIT WORK;
```

Виды оператора JOIN

Для дальнейших пояснений будут использоваться следующие таблицы:

Люди, проживающие в городах (таблица Person)

Name	CityId
Андрей	1
Леонид	2
Сергей	1
Григорий	4

Города (таблица City)

Id	Name
1	Москва
2	Санкт-Петербург
3	Казань

INNER JOIN

Оператор *внутреннего соединения* INNER JOIN соединяет две таблицы. Порядок таблиц для оператора неважен, поскольку оператор является [симметричным](#).

Заголовок таблицы-результата является объединением ([конкатенацией](#)) заголовков соединяемых таблиц.

Тело результата логически формируется следующим образом. Каждая строка одной таблицы сопоставляется с каждой строкой второй таблицы, после чего для полученной «соединённой» строки проверяется условие соединения (вычисляется предикат соединения). Если условие истинно, в таблицу-результат добавляется соответствующая «соединённая» строка.

```
SELECT *
FROM
    Person
```

INNER JOIN

City

ON Person.CityId = City.Id

Результат:

Person.Name	Person.CityId	City.Id	City.Name
Андрей	1	1	Москва
Леонид	2	2	Санкт-Петербург
Сергей	1	1	Москва

OUTER JOIN

Соединение двух таблиц, в результат которого в обязательном порядке входят строки либо одной, либо обеих таблиц.

LEFT OUTER JOIN

Оператор *левого внешнего соединения* LEFT OUTER JOIN соединяет две таблицы. Порядок таблиц для оператора важен, поскольку оператор не является [симметричным](#).

Тело результата логически формируется следующим образом. Пусть выполняется соединение левой и правой таблиц по предикату (условию) *p*.

1. В результат включается внутреннее соединение (INNER JOIN) левой и правой таблиц по предикату *p*.
2. Затем в результат добавляются те записи левой таблицы, которые не вошли во внутреннее соединение на шаге 1. Для таких записей поля, соответствующие правой таблице, заполняются значениями *NULL*.

```
SELECT *  
FROM  
    Person  
    LEFT OUTER JOIN  
    City  
    ON Person.CityId = City.Id
```

Результат:

Person.Name	Person.CityId	City.Id	City.Name
Андрей	1	1	Москва
Леонид	2	2	Санкт-Петербург
Сергей	1	1	Москва
Григорий	4	NULL	NULL

RIGHT OUTER JOIN

```
SELECT *  
FROM  
    Person  
    RIGHT OUTER JOIN  
    City  
    ON Person.CityId = City.Id
```

Результат:

Person.Name	Person.CityId	City.Id	City.Name
Андрей	1	1	Москва
Сергей	1	1	Москва

Леонид	2	2	Санкт-Петербург
NULL	NULL	3	Казань

FULL OUTER JOIN

Оператор *полного внешнего соединения* FULL OUTER JOIN соединяет две таблицы. Порядок таблиц для оператора неважен, поскольку оператор является [симметричным](#).

Тело результата логически формируется следующим образом. Пусть выполняется соединение первой и второй таблиц по предикату (условию) *p*. Слова «первой» и «второй» здесь не обозначают порядок в записи (который неважен), а используются лишь для различения таблиц.

1. В результат включается внутреннее соединение (INNER JOIN) первой и второй таблиц по предикату *p*.
2. В результат добавляются те записи первой таблицы, которые не вошли во внутреннее соединение на шаге 1. Для таких записей поля, соответствующие второй таблице, заполняются значениями *NULL*.
3. В результат добавляются те записи второй таблицы, которые не вошли во внутреннее соединение на шаге 1. Для таких записей поля, соответствующие первой таблице, заполняются значениями *NULL*.

```
SELECT *
FROM
    Person
    FULL OUTER JOIN
    City
    ON Person.CityId = City.Id
```

Результат:

Person.Name	Person.CityId	City.Id	City.Name
Андрей	1	1	Москва
Сергей	1	1	Москва
Леонид	2	2	Санкт-Петербург
NULL	NULL	3	Казань
Григорий	4	NULL	NULL

CROSS JOIN

Оператор *перекрёстного соединения*, или *декартова произведения* CROSS JOIN соединяет две таблицы. Порядок таблиц для оператора неважен, поскольку оператор является [симметричным](#).

Тело результата логически формируется следующим образом. Каждая строка одной таблицы соединяется с каждой строкой второй таблицы, давая тем самым в результате все возможные сочетания строк двух таблиц.

```
SELECT *
FROM
    Person
    CROSS JOIN
    City
```

Или

```
SELECT *
FROM
```


Person,
City

Результат:

Person.Name	Person.CityId	City.Id	City.Name
Андрей	1	1	Москва
Андрей	1	2	Санкт-Петербург
Андрей	1	3	Казань
Леонид	2	1	Москва
Леонид	2	2	Санкт-Петербург
Леонид	2	3	Казань
Сергей	1	1	Москва
Сергей	1	2	Санкт-Петербург
Сергей	1	3	Казань
Григорий	4	1	Москва
Григорий	4	2	Санкт-Петербург
Григорий	4	3	Казань

3.5. Признаки реляционной СУБД

Несмотря на то, что существует несколько сотен реляционных СУБД, некоторые из них, строго говоря, не соответствуют определению реляционной модели. В 1985 году Э. Ф. Кодд предложил 12 правил определения реляционных систем (точнее 13, если учитывать фундаментальное правило 0). Эти правила образуют своего рода эталон, по которому можно определить принадлежность СУБД к разряду действительно реляционных систем. Их можно разделить на пять функциональных групп: фундаментальные правила; структурные правила; правила целостности; правила управления данными; правила независимости от данных.

Фундаментальные правила (правила 0 и 12)

Правило 0 – фундаментальное правило

Любая система, которая рекламируется или представляется как реляционная СУБД, должна быть способна управлять базами данных исключительно с помощью ее реляционных функций.

Правило 12 – правило запрета обходных путей

Если реляционная система имеет низкоуровневый язык (с последовательной построчной обработкой), он не может быть использован для отмены или обхода правил и ограничений целостности, составленных на реляционном языке более высокого уровня (с обработкой сразу нескольких строк).

Структурные правила (правила 1 и 6)

Правило 1 – представление информации

Вся информация в реляционной базе данных представляется в явном виде на логическом уровне и только одним способом – в виде значений в таблицах.

Правило 6 – обновление представления

Все представления, которые являются теоретически обновляемыми, должны быть обновляемы и в данной системе.

Правила целостности (правила 3 и 10)

Правило 3 – систематическая обработка неопределенных значений (NULL)

Неопределенные значения (задаваемые с помощью определителя NULL), т. е. значения, отличные от пустой строки или строки пустых символов, а также от нуля или любого другого конкретного значения, поддерживаются для систематического представления отсутствующей или неприемлемой информации, причем независимо от типа данных.

Правило 10 – независимость ограничений целостности

Специфические для данной РСУБД ограничения целостности должны определяться на подязыке реляционных данных и храниться в системном каталоге, а не в прикладных программах.

Правила манипулирования данными (правила 2, 4, 5 и 7)

Правило 2 – гарантированный доступ

Для всех и каждого элемента данных (т. е. его атомарного значения) реляционной базы данных должен быть гарантирован логический доступ на основе комбинации имени таблицы, значения первичного ключа и значения имени столбца.

Правило 4 – динамический интерактивный каталог, построенный по правилам реляционной модели

Описание базы данных должно представляться на логическом уровне таким же образом, как и обычные данные, что позволит санкционированным пользователями использовать для обращений к этому описанию тот же реляционный язык, что и при обращении к данным.

Правило 5 – исчерпывающий подязык данных

Реляционная система может поддерживать несколько языков и различные режимы работы с терминалами (например, режим заполнения формы – fill-in-the-blanks). Однако должен существовать по крайней мере один язык, операторы которого позволяли бы выражать все следующие конструкции: 1) определение данных; 2) определение представлений; 3) команды манипулирования данными (доступные как интерактивно, так и из программ); 4) ограничения целостности; 5) авторизация пользователей; 6) организация транзакций (запуск, фиксация и откат).

Правило 7 – высокоуровневые операции вставки, обновления и удаления

Способность обрабатывать базовые или производные отношения (т. е. представления) как единый операнд должна относиться не только к процедурам извлечения данных, но и к операциям вставки, обновления и удаления данных.

Правила независимости от данных (правила 8, 9 и 11)

Правило 8 – физическая независимость от данных

Прикладные программы и средства работы с терминалами должны оставаться логически незатронутыми при внесении любых изменений в способы хранения данных или методы доступа к ним.

Правило 9 – логическая независимость от данных

Прикладные программы и средства работы с терминалами должны оставаться логически незатронутыми при внесении в базовые таблицы любых не меняющих информацию изменений, которые теоретически не должны затрагивать прикладное программное обеспечение.

Правило 11 – независимость от распределения данных

Подязык манипулирования данными в реляционной СУБД должен позволять прикладным программам и запросам оставаться логически неизменными независимо от того, как хранятся данные – физически централизованно или в распределенном виде.

ГЛАВА 4. ПЛАНИРОВАНИЕ, ПРОЕКТИРОВАНИЕ И АДМИНИСТРИРОВАНИЕ БАЗЫ ДАННЫХ

4.1. Обзор жизненного цикла информационных систем

Определение. Информационная система – это ресурсы, которые позволяют выполнять сбор, корректировку и распространение информации внутри организации.

Типичная компьютеризированная информационная система включает такие компоненты, как:

- база данных;
- программное обеспечение базы данных;
- прикладное программное обеспечение;
- аппаратное обеспечение, в том числе устройства хранения;
- персонал, использующий и разрабатывающий эту систему.

База данных является фундаментальным компонентом информационной системы, а ее разработку и использование следует рассматривать с точки зрения самых широких требований организации. Следовательно, **жизненный цикл информационной системы** (Information Systems Lifecycle) организации неотъемлемым образом связан с жизненным циклом системы базы данных, поддерживающей его.

4.2. Жизненный цикл приложения баз данных

Этапы жизненного цикла приложения базы данных показаны на рис. 4.1. Эти этапы не являются строго последовательными и могут включать некоторое количество *циклов обратной связи* (feedback loop).

Основные действия, выполняемые на каждом этапе жизненного цикла базы данных:

- *планирование разработки базы данных*. Планирование самого эффективного способа реализации этапов жизненного цикла;
- *определение требований к системе*. Определение диапазона действия и границ приложения базы данных, состава его пользователей и областей применения;
- *сбор и анализ требований пользователей*.
- *проектирование базы данных*. Концептуальное, логическое и физическое проектирование базы данных;
- *выбор целевой СУБД* (необязательно). Выбор наиболее подходящей СУБД для приложения базы данных;
- *разработка приложений*. Определение пользовательского интерфейса и прикладных программ;
- *создание прототипов* (необязательно). На этом этапе создается рабочая модель приложения базы данных;
- *реализация*. Этот этап включает создание внешнего, концептуального и внутреннего определений базы данных и прикладных программ;
- *конвертирование и загрузка данных*. На этом этапе выполняется преобразование и загрузка данных из старой системы в новую;

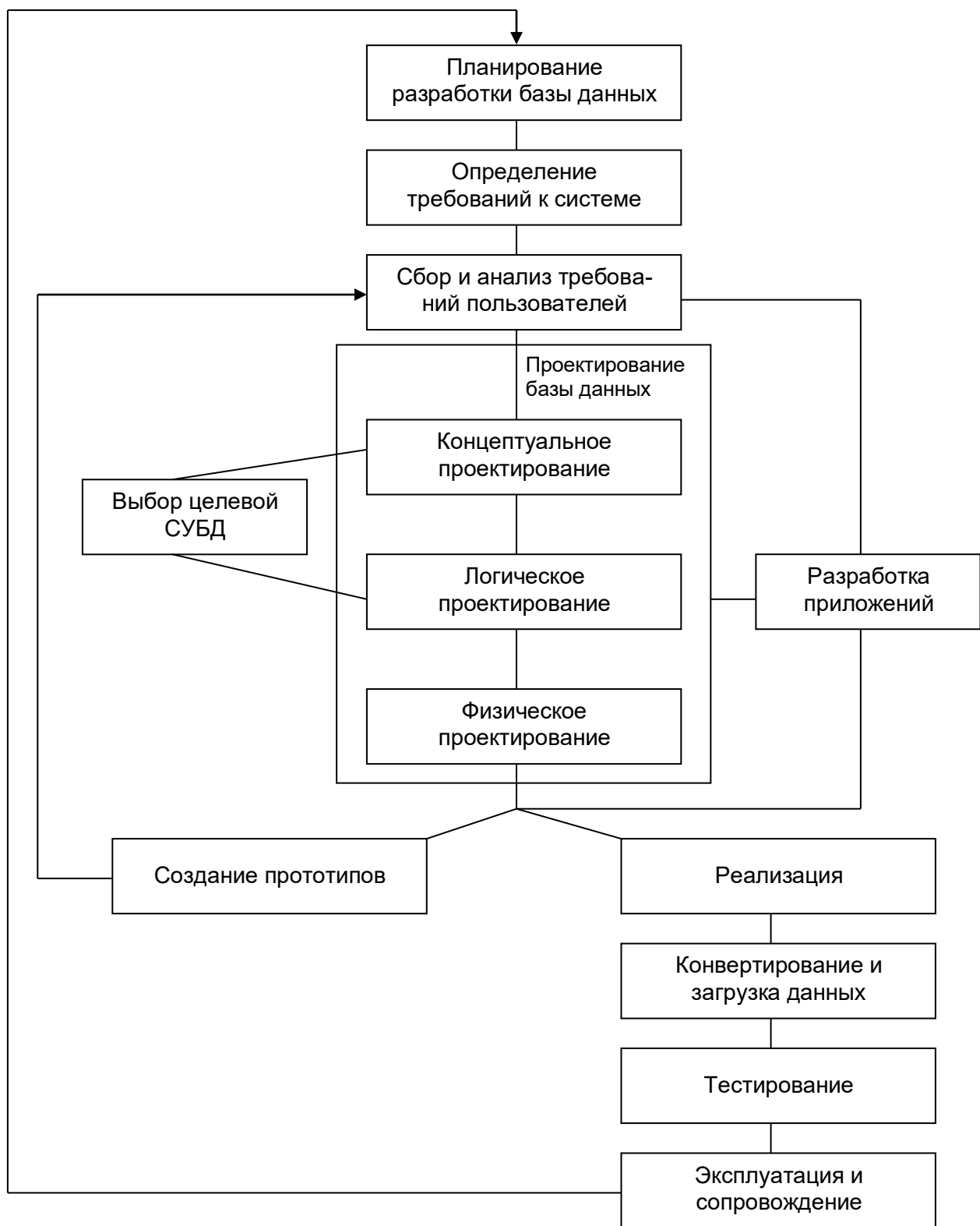


Рис. 4.1. Жизненный цикл приложения баз данных

- *тестирование*. Приложение базы данных тестируется с целью обнаружения ошибок;
- *эксплуатация и сопровождение*. Вся система должна находиться под постоянным наблюдением и соответствующим образом поддерживаться.

4.3.2. Концептуальное проектирование базы данных

Определение. Концептуальное проектирование базы данных – это процесс создания модели используемой на предприятии информации, не зависящей от *любых* физических аспектов ее представления.

Концептуальное проектирование базы данных не зависит от таких подробностей реализации, как тип выбранной целевой СУБД, набор создаваемых прикладных программ, используемые языки программирования, тип выбранной вычислительной платформы, а также от других особенностей физической реализации. Созданная концептуальная модель данных предприятия является источником информации для фазы логического проектирования базы данных.

4.3.3. Логическое проектирование базы данных

Определение. Логическое проектирование базы данных – это процесс создания модели используемой на предприятии информации с учетом выбранной модели организации данных, но независимо от типа целевой СУБД и других физических аспектов реализации.

Логическая модель данных учитывает особенности выбранной модели организации данных в целевой СУБД (например, реляционная, иерархическая, сетевая или объектно-ориентированная модель). Однако на этом этапе игнорируются все остальные аспекты выбранной СУБД (например, любые особенности физической организации ее структур хранения данных и построения индексов).

4.3.4. Физическое проектирование базы данных

Определение. Физическое проектирование данных – это процесс создания описания реализации базы данных на вторичных запоминающих устройствах с указанием структур хранения и методов доступа, используемых для организации эффективной обработки данных.

Основная цель физического проектирования базы данных состоит в описании способа физической реализации логического проекта базы данных. В случае реляционной модели данных под этим подразумевается следующее:

- создание набора реляционных таблиц и ограничений для них на основе информации, представленной в глобальной логической модели данных;
- определение конкретных структур хранения данных и методов доступа к ним, обеспечивающих оптимальную производительность системы с базой данных;
- разработка средств защиты создаваемой системы.

4.4.1. Проектирование транзакций

Определение. Транзакция – это действие или последовательность действий, выполняемых одним и тем же пользователем (или прикладной программой), осуществляющим доступ к базе данных или изменение ее содержимого.

Транзакции представляют собой события реального мира (регистрация нового клиента, регистрация предлагаемого в аренду объекта недвижимости и т. п.).

Существует три основных типа транзакций: транзакции извлечения, транзакции обновления и смешанные транзакции:

- **транзакции извлечения** используются для выборки некоторых данных с целью отображения их на экране или помещения в отчет;
- **транзакции обновления** используются для вставки новых, удаления старых или же изменения уже существующих записей базы данных;
- **смешанные транзакции** включают как операции извлечения, так и операции обновления данных.

4.7.1. Администрирование данных

Определение. Администрирование данных – это управление информационными ресурсами, включая планирование базы данных, разработку и внедрение стандартов, определение ограничений и процедур, а также концептуальное и логическое проектирование баз данных.

Основные задачи администрирования данных:

- выбор подходящих инструментов разработки;
- помощь в разработке корпоративных стратегий построения информационной системы, развития информационных технологий и бизнес-стратегий;
- предварительная оценка осуществимости и планирование процесса создания базы данных;
- разработка корпоративной модели данных;
- определение требований организации к используемым данным;
- определение стандартов сбора данных и выбор формата их представления;
- оценка объемов данных и вероятности их роста;
- определение способов и интенсивности использования данных;
- определение правил доступа к данным и мер безопасности, соответствующих правовым нормам и внутренним требованиям организации;
- концептуальное и логическое проектирование базы данных;
- взаимодействие с АБД и разработчиками приложений с целью обеспечения соответствия создаваемых приложений всем существующим требованиям;
- обучение пользователей – изучение существующих стандартов обработки данных и юридической ответственности за их некорректное применение;
- постоянная модернизация используемых информационных систем и технологий по мере развития бизнес-процессов;
- обеспечение полноты всей требуемой документации, включая корпоративную модель, стандарты, ограничения, процедуры, использование словаря данных, а также элементы управления работой конечных пользователей;
- поддержка словаря данных организации;
- взаимодействие с конечными пользователями для определения новых требований и разрешения проблем, связанных с доступом к данным и недостаточной производительностью их обработки.

4.7.2. Администрирование базы данных

Определение. Администрирование базы данных – это управление физической реализацией приложений баз данных: физическое проектирование базы данных и ее реализация, организация поддержки целостности и защиты данных, наблюдение за текущим уровнем производительности системы, а также реорганизация базы данных по мере необходимости.

Основные задачи администрирования базы данных:

- оценка и выбор целевой СУБД;
- физическое проектирование базы данных;
- реализация физического проекта базы данных в среде целевой СУБД;
- определение требований защиты и поддержки целостности данных;
- взаимодействие с разработчиками приложений баз данных;
- разработка стратегии тестирования;
- обучение пользователей;
- ответственность за сдачу в эксплуатацию готового приложения базы данных;
- контроль текущей производительности системы и соответствующая настройка базы данных;
- регулярное резервное копирование;
- разработка требуемых механизмов и процедур восстановления;

- обеспечение полноты используемой документации, включая материалы, разработанные внутри организации;
- поддержка актуальности используемого программного и аппаратного обеспечения, включая заказ и установку пакетов обновлений в случае необходимости.

ГЛАВА 5. РЕЛЯЦИОННАЯ АЛГЕБРА

5.1. Реляционные языки

Для управления отношениями в реляционных СУБД используются самые разнообразные языки. В 1971 году Э. Ф. Коддом было предложено использовать **реляционную алгебру** и **реляционное исчисление** в качестве основы для создания реляционных языков. Реляционную алгебру можно описать как высокоуровневый процедурный язык, который используется для того, чтобы сообщить СУБД о том, *как* следует построить требуемое отношение.

5.2. Обзор начальной алгебры

Определение. Реляционная алгебра – это теоретический язык операций, которые на основе одного или нескольких отношений позволяют создавать другое отношение без изменения самих исходных отношений.

Реляционная алгебра является языком последовательного использования отношений, в котором все кортежи, возможно, даже взятые из разных отношений, обрабатываются одной командой без организации циклов. Э. Ф. Кодд определил так называемую «начальную алгебру» – набор из восьми операторов, символически показанный на рис. 5.1, составляющих две группы по четыре оператора в каждой:

- традиционные операции над множествами: *объединение, пересечение, вычитание и декартово произведение* (все они модифицированы с учетом того, что их операндами являются отношения, а не произвольные множества);
- специальные реляционные операции: *выборка, проекция, соединение и деление*.

Операции выборки и проекции являются **унарными**, поскольку они работают с одним отношением. Другие операции работают с парами отношений, и поэтому их называют **бинарными** операциями. Приведем упрощенные определения восьми базовых операторов.

Выборка возвращает отношение, содержащее все кортежи из определенного отношения, которые удовлетворяют определенным условиям.

Проекция возвращает отношение, содержащее все кортежи (называемые подкортежами) определенного отношения после исключения из него некоторых атрибутов.

Произведение возвращает отношение, содержащее всевозможные кортежи, которые являются сочетанием двух кортежей, принадлежащих соответственно двум определенным отношениям.

Объединение возвращает отношение, содержащее все кортежи, которые принадлежат или одному из двух определенных отношений, или обоим отношениям.

Пересечение возвращает отношение, содержащее все кортежи, которые принадлежат одновременно двум определенным отношениям.

Вычитание возвращает отношение, содержащее все кортежи, которые принадлежат первому из двух определенных отношений и не принадлежат второму.

Соединение возвращает отношение, кортежи которого являются сочетанием двух кортежей (принадлежащих соответственно двум определенным отношениям), имеющих общее значение для одного или нескольких общих атрибутов этих двух отношений (и такие общие значения в результирующем кортеже появляются только один раз, а не дважды).

Деление для двух отношений, бинарного и унарного, возвращает отношение, содержащее все значения одного атрибута бинарного отношения, которые соответствуют (в другом атрибуте) всем значениям в унарном отношении.

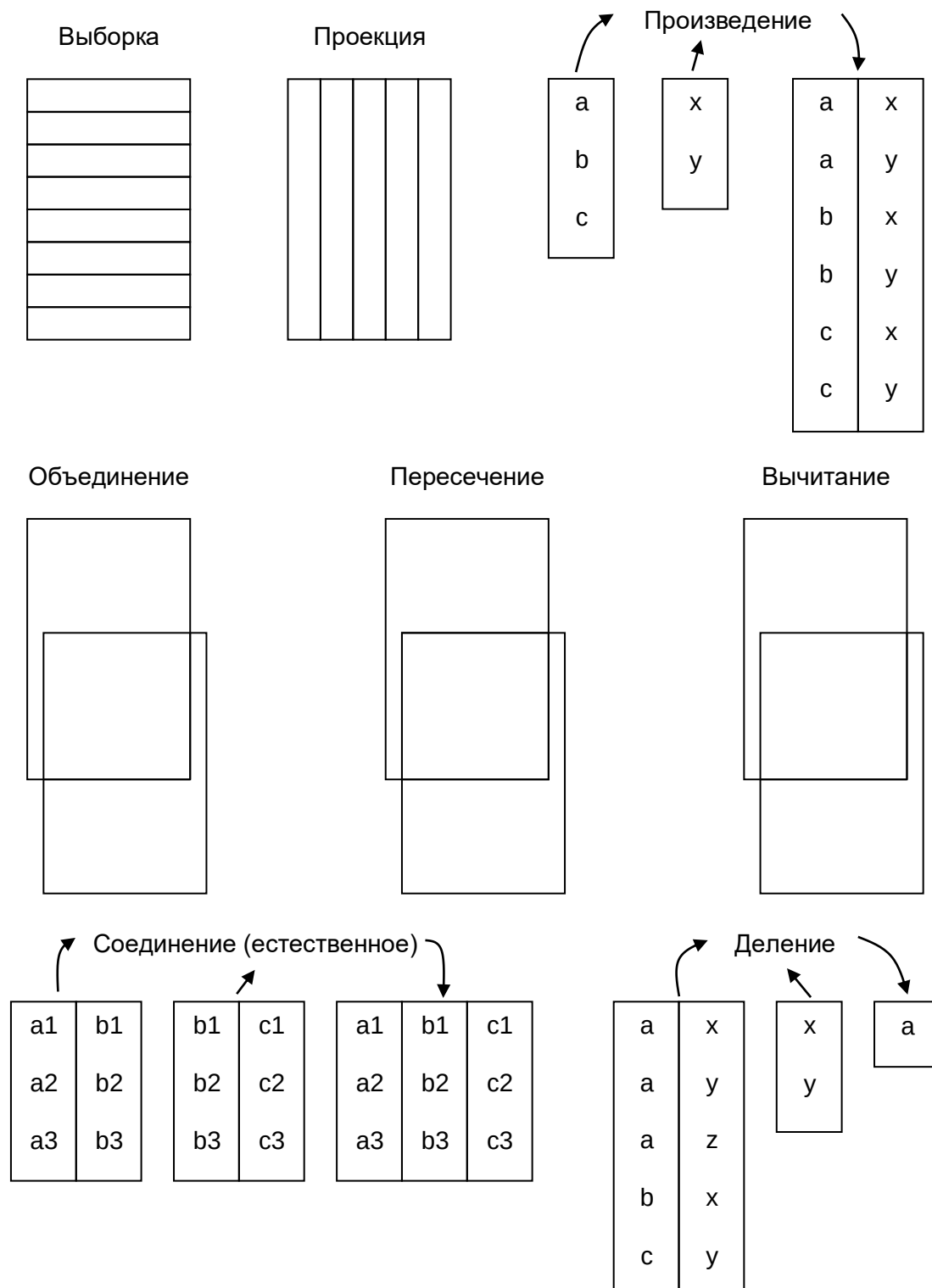


Рис. 7.1. Схематичное представление функций операторов реляционной алгебры

ГЛАВА 7. НОРМАЛИЗАЦИЯ

7.1. Цель нормализации

Нормальная форма — свойство отношения в реляционной модели данных, характеризующее его с точки зрения избыточности, которая потенциально может привести к логически ошибочным результатам выборки или изменения данных. Нормальная форма определяется как совокупность требований, которым должно удовлетворять отношение.

Процесс преобразования отношений базы данных к виду, отвечающему нормальным формам, называется **нормализацией**. Нормализация предназначена для приведения структуры базы данных к виду, обеспечивающему минимальную логическую избыточность, и не имеет целью уменьшение или увеличение производительности работы или же уменьшение или увеличение физического объёма БД. Конечной целью нормализации является уменьшение потенциальной противоречивости хранимой в БД информации. Как отмечает К. Дейт, общее назначение процесса нормализации заключается в следующем:

- исключение некоторых типов избыточности;
- устранение некоторых аномалий обновления;
- разработка проекта базы данных, который является достаточно «качественным» представлением реального мира, интуитивно понятен и может служить хорошей основой для последующего расширения;
- упрощение процедуры применения необходимых ограничений целостности.

Первая нормальная форма (1NF)

Отношение находится в первой нормальной форме тогда и только тогда, когда в любом допустимом значении отношения каждый его кортеж содержит только одно значение для каждого из атрибутов.

- Нет повторяющихся строк.
- Каждое пересечение строки и столбца содержит ровно одно значение из соответствующего домена (и больше ничего).

Исходная ненормализованная (то есть не являющаяся правильным представлением некоторого отношения) таблица:

Сотрудник	Номер телефона
Иванов И. И.	283-56-82
	390-57-34
Петров П. П.	708-62-34

Таблица, приведённая к 1NF (являющаяся правильным представлением некоторого отношения):

Сотрудник	Номер телефона
Иванов И. И.	283-56-82
Иванов И. И.	390-57-34
Петров П. П.	708-62-34

Вторая нормальная форма (2NF)

Отношение находится во второй нормальной форме, если оно находится в первой нормальной форме, и не ключевые столбцы таблиц зависят от первичного ключа в целом, а не от его части. Если таблица находится в первой нормальной форме и первичный ключ у

нее состоит из одного столбца, то она автоматически находится и во второй нормальной форме.

Пусть Сотрудник и Должность вместе образуют **первичный ключ** в такой таблице:

Сотрудник	Должность	Зарплата	Наличие компьютера
Гришин	Кладовщик	20000	Нет
Васильев	Программист	40000	Есть
Васильев	Кладовщик	25000	Нет

Зарплату сотруднику каждый начальник устанавливает сам, но её границы зависят от должности. Наличие же компьютера у сотрудника зависит только от должности, то есть зависимость от первичного ключа неполная.

В результате приведения к 2NF получаются две таблицы:

Сотрудник	Должность	Зарплата
Гришин	Кладовщик	20000
Васильев	Программист	40000
Васильев	Кладовщик	25000

Здесь первичный ключ, как и в исходной таблице, составной, но единственный не входящий в него атрибут Зарплата зависит теперь от всего ключа, то есть полно.

Должность	Наличие компьютера
Кладовщик	Нет
Программист	Есть

Третья нормальная форма (3NF)

Чтобы таблица находилась в третьей нормальной форме, необходимо, чтобы таблица была в 2NF и не ключевые столбцы в ней не зависели от других не ключевых столбцов, а зависели только от первичного ключа. Самая распространенная ситуация в - это расчетные столбцы, значения которых можно получить путем каких-либо манипуляций с другими столбцами таблицы. Для приведения таблицы в третью нормальную форму такие столбцы из таблиц надо удалить.

В исходной таблице одно-единственное ключевое поле (фамилия), такая таблица по определению имеет вторую нормальную форму. Личных телефонов у сотрудников нет, и телефон сотрудника зависит не от фамилии, а от отдела. Получается транзитивная связь: фамилия → отдел → телефон.

Фамилия	Отдел	Телефон
Гришин	Бухгалтерия	11-22-33
Васильев	Бухгалтерия	11-22-33
Петров	Снабжение	44-55-66

В результате приведения к 3NF получаются две таблицы:

Отдел	Наименование отдела	Телефон
1	Бухгалтерия	11-22-33
2	Снабжение	44-55-66

Фамилия	Отдел
Гришин	1
Васильев	1
Петров	2

Нормальная форма Бойса-Кодда (BCNF)

Это более строгая версия третьей нормальной формы, которая приобретает актуальность при наличии в отношении нескольких потенциальных ключей, хотя бы один из которых является составным. Нормальная форма Бойса-Кодда требует, чтобы в таблице был только один потенциальный первичный ключ. Если обнаружился второй столбец (комбинация столбцов), позволяющий однозначно идентифицировать строку, то для приведения к нормальной форме Бойса-Кодда такие данные необходимо вынести в отдельную таблицу.

Четвёртая нормальная форма (4NF)

Для приведения таблицы, находящейся в нормальной форме Бойса-Кодда, к четвертой нормальной форме необходимо устранить имеющиеся в ней многозначные зависимости. То есть обеспечить, чтобы вставка / удаление любой строки таблицы не требовала бы вставки / удаления / модификации других строк этой же таблицы.

Пятая нормальная форма (5NF)

Означает, что отношения в такой форме не имеют зависимостей соединения.

Отношение находится в 5НФ, если оно находится в 4НФ и для него отсутствуют зависимости соединения. Отношение находится в 5НФ тогда и только тогда, когда любая зависимость по соединению в нем определяется только его возможными ключами. Другими словами, каждая проекция такого отношения содержит не менее одного возможного ключа и не менее одного неключевого атрибута. 5НФ связана с зависимостью соединения, которая имеет довольно неопределенный характер, ввиду что не существует однозначных способов приведения отношения к 5НФ.

Зачем нужна нормализация.

Основная цель нормализации – это устранение (или, серьезное сокращение) избыточности, дублирования данных. Как следствие, значительно сокращается вероятность появления противоречивых данных, облегчается администрирование базы и обновление информации в ней, сокращается объем дискового пространства.

Зачастую, чтобы извлечь информацию из нормализованной базы данных, приходится конструировать очень сложные запросы, которые к тому же, бывает, работают довольно медленно - из-за, главным образом, большого количества соединений таблиц. Поэтому, чтобы увеличить скорость выборки данных и упростить программирование запросов, нередко приходится идти на выборочную денормализацию базы.

ГЛАВА 11. УПРАВЛЕНИЕ ТРАНЗАКЦИЯМИ

В функциональные возможности любой типичной СУБД должны входить три взаимосвязанных функции: механизмы поддержки транзакций, управление параллельностью и средства восстановления базы данных, – назначение которых состоит в гарантированном поддержании базы данных в достоверном и согласованном состоянии.

Определение. Транзакция – это действие или серия действий, выполняемых одним пользователем или прикладной программой, которые осуществляют доступ или изменение содержимого базы данных.

Любая транзакция всегда должна переводить базу данных из одного согласованного состояния в другое. Таким образом, любая транзакция завершается одним из двух возможных способов. В случае успешного завершения результаты транзакции **фиксируются** (commit) в базе данных. Если выполнение транзакции завершилось неудачно, она **отменяется**. В этом

случае в базе данных должно быть восстановлено то согласованное состояние, в котором она находилась до начала данной транзакции. Этот процесс называется **откатом** (roll back) транзакции. Зафиксированная транзакция не может быть отменена.

Свойства, которыми должна обладать транзакция:

- **атомарность** (atomicity). Любая транзакция представляет собой неделимую единицу работы, которая может быть либо выполнена вся целиком, либо не выполнена вовсе;
- **согласованность** (consistency). Каждая транзакция должна переводить базу данных из одного согласованного состояния в другое согласованное состояние;
- **изолированность** (isolation). Все транзакции выполняются независимо одна от другой, промежуточные результаты незавершенной транзакции не должны быть доступны другим транзакциям;
- **продолжительность** (durability). Результаты успешно завершенной (зафиксированной) транзакции должны сохраняться в базе данных постоянно и не должны быть утеряны в результате последующих сбоев.

Определение. Управление параллельностью – это процесс организации одновременного выполнения в базе данных различных операций, гарантирующий исключение их взаимного влияния друг на друга.

Определение. Восстановление базы данных – это процесс возвращения базы данных в корректное состояние, утраченное в результате сбоя или отказа.

Существует множество различных типов отказов, способных повлиять на функционирование базы данных:

- *аварийное прекращение работы системы*, вызванное ошибкой оборудования или программного обеспечения, приведшей к разрушению содержимого оперативной памяти;
- *отказ носителей информации*, например, разрушение магнитной головки или появление неустраняемого сбоя чтения, имеющее следствием потерю части вторичной памяти системы;
- *ошибка прикладных программ*, например, логические ошибки в программах, получающих доступ к базе данных, послужившие причиной сбоев при выполнении одной или нескольких транзакций;
- *стихийные бедствия*, например, пожары, наводнения, землетрясения или отказы в сети электропитания;
- *небрежное или легкомысленное отношение*, послужившее причиной непреднамеренного разрушения данных или программ со стороны оператора или пользователей системы;
- *диверсии* и преднамеренное разрушение или уничтожение данных, оборудования или программного обеспечения.

Типичная СУБД должна предоставлять такие функции восстановления, как:

- механизм резервного копирования, предназначенный для периодического создания копий базы данных;
- средства ведения журнала, в котором фиксируются текущее состояние транзакций и вносимые в базу данных изменения;
- функция создания контрольных точек, обеспечивающая перенос выполняемых в базе данных изменений во вторичную память с целью сделать их постоянными;
- менеджер восстановления, обеспечивающий восстановление согласованного состояния базы данных, нарушенного в результате отказа.

ГЛАВА 12. БЕЗОПАСНОСТЬ БАЗ ДАННЫХ

Среди многочисленных аспектов проблемы безопасности отметим следующие:

- правовые, общественные и этические аспекты (имеет ли право некоторое лицо получить запрашиваемую информацию, например, о кредите клиента);
- физические условия (например, закрыт ли данный компьютер или терминальная комната);

- организационные вопросы (например, как в рамках предприятия организован доступ к данным);
- вопросы реализации управления (например, если используется метод доступа по паролю, то как организована реализация управления и как часто меняются пароли);
- аппаратное обеспечение (обеспечиваются ли меры безопасности на аппаратном уровне, например, с помощью защитных ключей или привилегированного режима управления);
- безопасность операционной системы (например, затирает ли базовая операционная система содержание структуры хранения и файлов с данными при прекращении работы с ними);
- безопасность системы управления базами данных (например, существует ли для базы данных некоторая концепция предоставления прав владения данными).

Ограничимся рассмотрением вопросов, касающихся последнего пункта этого перечня.

В современных СУБД поддерживается один из двух широко распространенных подходов к вопросу обеспечения безопасности данных, а именно избирательный подход или обязательный подход.

– в случае **избирательного управления** пользователь обладает различными правами (**привилегиями** или **полномочиями**) при работе с разными объектами. Более того, различные пользователи обычно обладают и разными правами доступа к одному и тому же объекту. Таким образом, избирательные схемы характеризуются значительной гибкостью;

– в случае **обязательного управления**, наоборот, каждому объекту данных присваивается некоторый **классификационный уровень**, а каждый пользователь обладает некоторым **уровнем допуска**. Следовательно, при таком подходе доступом к определенному объекту данных обладают только пользователи с соответствующим уровнем допуска. Таким образом, обязательные схемы являются достаточно жесткими и статичными.

12.3. Шифрование данных

Определение. Шифрование – это кодирование данных с использованием специального алгоритма, в результате чего данные становятся недоступными для чтения любой программой, не имеющей ключа дешифрования.

Для организации защищенной передачи данных по незащищенным сетям должны использоваться **системы шифрования**, включающие следующие компоненты:

- ключ шифрования, предназначенный для шифрования исходных данных (обычного текста);
- алгоритм шифрования, который описывает, как с помощью ключа шифрования преобразовать обычный текст в шифротекст;
- ключ дешифрования, предназначенный для дешифрования шифротекста;
- алгоритм дешифрования, который описывает, как с помощью ключа дешифрования преобразовать шифротекст в исходный обычный текст.

Некоторые системы шифрования, называемые **симметричными**, используют один и тот же ключ как для шифрования, так и для дешифрования; при этом предполагается наличие защищенных линий связи, предназначенных для обмена ключами (DES).

Другой тип систем шифрования предусматривает использование для шифрования и дешифрования сообщений различных ключей. Такие системы принято называть **несимметричными**. Примером такой системы является система с открытым ключом, предусматривающая использование двух ключей, один из которых является открытым, а другой хранится в секрете. Алгоритм шифрования также может быть открытым, поэтому любой пользователь, желающий направить владельцу ключей зашифрованное сообщение, может использовать его открытый ключ и соответствующий алгоритм шифрования. Однако дешифровать данное сообщение сможет только тот, кто знает парный закрытый ключ шифрования. Наиболее популярной несимметричной системой шифрования является RSA (инициалы трех разработчиков данного алгоритма – Rivest, Shamir, Adleman).

ГЛАВА 13. ПРИНЦИПЫ УПРАВЛЕНИЯ РАСПРЕДЕЛЕННОЙ ИНФОРМАЦИЕЙ

13.1. Основные концепции

Определение. Распределенная база данных – это набор логически связанных между собой разделяемых данных (и их описаний), которые физически распределены в некоторой компьютерной сети.

Определение. Распределенная СУБД – это программный комплекс, предназначенный для управления распределенными базами данных и позволяющий сделать распределенность информации прозрачной для конечного пользователя.

Система управления распределенными базами данных (СУРБД) состоит из единой логической базы данных, разделенной на некоторое количество **фрагментов**. Пользователи взаимодействуют с распределенной базой данных через приложения. Приложения могут быть классифицированы как те, которые не требуют доступа к данным на других сайтах (**локальные приложения**), и те, которые требуют подобного доступа (**глобальные приложения**). В распределенной СУБД должно существовать хотя бы одно глобальное приложение, поэтому любая СУРБД должна иметь следующие особенности:

- набор логически связанных разделяемых данных;
- сохраняемые данные должны быть разбиты на некоторое количество фрагментов;
- между фрагментами может быть организована репликация данных;
- фрагменты и их реплики распределены по различным сайтам;
- сайты связаны между собой сетевыми соединениями;
- работа с данными на каждом сайте управляется СУБД;
- СУБД на каждом сайте способна поддерживать автономную работу локальных приложений;
- СУБД каждого сайта поддерживает хотя бы одно глобальное приложение.

Из определения СУРБД следует, что для конечного пользователя распределенность системы должна быть совершенно **прозрачна** (невидима). Другими словами, от пользователей должен быть полностью скрыт тот факт, что распределенная база данных состоит из нескольких фрагментов, которые могут размещаться на различных компьютерах и для которых, возможно, организована служба репликации данных.

Приведем двенадцать правил, сформулированных Дейтом (Date, 1987) для типичной СУРБД. Данные правила подобны двенадцати правилам Кодда для реляционных систем:

Правило 0. Основной принцип

С точки зрения конечного пользователя распределенная система должна выглядеть в точности так, как и обычная, нераспределенная система.

Правило 1. Локальная автономность

Сайты в распределенной системе должны быть автономными. В данном контексте автономность означает следующее:

- локальные данные принадлежат локальным владельцам и сопровождаются локально;
- все локальные процессы остаются чисто локальными;
- все процессы на заданном сайте контролируются только этим сайтом.

Правило 2. Отсутствие опоры на центральный сайт

В системе не должно быть ни одного сайта, без которого система не сможет функционировать. Это означает, что в системе не должно существовать центральных серверов таких служб, как управление транзакциями, выявление взаимных блокировок, оптимизация запросов и управление глобальным системным каталогом.

Правило 3. Локальная автономность

В идеале в системе никогда не должна возникать потребность в плановом останове ее функционирования для выполнения таких операций, как:

- добавление или удаление сайта из системы;
- динамическое создание или удаление фрагментов из одного или нескольких сайтов.

Правило 4. Независимость от расположения

Пользователь должен получать доступ к базе данных с любого из сайтов. Более того, пользователь должен получать доступ к любым данным так, как если бы они хранились на его сайте независимо от того, где они физически сохраняются.

Правило 5. Независимость от фрагментации

Пользователь должен получать доступ к данным независимо от способа их фрагментации.

Правило 6. Независимость от репликации

Пользователь не должен нуждаться в сведениях о наличии репликации данных. Это значит, пользователь не будет иметь средств для получения прямого доступа к конкретной копии элемента данных, а также не должен заботиться об обновлении всех имеющихся копий элемента данных.

Правило 7. Обработка распределенных запросов

Система должна поддерживать обработку запросов, ссылающихся на данные, расположенные на более чем одном сайте.

Правило 8. Обработка распределенных транзакций

Система должна поддерживать выполнение транзакций как единицы восстановления. Система должна гарантировать, что выполнение как глобальных, так и локальных транзакций будет происходить с сохранением четырех основных свойств транзакций, а именно: атомарности, согласованности, изолированности и продолжительности.

Правило 9. Независимость от типа оборудования

СУРБД должна быть способна функционировать на оборудовании с различными вычислительными платформами.

Правило 10. Независимость от операционной системы

Прямым следствием предыдущего правила является требование, согласно которому СУРБД должна быть способна функционировать под управлением различных операционных систем.

Правило 11. Независимость от сетевой архитектуры

СУРБД должна быть способна функционировать в сетях с различной архитектурой и типами носителей.

Правило 12. Независимость от типа СУБД

СУРБД должна быть способна функционировать поверх различных локальных СУБД, возможно, с разным типом используемой модели данных (иными словами, СУРБД должна поддерживать гетерогенность).

13.2. Преимущества и недостатки распределенных СУБД

Преимущества

Рассмотрим преимущества систем с распределенными базами данных перед традиционными централизованными системами баз данных:

– *отражение структуры организации.* Крупные организации, как правило, имеют множество отделений, которые могут находиться в разных концах страны и даже за ее пределами. При этом руководству компании может потребоваться выполнять глобальные запросы, предусматривающие получение доступа к данным, сохраняемым во всех существующих отделениях компании, а персонал отделения сможет выполнять необходимые ему локальные запросы;

– *разделяемость и локальная автономность.* Географическая распределенность организации может быть отражена в распределении ее данных, причем пользователи одного сайта смогут получить доступ к данным, сохраняемым на других сайтах. Данные могут быть помещены на тот сайт, на котором зарегистрированы пользователи, которые чаще всего их используют. В результате заинтересованные пользователи получают локальный контроль над требуемыми данными и могут устанавливать или регулировать локальные ограничения на их использование. Администратор глобальной базы данных (АБД) отвечает за систему в

целом. Как правило, часть этой ответственности делегируется на локальный уровень, благодаря чему АБД локального уровня получает возможность управлять локальной СУБД;

- *повышение доступности данных.* В централизованных СУБД отказ центрального компьютера вызывает прекращение функционирования всей СУБД. Однако отказ одного из сайтов СУРБД или линии связи между сайтами сделает недоступным лишь некоторые сайты, тогда как вся система в целом сохранит свою работоспособность. Распределенные СУБД проектируются таким образом, чтобы обеспечивать продолжение функционирования системы несмотря на подобные отказы. Если выходит из строя один из узлов, система сможет перенаправить запросы к отказавшему узлу в адрес другого сайта;

- *повышение надежности.* Если организована репликация данных, в результате чего данные и их копии будут размещены на более чем одном сайте, отказ отдельного узла или соединительной связи между узлами не приведет к недоступности данных в системе;

- *повышение производительности.* Если данные размещены на самом нагруженном сайте, который унаследовал от систем-предшественников высокий уровень параллельности обработки, то развертывание распределенной СУБД может способствовать повышению скорости доступа к базе данных (по сравнению с доступом к удаленной централизованной СУБД). Более того, поскольку каждый сайт работает только с частью базы данных, уровень использования центрального процессора и служб ввода/вывода может оказаться ниже, чем в случае централизованной СУБД;

- *экономические выгоды.* В настоящее время считается общепринятым положение, согласно которому намного дешевле собрать из небольших компьютеров систему, мощность которой будет эквивалентна мощности одного большого компьютера. Оказывается, что намного выгоднее устанавливать в подразделениях организации собственные маломощные компьютеры, кроме того, гораздо дешевле добавить в сеть новые рабочие станции, чем модернизировать систему с мейнфреймом. Второй потенциальный источник экономии имеет место в том случае, когда базы данных географически удалены друг от друга и приложения требуют осуществления доступа к распределенным данным. В этом случае из-за относительно высокой стоимости передаваемых по сети данных (по сравнению со стоимостью их локальной обработки) может оказаться экономически выгодным разделить приложение на соответствующие части и выполнять необходимую обработку на каждом из сайтов локально;

- *модульность системы.* В распределенной среде расширение существующей системы осуществляется намного проще. Добавление в сеть нового сайта не оказывает влияния на функционирование уже существующих. Подобная гибкость позволяет организации легко расширяться. Перегрузки из-за увеличения размера базы данных обычно устраняются путем добавления в сеть новых вычислительных мощностей и устройств дисковой памяти. В централизованных СУБД рост размера базы данных может потребовать замены и оборудования, и используемого программного обеспечения.

Недостатки

Рассмотрим недостатки систем с распределенными базами данных:

- *повышение сложности.* Распределенные СУБД, способные скрыть от конечных пользователей распределенную природу используемых ими данных и обеспечить необходимый уровень производительности, надежности и доступности, безусловно являются более сложными программными комплексами, чем централизованные СУБД. Тот факт, что данные могут подвергаться репликации, также добавляет дополнительный уровень сложности в программное обеспечение СУРБД. Если репликация данных не будет поддерживаться на требуемом уровне, система будет иметь более низкий уровень доступности данных, надежности и производительности, чем централизованные системы, а все изложенные выше преимущества превратятся в недостатки;

- *увеличение стоимости.* Увеличение сложности означает и увеличение затрат на приобретение и сопровождение СУРБД (по сравнению с обычными централизованными СУБД). Разворачивание распределенной СУБД потребует дополнительного оборудования, необхо-

димого для установки сетевых соединений между сайтами. Следует ожидать и роста расходов на оплату каналов связи, вызванных возрастанием сетевого трафика. Кроме того, возрастут затраты на оплату труда персонала, который потребуется для обслуживания локальных СУБД и сетевых соединений;

- *проблемы защиты*. В централизованных системах доступ к данным легко контролируется. Однако в распределенных системах потребуется организовать контроль доступа не только к данным, реплицируемым на нескольких различных сайтах, но и защиту сетевых соединений;

- *усложнение контроля за целостностью данных*. Целостность базы данных означает корректность и согласованность сохраняемых в ней данных. Требования обеспечения целостности обычно формулируются в виде некоторых ограничений, выполнение которых будет гарантировать защиту информации в базе данных от разрушения. Реализация ограничений поддержки целостности обычно требует доступа к большому количеству данных, используемых при выполнении проверок, но не требует выполнения операций обновления. В распределенных СУБД повышенная стоимость передачи и обработки данных может препятствовать организации эффективной защиты от нарушений целостности данных;

- *отсутствие стандартов*. Отсутствие стандартов на каналы связи и протоколы доступа к данным существенно ограничивает потенциальные возможности распределенных СУБД. Кроме того, не существует инструментальных средств и методологий, способных помочь пользователям преобразовать централизованные системы в распределенные;

- *недостаток опыта*. В настоящее время в эксплуатации находится уже несколько систем-прототипов и распределенных СУБД специального назначения, что позволит уточнить требования к используемым протоколам и установить круг основных проблем. Однако распределенные системы общего назначения еще не получили широкого распространения. Следовательно, еще не накоплен необходимый опыт промышленной эксплуатации распределенных систем, сравнимый с опытом эксплуатации централизованных систем. Такое положение дел является серьезным сдерживающим фактором для многих потенциальных сторонников данной технологии;

- *усложнение процедуры разработки базы данных*. Разработка распределенных баз данных, помимо обычных трудностей, связанных с процессом проектирования централизованных баз данных, требует принятия решения о фрагментации данных, распределении фрагментов по отдельным сайтам в организации процедур репликации данных.

13.3. Обеспечение прозрачности в СУРБД

В определении СУРБД утверждается, что система должна обеспечить прозрачность распределенного хранения данных для конечного пользователя. Под **прозрачностью** понимается сокрытие от пользователей деталей реализации системы.

Рассмотрим четыре основных типа прозрачности, которые могут иметь место в системе с распределенной базой данных:

- прозрачность распределенности;
- прозрачность транзакций;
- прозрачность выполнения;
- прозрачность использования СУБД.

Прозрачность распределенности

Прозрачность распределенности базы данных позволяет конечным пользователям воспринимать базу данных как единое логическое целое. Если СУРБД обеспечивает прозрачность распределенности, то пользователю не требуется каких-либо знаний о фрагментации данных (прозрачность фрагментации) или их размещения (прозрачность расположения).

- **прозрачность фрагментации.** пользователю не требуется знать, как именно фрагментированы данные, нет необходимости указывать имена фрагментов или расположение данных;

- **прозрачность расположения.** пользователь должен иметь сведения о способах фрагментации данных в системе, но не нуждается в сведениях о расположении данных. Основное преимущество база данных может быть подвергнута физической реорганизации и это не окажет никакого влияния на программы приложений;

- **прозрачность репликации.** пользователю не требуется иметь сведения о существующей репликации фрагментов. Под прозрачностью репликации подразумевается прозрачность расположения реплик.

- **прозрачность локального отображения.** пользователю необходимо указывать как имена используемых фрагментов, так и расположение соответствующих элементов данных. В данном случае запросы могут иметь более сложный вид и на подготовку требуется больше времени, чем в предыдущих случаях.

- **прозрачность именования.** СУРБД должна гарантировать, что никакие два сайта системы не смогут создать некоторый объект базы данных, имеющий одно и то же имя.

Прозрачность транзакций

Прозрачность транзакций в среде распределенных СУБД означает, что при выполнении любых распределенных транзакций гарантируется сохранение целостности и согласованности распределенной базы данных. **Распределенная транзакция** осуществляет доступ к данным, сохраняемым более чем в одном местоположении. Каждая из транзакций разделяется на несколько **субтранзакций** – по одной для каждого сайта, к данным которого осуществляется доступ. На удаленных сайтах субтранзакции представляются **агентами**.

Прозрачность параллельности обеспечивается СУРБД в том случае, если результаты всех параллельно выполняемых транзакций идентичны тем, если бы все эти транзакции выполнялись последовательно в некотором произвольном порядке.

Рассмотрим проблемы, связанные с **прозрачностью отказов**. Напомним, что в централизованной СУБД имеют место следующие типы отказов: сбои системы; отказ носителей; ошибки в программах; небрежность персонала; стихийные бедствия и действия злоумышленников. В распределенной среде СУБД должна учитывать дополнительные факторы:

- возможность потери сообщения;
- возможность отказа линия связи;
- аварийный останов одного из сайтов;
- расчленение сетевых соединений.

Прозрачность выполнения

Прозрачность выполнения требует, чтобы работа в среде СУРБД выполнялась точно так же, как и в среде централизованной СУБД. В распределенной среде работа системы не должна демонстрировать никакого снижения производительности, связанного с ее распределенной архитектурой, например, с присутствием медленных сетевых соединений.

Обработчик распределенных запросов должен выяснить:

- к какому фрагменту следует обратиться;

- какую копию фрагмента использовать, если его данные реплицируются;
- какое из местоположений должно использоваться.

Обработчик распределенных запросов вырабатывает стратегию выполнения, которая является оптимальной с точки зрения некоторой стоимостной функции. Обычно распределенные запросы оцениваются по таким показателям:

- время доступа, включающее физический доступ к данным на диске;
 - время работы центрального процессора, затрачиваемое на обработку данных в первичной памяти;
 - время, необходимое для передачи данных по сетевым соединениям.
- систем. Как правило, это один из самых сложных в реализации типов прозрачности.

ГЛАВА 14. ВВЕДЕНИЕ В ОБЪЕКТНЫЕ СУБД

Специализированные приложения баз данных

Рассмотрим специализированные приложения, чьи требования сильно отличаются от требований традиционных бизнес-приложений и для которых использование реляционных СУБД становится неадекватным:

- автоматизированное проектирование;
- автоматизированное производство;
- автоматизированная разработка программного обеспечения;
- офисные информационные системы и системы мультимедиа;
- цифровое издательское дело;
- геоинформационные системы.

Основные концепции объектно-ориентированного подхода

Абстракция, инкапсуляция и сокрытие информации

Определение. Абстракция – это процесс идентификации наиболее важных аспектов сущности и игнорирование всех остальных ее малозначащих свойств.

Двумя основными аспектами абстракции являются инкапсуляция и сокрытие информации.

Определение. Понятие **инкапсуляции** означает, что объект содержит как структуру данных, так и набор операций, с помощью которых этой структурой можно манипулировать.

Определение. Концепция **сокрытия информации** означает, что все внешние аспекты объекта отделяются от подробностей его внутреннего устройства, скрытых от внешнего мира.

В некоторых ООЯП инкапсуляция достигается за счет использования **абстрактных типов данных** (Abstract Data Types – ADT). В этом представлении объект состоит из интерфейса и реализации. В представлении базы данных должная инкапсуляция достигается благодаря тому, что программисты обладают правом доступа только к интерфейсу. Таким образом, инкапсуляция обеспечивает *логическую независимость* от данных: внутреннюю реализацию ADT можно изменять, не затрагивая приложения, которые используют этот ADT.

Объекты и атрибуты

Определение. Объект – это уникально идентифицируемая сущность, которая содержит атрибуты, описывающие состояния объектов «реального мира», и связанные с ними действия.

Текущее состояние объекта описывается одним или несколькими **атрибутами** или **переменными экземпляра** (instance variables). Атрибуты могут быть простыми и составными. **Простой атрибут** может иметь примитивный тип (целое число, строка, действительное число и т. д.) и принимать литеральное значение. **Составной атрибут** может содержать коллекции и/или ссылки. **Ссылочный атрибут** представляет связь между объектами.

Идентификация объектов

Ключевой частью определения объекта является уникальность его идентификации. В объектно-ориентированной системе каждому объекту в момент его создания присваивается **идентификатор объекта** (Object Identifier – OID), который обладает следующими свойствами:

- генерируется системой;
- уникально обозначает этот объект;
- инвариантен в том смысле, что его нельзя изменить во время жизненного цикла программы: после создания объекта его OID-идентификатор не может быть использован повторно ни для какого другого объекта, даже после удаления данного объекта;
- не зависит от значений его атрибутов (т. е. от текущего состояния, два объекта могут иметь одинаковое состояние, но всегда обладают разными OID-идентификаторами);
- скрыт от пользователя (в идеале).

Таким образом, идентичность гарантируется тем, что объект всегда можно единственным образом идентифицировать, что автоматически гарантирует целостность сущностей. Существует несколько способов реализации идентификации объектов. В реляционных СУБД идентичность объектов *основана на первичных ключах*. Первичные ключи не обеспечивают того уровня идентичности объекта, который требуется в объектно-ориентированных системах. Во-первых, первичный ключ является уникальным внутри отношения, но не для всей системы. Во-вторых, первичный ключ обычно выбирается на основании атрибутов данного отношения, что означает его зависимость от текущего состояния объекта.

Укажем преимущества использования OID-идентификаторов в качестве идентификаторов объектов:

- *эффективность*. Для хранения OID-идентификаторов внутри составного объекта требуется очень мало места. Обычно они меньше текстовых имен, внешних ключей или семантических ссылок;
- *быстрота*. OID-идентификатор указывает на фактический адрес или место внутри таблицы, в котором находится адрес данного объекта. Это значит, что объекты могут быть быстро обнаружены, независимо от места их текущего хранения: в оперативной памяти или на жестком диске;
- *невозможность изменения пользователем*. Если OID-идентификаторы генерируются системой и скрыты от пользователей или, по крайней мере, доступны только для чтения, то в такой системе проще гарантируется целостность сущностей и связей. Более того, это позволяет пользователю не заботиться о поддержании целостности данных;
- *независимость от содержания данных*. OID-идентификаторы никак не зависят от данных, которые содержатся в данном объекте. Это позволяет изменять значение каждого атрибута объекта, но при этом данный объект остается тем же объектом с прежним OID-идентификатором.

Методы и сообщения

Объект инкапсулирует данные и функции в замкнутом пакете. В объектной технологии функции обычно называются **методами**, которые защищают от внешнего мира расположенные внутри атрибуты. **Сообщения** являются средством взаимодействия объектов. Сообщение представляет собой запрос, направленный одним объектом (отправителем) в адрес другого объекта (получателя) и требующий, чтобы объект-получатель выполнил один из своих методов.

Подклассы, суперклассы и наследование

Некоторые объекты могут иметь подобные, но не идентичные атрибуты и методы. Если степень такого подобия достаточно высока, то имеет смысл совместно использовать некоторые свойства (атрибуты и методы). Наследование (inheritance) позволяет определять один класс на основе более общего класса. Такие менее общие классы называются **подклассами**, а

более общие – **суперклассами**. Процесс образования суперкласса называется **обобщением** (geheralization), а процесс образования подкласса – **специализацией**. По умолчанию подкласс наследует все свойства его суперкласса и в дополнение к ним определяет свои собственные уникальные свойства.

Существует несколько видов наследования: единичное (single inheritance), множественное (multiple inheritance), повторное (repeated inheritance) и избирательное (selective inheritance). Термин «**единичное наследование**» означает, что подклассы наследуют не более чем от одного суперкласса. Организация механизма **множественного наследования** может оказаться очень проблематичной, потому что он должен обеспечить разрешение конфликтов, которые возникают в случаях, когда суперклассы содержат одинаковые атрибуты или методы. Не во всех объектно-ориентированных языках и системах баз данных принципиально поддерживается множественное наследование. **Повторное наследование** – это особый случай множественного наследования, в котором суперклассы происходят от общего суперкласса. **Селективное наследование** позволяет подклассу наследовать ограниченное количество свойств его суперкласса.

Полиморфизм и динамическое связывание

Перегрузка (overloading) позволяет повторно использовать имя метода внутри определения класса и определений нескольких классов. Перегрузка является частным случаем более общего понятия **полиморфизма** (polymorphism). Существуют три типа полиморфизма: рабочий, включения и параметрический. Перегрузка, основанная на том, что многие классы могут иметь один и тот же метод, называется **рабочим полиморфизмом** (operation polymorphism). Метод, определенный в суперклассе и унаследованный в его подклассе, является примером **полиморфизма включения** (inclusion polymorphism). **Параметрический полиморфизм** (parametric polymorphism) или **универсальность** (genericity) означает использование типов в качестве параметров в объявлениях универсального типа или класса.

Процесс выбора соответствующего метода, основанный на типе объекта, называется **связыванием** (binding). Если определение типа объекта может быть отложено до наступления времени исполнения (а не во время компиляции), то такой выбор называется **динамическим** (dynamic) или **поздним** (late) **связыванием**.

ГЛАВА 15. ОБЪЕКТНО-ОРИЕНТИРОВАННЫЕ СУБД

15.1. Введение в объектно-ориентированные модели данных и СУБД

Рассмотрим различные определения, которые были предложены для объектно-ориентированной модели данных. В частности, Ким предложил следующие определения для объектно-ориентированной модели данных (ООМД), объектно-ориентированной базы данных (ООБД) и объектно-ориентированной СУБД (ООСУБД) (Kim, 1991).

Определение. ООМД – это (логическая) модель данных, которая учитывает семантику объектов, применяемую в объектно-ориентированном программировании.

Определение. ООБД – это перманентный, совместно используемый набор (коллекция) объектов, определенный средствами ООМД.

Определение. ООСУБД – это система управления (менеджер) ООБД.

Эти определения очень ненаглядны и явно отражают тот факт, что не существует никакой объектно-ориентированной модели данных, эквивалентной базовой модели данных для реляционных систем. В каждой системе предлагается своя собственная интерпретация базовой функциональности. Например, Здоник и Майер предложили следующую минимальную модель, которой обязательно должна удовлетворять любая ООСУБД (Zdonik and Maier, 1990):

1. Обеспечивать функциональность базы данных.
2. Поддерживать идентичность объектов.

3. Обеспечивать инкапсуляцию.
4. Поддерживать объекты со сложным состоянием.

Авторы утверждают, что несмотря на возможную полезность механизм наследования не является существенной частью определения, а потому объектно-ориентированная СУБД вполне могла бы существовать и без него. С другой стороны, Хошафян и Абноус предложили собственное определение объектно-ориентированной СУБД (Khoshafian and Abnous, 1990):

1. «Объектно-ориентированный подход» = «абстрактные типы данных» + «наследование» + «идентичность объектов».
2. «Объектно-ориентированная СУБД» = «объектно-ориентированный подход» + «возможности базы данных».

Приведем еще одно определение ООСУБД, построенное посредством указания ее обязательных компонентов (Parsaye, 1989):

1. Высокоуровневый язык запросов со средствами оптимизации, реализованными в базовой системе.
2. Поддержка перманентности и сохранение атомарности транзакций, обеспечиваемые механизмами управления параллельностью и восстановлением.
3. Поддержка сложных объектных хранилищ, индексов и методов доступа, предназначенных для быстрого и эффективного извлечения данных.
4. «Объектно-ориентированная СУБД» = «объектно-ориентированная система» + «условия пунктов 1, 2 и 3».

15.2. Манифест объектно-ориентированных СУБД

В манифесте ООСУБД (Atkinson, 1989) предлагается тринадцать обязательных характеристик, которым должна отвечать любая ООСУБД. Их выбор основан на двух критериях: система должна быть объектно-ориентированной и представлять собой СУБД. Первые восемь правил относятся к критерию объектной ориентированности, остальные – к характеристикам СУБД.

Правило 1. Поддержка составных объектов

В системе должна быть предусмотрена возможность создания составных объектов за счет применения конструкторов основных объектов. Минимальный набор включает конструкторы типов SET, TUPLE и LIST (или ARRAY).

Правило 2. Поддержка идентичности объектов

Все объекты должны иметь уникальный идентификатор, который не зависит от значений их атрибутов.

Правило 3. Поддержка инкапсуляции

В ООСУБД корректная инкапсуляция достигается за счет того, что программисты обладают правом доступа только к спецификации интерфейсов методов, а данные и реализация методов скрыты внутри объекта. Однако могут быть случаи, когда введение инкапсуляции не требуется, например, в произвольных запросах.

Правило 4. Поддержка типов или классов

В данном манифесте требуется, чтобы в ООСУБД поддерживалась только одна из этих концепций. Схема базы данных в объектно-ориентированной системе включает набор классов или набор типов.

Правило 5. Поддержка наследования типов или классов от их предков

Подтип или подкласс должен наследовать атрибуты и методы от его супертипа или суперкласса соответственно.

Правило 6. Поддержка динамического связывания

Методы должны применяться к объектам разных типов (перегрузка). Реализация метода должна зависеть от типа объекта, к которому данный метод применяется (переопределе-

ние). Для обеспечения этой функциональности связывание имен методов в системе не должно выполняться до времени выполнения программы (динамическое связывание).

Правило 7. Язык DML должен обладать вычислительной полнотой

Иными словами, язык манипулирования данными (DML) в ООСУБД должен быть языком программирования общего назначения.

Правило 8. Набор типов данных должен быть расширяемым

Пользователь должен иметь средства создания новых типов данных на основе набора предопределенных системных типов. Более того, между способами использования системных и пользовательских типов данных не должно быть никаких различий.

Правило 9. Поддержка перманентности данных

Как и в обычной СУБД, данные должны существовать даже после завершения работы того приложения, которое их создало. При этом пользователь не должен явным образом перемещать или копировать данные с целью обеспечения их перманентности.

Правило 10. Поддержка очень больших баз данных

В обычной СУБД существуют механизмы эффективного управления вторичными устройствами хранения, например, индексы и буферы. ООСУБД должна иметь аналогичные, скрытые от пользователя механизмы, обеспечивающие полную независимость логических и физических уровней системы.

Правило 11. Поддержка параллельной работы многих пользователей

В ООСУБД должны быть предусмотрены механизмы управления параллельностью, аналогичные механизмам, применяемым для этой цели в традиционных системах.

Правило 12. Обеспечение восстановления после сбоев аппаратного и программного обеспечения

Правило 13. Предоставление простых способов создания запросов к данным

ООСУБД должна предоставлять высокоуровневые (т. е. в той или иной степени декларативные), эффективные (т. е. пригодные для оптимизации) и независимые от приложений средства создания произвольных запросов..

В этом манифесте также предлагается несколько необязательных функциональных возможностей: множественное наследование, контроль типов и логический вывод типов, распределенная обработка в сети, проектирование транзакций, а также поддержка версий. Интересно, что при этом открыто не упоминается поддержка безопасности или механизма представлений, а также полностью декларативный язык запросов не является обязательным.

15.3. Преимущества и недостатки ООСУБД

На базе использования ООСУБД можно предложить адекватные решения для многих типов специализированных приложений, однако они также обладают и некоторыми недостатками. Рассмотрим основные преимущества, свойственные ООСУБД:

– *улучшение возможности моделирования*. Объектно-ориентированная модель данных позволяет более точно моделировать реальный мир. Объект, который инкапсулирует состояние и поведение, является более естественным и реалистическим представлением объектов окружающего мира. Объект может хранить все связи, которыми он связан с другими объектами, включая связи типа «многие ко многим», а сами объекты могут быть преобразованы в сложные объекты, с которыми очень трудно работать с помощью традиционных моделей данных;

– *расширяемость*. В ООСУБД допускается создание новых абстрактных типов данных на основе существующих типов. Способность группировать общие свойства нескольких классов и образовывать на их основе совместно используемые суперклассы может существенно сократить избыточность систем, а потому рассматривается как одно из основных преимуществ объектно-ориентированного подхода. Повторное использование классов способствует ускорению разработки и упрощению сопровождения базы данных и ее приложений;

– *устранение проблемы несоответствия*. Простой интерфейс между языком манипулирования данными (DML) и языком программирования позволяет устранить проблему их несоответствия. При этом во многом исключается неэффективность, связанная с отображением декларативных языков, подобных SQL, на императивные языки, подобные С;

– *более выразительный язык запросов*. Навигационный способ доступа от одного объекта к другому, соседнему объекту, является наиболее распространенной формой доступа к данным в языке SQL. Навигационный способ доступа больше подходит для обработки случаев лавинообразного увеличения количества частей, рекурсивных запросов и т. д. Однако при этом часто можно услышать, что ООСУБД привязаны к некоторому языку программирования, который, несмотря на его удобство для программистов, обычно не используется конечными пользователями системы, нуждающимися в декларативном языке. Признавая это, в новом стандарте группы ODMG определен декларативный язык запросов, построенный на объектно-ориентированной форме языка SQL;

– *поддержка эволюции схемы*. Строгая связь между данными и приложениями в ООСУБД делает эволюцию схемы более гибкой. Механизмы генерализации и наследования позволяют создать более структурированную и интуитивно понятную схему, а также точнее выразить семантику приложения;

– *поддержка долговременных транзакций*;

– *применимость для сложных специализированных приложений баз данных*. Существуют прикладные области, в которых применение традиционных СУБД не позволяет достичь успеха, например, в автоматизированном проектировании (CAD), автоматизированной разработке программ (CASE), в офисных информационных системах (OIS) и мультимедиа-системах. Заложенные в ООСУБД возможности моделирования делают их особенно удобными для применения в этих классах приложений;

– *повышенная производительность*. Универсальным средством измерения производительности ООСУБД является методика тестирования Object Operations Version 1 (OO1) (Cattell and Skeen, 1992), разработанная для имитации операций, наиболее распространенных в сложных специализированных инженерных приложениях (например, для поиска всех частей, связанных с произвольно выбранной частью, затем всех частей, связанных с одной из этих частей, и т. д., вплоть до семи уровней вложенности). В 1989 и 1990 годах тестирование по методике OO1 было проведено для ООСУБД GemStone, Ontos, ObjectStore, Objectivity/DB и Versant, а также для РСУБД INGRES и Sybase. Результаты тестирования показали в среднем 30-кратное превосходство ООСУБД над РСУБД.

Прежде чем перейти к рассмотрению недостатков, стоит указать, что при правильной реализации доменов реляционные системы в состоянии обеспечить те же функциональные возможности, которыми должны обладать ООСУБД. Домен может рассматриваться как тип данных произвольной сложности со скалярными величинами, которые инкапсулированы и могут обрабатываться только предопределенными функциями. Следовательно, заданный для данного домена атрибут в реляционной модели может содержать любые данные, например, чертежи, документы, изображения, массивы и т. д.

Рассмотрим основные недостатки, свойственные ООСУБД:

– *отсутствие универсальной модели данных*. Для объектно-ориентированной модели данных не существует универсальной общепринятой модели данных, и большинство используемых моделей не имеет теоретического обоснования. Этот недостаток имеет очень большое значение и приравнивает ООСУБД к дореляционным системам. Однако группа ODMG предложила объектную модель, которая стала *фактическим* стандартом для ООСУБД;

– *недостаточность опыта эксплуатации*. Использование ООСУБД все еще очень ограничено. Это значит, что для них отсутствует такой же опыт работы, какой имеется у традиционных систем. Объектно-ориентированные системы все еще в большей степени отвечают требованиям программиста, чем неопытного конечного пользователя;

– *отсутствие стандартов*. Отсутствие стандартов – общий недостаток ООСУБД. Для них не существует общепринятой модели данных и не существует стандартного объектно-

ориентированного языка запросов. Группа ODMG предложила спецификацию языка объектных запросов OQL (Object Query Language), который фактически стал стандартом (по крайней мере, на короткий срок);

- *влияние оптимизации запросов на инкапсуляцию*. Для оптимизации запросов и организации эффективного доступа к базе данных необходимо обладать знаниями об особенностях реализации. Однако при этом нарушается принцип инкапсуляции;

- *влияние блокировки на уровне объекта на производительность*. Во многих СУБД блокировка используется как основа построения протокола управления параллельностью. Однако применение блокировки на уровне объекта может вызвать проблемы с блокировкой в отношении иерархии наследования, а также оказать существенное влияние на общую производительность системы;

- *сложность*. Повышенные функциональные возможности ООСУБД, например, иллюзия одноуровневой модели хранения, подстановка указателей, обработка долговременных транзакций, управление версиями и эволюция схемы, по определению более сложные, чем функциональные возможности традиционных СУБД. Повышенная сложность систем приводит к созданию более дорогих и трудных в использовании продуктов;

- *отсутствие поддержки представлений*. В настоящее время в большинстве ООСУБД не предусмотрено использование механизма представлений, которые обладают такими преимуществами, как независимость от данных, безопасность, пониженная сложность и возможность настройки;

- *недостаточность средств обеспечения безопасности*. На данный момент в ООСУБД не реализованы адекватные механизмы обеспечения безопасности. Большинство механизмов основано на высоком уровне детализации блокировок, причем, пользователь не может предоставлять права доступа к отдельным объектам или классам.

ГЛАВА 16. ХРАНИЛИЩА ДАННЫХ

Реляционные СУБД проектировались для управления большим потоком транзакций, каждая из которых сопровождалась внесением незначительных изменений в оперативные данные предприятия. Системы подобного типа называются системами оперативной обработки транзакций или OLTP-системами (OnLine Transaction Processing – OLTP). Размер баз данных для OLTP-систем может варьироваться от нескольких мегабайт до нескольких терабайт или даже петабайт.

Лицам, ответственным за принятие корпоративных решений, необходимо иметь доступ ко всем данным организации независимо от их расположения. Для выполнения полного анализа деятельности организации, определения ее бизнес-показателей, выяснения характеристик существующего спроса и тенденций его изменения необходимо иметь доступ не только к текущим данным, но и к ранее накопленным (историческим) данным. Для упрощения подобного анализа была разработана концепция *хранилища данных* (data warehouse). Предполагается, что такое хранилище содержит сведения, поступающие из самых разных источников данных, функционирующих под управлением разных операционных систем, а также различные накопительные и сводные данные. Однако лицам, ответственным за принятие корпоративных решений, необходимо иметь мощные инструменты анализа накопленных данных. Основными средствами анализа в последние годы стали инструменты оперативной аналитической обработки (OnLine Analytical Processing – OLAP) и инструменты разработки данных (data mining).

16.1. Введение в хранилища данных

Концепции хранилищ данных

Исходная концепция хранилища данных была предложена специалистами фирмы IBM в виде «информационного хранилища» и первоначально представлена ими как решение, обеспечивающее доступ к данным, накопленным в нереляционных системах. Предполага-

лось, что такое информационное хранилище позволит организациям использовать их архивы данных для эффективного решения бизнес-задач. Однако из-за чрезвычайной сложности и невысокой производительности подобных систем, созданных на начальном этапах, первые попытки создания информационных хранилищ в основном были отвергнуты. С тех пор к концепции хранилищ информации возвращались вновь и вновь, но только в последние годы потенциал технологии хранилищ данных стал рассматриваться как достаточно ценное и жизнеспособное решение. Основателем хранилищ данных считается Билл Инмон (Bill Inmon).

Определение. Хранилище данных – это предметно-ориентированный, интегрированный, привязанный ко времени и неизменяемый набор данных, предназначенный для поддержки принятия решений.

В приведенном определении Инмона (Inmon, 1993) указанные характеристики данных понимаются следующим образом:

- **предметная ориентированность.** Хранилище данных организовано вокруг основных предметов или субъектов организации (например, клиенты, товары, продажи), а не вокруг прикладных областей деятельности (выписка счета клиенту, контроль товарных запасов, продажа товаров). Это свойство отражает необходимость хранения данных, предназначенных для поддержки принятия решений, а не обычных оперативно-прикладных данных;

- **интегрированность.** Смысл этой характеристики состоит в том, что оперативно-прикладные данные обычно поступают из разных источников, которые часто имеют несогласованное представление одних и тех же данных (например, используют разный формат). Для предоставления пользователю единого обобщенного представления данных необходимо создать интегрированный источник, обеспечивающий согласованность хранимой информации;

- **привязка ко времени.** Данные в хранилище точны и корректны только в том случае, когда они привязаны к некоторому моменту или промежутку времени. Привязка хранилища данных ко времени следует из большой длительности того периода, за который была накоплена сохраняемая в нем информация из явной или неявной связи временных отметок со всеми сохраняемыми данными, а также из того факта, что хранимая информация фактически представляет собой набор моментальных снимков состояния данных;

- **неизменяемость.** Это означает, что данные не обновляются в оперативном режиме, а лишь регулярно пополняются за счет информации из оперативных систем обработки. При этом новые данные никогда не заменяют прежние, а лишь дополняют их. Таким образом, база данных хранилища постоянно пополняется новыми данными, последовательно интегрируемыми с уже накопленной информацией.

Конечной целью создания хранилища данных является интеграция корпоративных данных в едином репозитории, обращаясь к которому пользователи смогут составлять запросы, генерировать отчеты и выполнять анализ данных. Хранилище данных является рабочей средой для систем поддержки принятия решений, которая извлекает данные, хранимые в различных оперативных источниках, организует их и передает лицам, ответственным за принятие решений в данной организации. Таким образом, можно сказать, что технология хранилищ данных – это технология управления данными и их анализа.

Преимущества технологии хранилищ данных

При успешной реализации хранилища данных в организации могут быть достигнуты определенные преимущества:

- *потенциально высокая отдача от инвестиций.* В случае применения данной технологии организации потребуется инвестировать значительные средства для того, чтобы гарантировать успешную реализацию проекта. В зависимости от используемых технических решений необходимая сумма инвестиций может варьироваться от 50 000 до 10 000 000 фунтов стерлингов. Однако по данным фирмы International Data Corporation (IDC) в 1996 году усредненная за три года прибыль на инвестированный капитал (ROI-прибыль – Return On Investment) в сфере хранилищ данных составила 401 %;

– *повышение конкурентоспособности*. Огромные прибыли на инвестированный капитал фирм, которые успешно применили технологию хранилищ данных, стали доказательством существенного повышения конкурентоспособности. Повышение конкурентоспособности достигается за счет того, что лица, ответственные за принятие решений в данной организации, получают доступ к ранее недоступной, неизвестной и никогда не использовавшейся информации, например, о клиентах, тенденциях рынка и спросе;

– *повышение эффективности труда лиц, ответственных за принятие решений*. Технология хранилищ данных повышает эффективность труда лиц, ответственных за принятие решений в данной организации, за счет создания интегрированной базы данных, состоящей из непротиворечивой, предметно-ориентированной и охватывающей обширный временной интервал информации. В этой базе данные, выбранные из нескольких, как правило, несовместимых между собой оперативных систем, интегрированы в форме, позволяющей получить единое, развернутое во времени представление о деятельности организации. Преобразуя исходные данные в осмысленную информацию, хранилище данных позволяет руководящему звену выполнить более содержательный, точный и согласованный анализ деятельности предприятия.

Сравнение OLTP-систем и хранилищ данных

СУБД, созданная для поддержки оперативной обработки транзакций (OLTP), обычно рассматривается как непригодная для организации хранилищ данных, поскольку к этим двум типам систем предъявляются совершенно разные требования. Например, OLTP-системы проектируются с целью обеспечения максимально интенсивной обработки фиксированных транзакций, тогда как хранилища данных – прежде всего для обработки единичных *произвольных запросов* (ad hoc query). В табл. 30.1 сравниваются основные характеристики типичных OLTP-систем и хранилищ данных (Singh, 1997).

Таблица 30.1

Сравнение основных характеристик типичной OLTP-системы и хранилища данных

OLTP-система	Хранилище данных
Содержит текущие данные	Содержит исторические данные
Хранит подробные сведения	Хранит подробные сведения, а также частично и значительно обобщенные данные
Данные являются динамическими	Данные, в основном, являются статическими
Повторяющийся способ обработки	Нерегламентированный, неструктурированный и эвристический способ обработки данных
Высокая интенсивность обработки транзакций	Средняя и низкая интенсивность обработки транзакций
Предсказуемый способ использования данных	Непредсказуемый способ использования данных
Предназначена для обработки транзакций	Предназначена для проведения анализа
Ориентирована на прикладные области	Ориентирована на предметные области
Поддержка принятия повседневных решений	Поддержка принятия стратегических решений
Обслуживает большое количество работников исполнительного звена	Обслуживает относительно малое количество работников руководящего звена

Организация обычно имеет несколько различных OLTP-систем, предназначенных для поддержки таких бизнес-процессов, как контроль товарных запасов, выписка счетов клиентам, продажа товаров. Эти системы генерируют оперативные данные, которые являются очень подробными, текущими и подверженными изменениям. OLTP-системы оптимизирова-

ны для интенсивной обработки транзакций, которые проектируются заранее, многократно повторяются и связаны преимущественно с обновлением данных. В соответствии с этими особенностями данные в OLTP-системах организованы согласно требованиям конкретных бизнес-приложений и позволяют принимать повседневные решения большому количеству параллельно работающих пользователей-исполнителей.

Однако в организации имеется только одно хранилище данных, которое содержит исторические, подробные, обобщенные до определенной степени и практически неизменяемые данные (т. е. новые данные могут только добавляться). Хранилища данных предназначены для обработки относительно небольшого количества транзакций, которые имеют непредсказуемую природу и требуют ответа на произвольные, неструктурированные и эвристические запросы. Информация в хранилище данных организована в соответствии с требованиями возможных запросов и предназначена для поддержки принятия долговременных стратегических решений относительно небольшим количеством руководящих работников.

Хотя OLTP-системы и хранилища данных имеют совершенно разные характеристики и создаются для различных целей, все же они тесно связаны в том смысле, что OLTP-системы являются источником информации для хранилища данных. Основная проблема при организации этой связи заключается в том, что поступающие из OLTP-систем данные могут быть несогласованными, фрагментированными, подверженными изменениям, содержащими дубликаты или пропуски. Поэтому до помещения в хранилище данные должны быть «очищены».

OLTP-системы не предназначены для получения быстрого ответа на произвольные запросы. Они также не используются для хранения устаревших исторических данных, которые требуются для анализа тенденций. OLTP-системы, в основном, поставляют огромное количество «сырых» данных, которые с трудом поддаются анализу. С помощью хранилищ данных можно получить ответы на более сложные запросы, например, «Какова средняя выручка от продажи товара в каждом отделении компании в сравнении с аналогичными показателями прошлого года?», «Какая связь наблюдается между ежегодной выручкой в каждом отделении компании и общим количеством торговых агентов в каждом из этих отделений?».

Проблемы хранилищ данных

Рассмотрим принципиальные проблемы, связанные с обработкой и сопровождением хранилищ данных:

- *недооценка ресурсов, необходимых для загрузки данных.* Многие разработчики склонны недооценивать время, необходимое для извлечения, очистки и загрузки данных в хранилище. На выполнение этого процесса может потребоваться до 80 % общего времени разработки (Inmon, 1990), хотя эту долю можно существенно сократить при использовании более совершенных инструментов очистки и сопровождения данных;

- *скрытые проблемы источников данных.* Скрытые проблемы, связанные с источниками данных, поставляющими информацию в хранилище, могут быть обнаружены только спустя несколько лет после начала их эксплуатации. При этом разработчику придется принять решение об устранении возникших проблем в хранилище данных и/или в источниках данных (например, некоторые поля могут остаться незаполненными – NULL);

- *отсутствие требуемых данных в имеющихся архивах.* В хранилищах данных часто возникает потребность получить некоторые сведения, которые не учитывались в оперативных системах, служащих источниками данных. В таком случае организация должна решить, стоит ли ей модифицировать существующие OLTP-системы или же лучше создать новую систему по сбору недостающих данных;

- *повышение требований конечных пользователей.* После того, как конечные пользователи получают в свое распоряжение инструменты составления запросов и отчетов, их требования к помощи и консультациям сотрудников информационной службы организации скорее возрастут, чем сократятся. Это вызвано тем, что пользователи хранилища данных начинают в большей степени осознавать истинные возможности и значение этого инструмента. Дан-

ную проблему можно частично разрешить, используя менее мощные, но простые и удобные инструменты, или уделяя большее внимание обучению пользователей. Еще одной причиной увеличения нагрузки на сотрудников информационной службы организации является то, что сразу после запуска хранилища данных возрастает количество пользователей и запросов, причем, сложность запросов также существенно возрастает;

- *гомогенизация данных*. Создание крупномасштабного хранилища данных может быть связано с решением серьезной задачи гомогенизации данных, что в итоге способно уменьшить ценность собранной информации;

- *высокие требования к ресурсам*. Для хранилища данных может потребоваться огромный объем дисковой памяти. Для многих реляционных систем поддержки принятия решений используются схемы типа «звезда», «снежинка» или «звезда – снежинка». Все эти варианты приводят к созданию очень больших таблиц с фактическими данными или таблиц фактов. При наличии множества размерностей фактических данных для хранения таблиц фактов вместе с итоговыми данными и индексами может потребоваться гораздо больше места, чем для хранения исходных необработанных данных;

- *владение данными*. Создание хранилища данных может потребовать изменить статус конечных пользователей в отношении прав владения данными. Наиболее критичные данные, которые ранее были доступны для просмотра и использования только отдельными подразделениями организации, занятыми в определенных бизнес-сферах (продажа или маркетинг), теперь потребуются сделать доступными и другим сотрудникам организации;

- *сложное сопровождение*. Хранилища данных обычно характеризуются сложностью сопровождения, поскольку любая реорганизация бизнес-процессов или источников данных может повлиять на происходящие в них процессы. Для того чтобы хранилище данных всегда оставалось ценным ресурсом, необходимо, чтобы оно постоянно полностью соответствовало организации, работу которой оно поддерживает;

- *долговременный характер проектов*. Хранилище данных представляет собой единый информационный ресурс организации. Однако для его создания может потребоваться до трех лет, а потому многие организации строят также свои собственные магазины. Магазины данных (data marts) предназначены для поддержки работы только какого-то одного подразделения организации или одной ее прикладной области, а потому создать их можно гораздо быстрее;

- *сложность интеграции*. Наиболее важной частью процесса сопровождения хранилища данных являются его интеграционные возможности. Это значит, что организация должна потратить значительное количество времени для того, чтобы определить, насколько хорошо могут интегрироваться различные инструменты хранилища данных для получения искомого общего решения. Это довольно трудная задача, потому что для выполнения различных операций с хранилищем данных могут использоваться самые разные инструменты, которые должны слаженно совместно работать на пользу всей организации в целом.

16.2. Архитектура хранилища данных

Типичная архитектура хранилища данных показана на рис. 16.1. Исходные данные, помещаемые в хранилище, поступают из следующих источников:

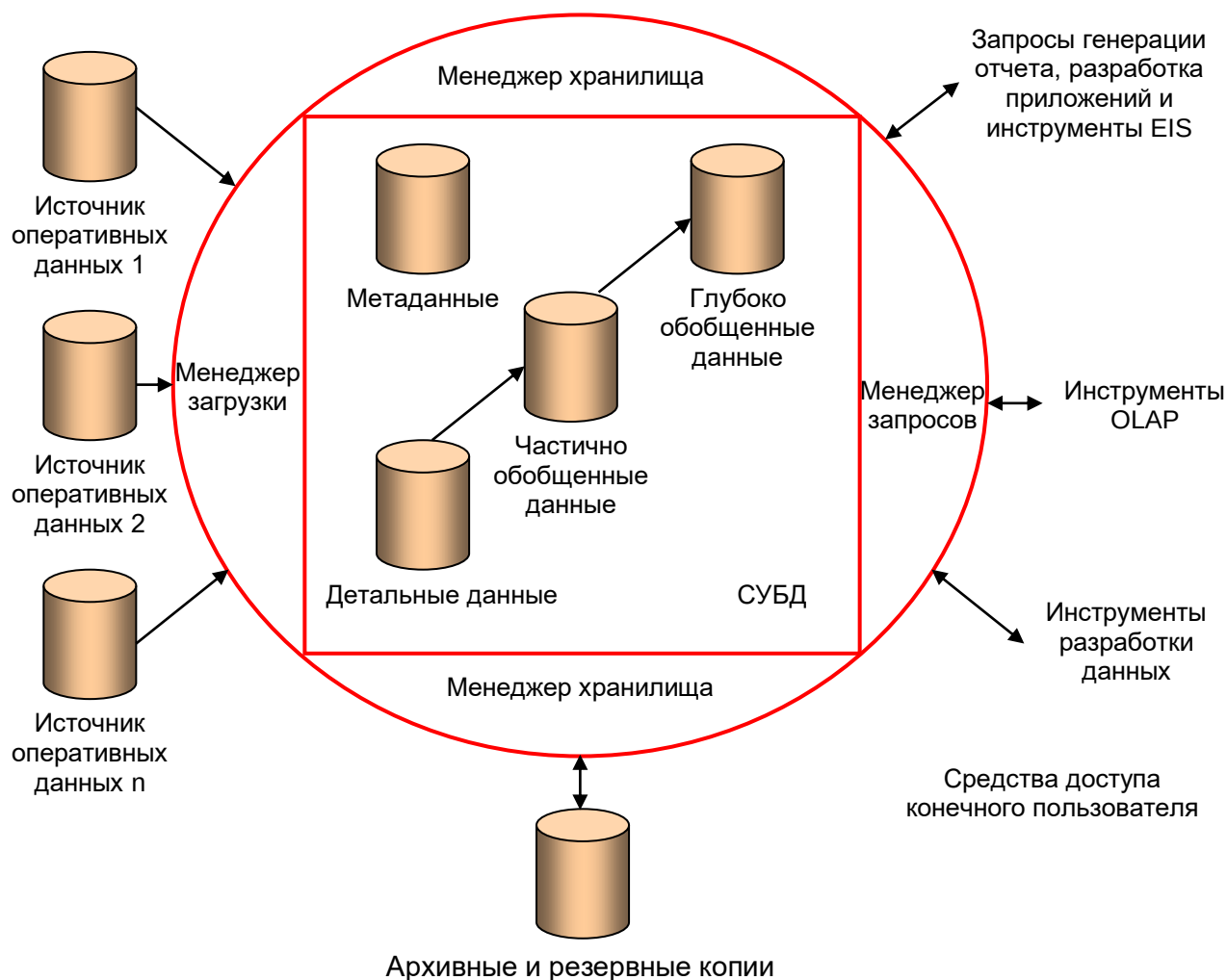


Рис. 16.1. Типичная архитектура хранилища данных

- оперативные данные мейнфреймов, содержащиеся в иерархических и сетевых базах данных первого поколения. По некоторым оценкам, большинство оперативных корпоративных данных хранится в системах этого типа;

- данные различных подразделений, сохраняемые в специализированных файловых системах типа VSAM, RMS и базах данных таких реляционных СУБД, как Informix и Oracle;

- закрытые данные, которые хранятся на рабочих станциях и закрытых серверах;

- внешние системы, например, Internet, коммерчески доступные базы данных или базы данных, принадлежащие поставщикам или клиентам организации.

Менеджер загрузки (load manager), который часто называют внешним (front-end) компонентом, выполняет все операции, связанные с извлечением и загрузкой данных в хранилище. Эти операции включают простые преобразования данных, необходимые для их подготовки к вводу в хранилище. Размеры и сложность данного компонента могут варьироваться в значительной степени, поскольку в его состав обычно входят не только программы собственной разработки, но и инструменты, созданные сторонними поставщиками.

Менеджер хранилища (warehouse manager) выполняет все операции, связанные с управлением информацией, помещенной в хранилище данных. Этот компонент также может включать программы собственной разработки и инструменты, предоставленные сторонними фирмами. Менеджер хранилища выполняет такие операции, как:

- анализ непротиворечивости данных;

- преобразование и перемещение исходных данных из временного хранилища в основные таблицы хранилища данных;

- создание индексов и представлений из базовых таблиц;
- денормализация данных (в случае необходимости);
- обобщение данных (в случае необходимости);
- резервное копирование и архивирование данных.

В некоторых случаях менеджер хранилища также генерирует профили запросов для определения необходимых индексов и требуемых предварительных обобщений данных. Профиль запроса может создаваться для отдельного пользователя, группы пользователей или хранилища данных в целом. Он строится на основе информации, описывающей такие характеристики запросов, как частота выполнения, набор используемых таблиц и размер результирующего набора данных.

Менеджер запросов (query manager), который часто называют внутренним (back-end) компонентом, выполняет все операции, связанные с управлением пользовательскими запросами. Этот компонент обычно создается на базе предоставляемых разработчиком СУБД инструментов доступа к данным, инструментов мониторинга хранилища и программ собственной разработки, использующих весь набор функциональных возможностей СУБД. Сложность менеджера запросов определяется функциональными возможностями, предоставляемыми инструментами доступа к данным и самой СУБД. К числу выполняемых этим компонентом операций относятся управление запросами к соответствующим таблицам и составление графиков выполнения этих запросов. В некоторых случаях менеджер запросов также генерирует профили запросов, позволяющие менеджеру хранилища определить набор необходимых индексов и обобщенных данных.

Само хранилище состоит из нескольких областей:

- *детальные данные*. В этой части хранилища хранятся все детальные данные, описанные в схеме базы данных. В большинстве случаев детальные данные хранятся не на оперативном уровне, а в виде информации, обобщенной до следующего уровня детализации. Как правило, детальные данные периодически добавляются в хранилище с автоматическим выполнением обобщения исходной информации до необходимого уровня;

- *частично и глубоко обобщенные данные*. В этой области хранилища размещаются все данные, предварительно обработанные менеджером хранилища с целью их частичного или глубокого обобщения (aggregate). Эта часть хранилища данных является временной, поскольку она постоянно подвергается изменениям в ответ на изменения профилей запросов. Назначение обобщенных данных состоит в повышении производительности запросов.;

- *архивные и резервные копии*. Этот компонент хранилища данных отвечает за подготовку детальной и обобщенной информации к размещению в резервные и архивные копии. Хотя обобщенные данные генерируются на основе детальных, может потребоваться помещать в резервную копию заранее обобщенные данные, если предполагаемый период их хранения превышает срок хранения тех детальных данных, на основе которых они были созданы. Как правило, резервные и архивные копии размещаются на таких носителях, как магнитная лента или оптический диск;

- *метаданные*. В этой области хранилища данных хранятся все те метаданные (данные о данных), которые используются любыми процессами хранилища. Метаданные могут применяться для разных целей: извлечение и загрузка данных (метаданные используются для отображения источников данных на общее представление информации внутри хранилища); обслуживание хранилища (метаданные применяются для автоматизации подготовки таблиц с обобщенными значениями); часть процесса обслуживания запросов (метаданные используются для направления запросов к наиболее подходящему источнику данных).

Основным назначением хранилища данных является предоставление конечным пользователям информации, необходимой им для принятия стратегических решений. Само хранилище данных должно обеспечивать эффективное выполнение произвольных запросов и предоставлять средства проведения анализа. Высокая производительность хранилища данных достигается за счет тщательного предварительного планирования операций соединения,

обобщения и составления периодических отчетов, которые могут потребоваться конечным пользователям. Хотя определения пользовательских инструментов доступа к данным могут перекрываться, представим их в виде пяти основных групп (Berson and Smith, 1997):

– *инструменты создания отчетов и запросов*. Подразделяются на инструменты создания итоговых отчетов и редакторы отчетов. Инструменты создания итоговых отчетов используются для создания регулярных оперативных отчетов и для подготовки таких объемных пакетных заданий, как выписка заказов/счетов для клиентов или составление платежных ведомостей для выдачи зарплаты сотрудникам. Редакторы отчетов – это недорогие настольные инструменты, предназначенные для нужд конечных пользователей. Инструменты создания запросов в реляционных СУБД служат для ввода или генерации SQL-команд, используемых для извлечения данных из хранилища. Подобные инструменты обычно скрывают от конечных пользователей сложность операторов языка SQL и структур баз данных за счет вставки между пользователем и базой данных промежуточного метауровня. Метауровень – это программное обеспечение, которое предоставляет пользователю некоторое предметно-ориентированное представление содержимого базы данных и позволяет генерировать SQL-операторы с помощью визуальных инструментов типа «выбери и щелкни». Примером подобного инструмента создания запросов является язык Query-By-Example (QBE). Различные инструменты создания запросов весьма популярны среди пользователей бизнес-приложений и могут применяться ими для любых целей, например, для выполнения демографического анализа или подготовки почтовых списков рассылки клиентов организации. Однако по мере усложнения запросов такие инструменты очень быстро становятся неэффективными;

– *инструменты разработки приложений*. Требования конечных пользователей могут быть такими, что встроенные средства создания отчетов и инструменты создания запросов окажутся неадекватными либо из-за того, что требуемый анализ не может быть ими выполнен в принципе, либо из-за того, что пользователю потребуется иметь чрезвычайно высокую квалификацию и немалый опыт. В такой ситуации для обеспечения доступа пользователей к требуемой информации потребуется разработка собственных приложений с использованием графических сред доступа к данным, предназначенных для использования в системах архитектуры клиент/сервер. Некоторые из этих платформ разработки приложений интегрируются с популярными OLAP-инструментами. Они позволяют осуществлять доступ ко всем основным типам СУБД, включая Oracle, Sybase и Informix. Примерами подобных платформ разработки приложений являются PowerBuilder фирмы PowerSoft, Visual Basic фирмы Microsoft, Forte фирмы Forte Software и Business Objects фирмы Business Objects;

– *инструменты информационной системы руководителя* (Exclusive Information System – EIS), которые в последнее время стали называть «информационными системами для всех» (Everybody Information System – EIS), первоначально разрабатывались для поддержки принятия высокоуровневых стратегических решений. Однако впоследствии область применения этих систем была несколько расширена с целью предоставления поддержки управляющему персоналу всех уровней.;

– *инструменты оперативной аналитической обработки* (OLAP-инструменты). Создаются на основе концепции многомерной базы данных. Они позволяют квалифицированным пользователям анализировать данные с помощью сложных многомерных представлений. Типичные бизнес-приложения для этих инструментов включают оценку эффективности маркетинговой компании, предсказание объемов продаж и планирование товарных запасов. При использовании подобных инструментов предполагается, что данные организованы согласно многомерной модели, которая поддерживается специальной многомерной базой данных (Multi-Dimensional Database – MDDb) или реляционной базой данных, предназначенной для работы с многомерными запросами;

– *инструменты разработки данных*. Разработка данных – это процесс открытия новых осмысленных корреляций, распределений и тенденций путем переработки огромного количества информации, извлеченной из хранилища или магазина данных с использованием статистических и математических методов, а также методов искусственного интеллекта. Мето-

ды разработки данных обладают достаточным потенциалом, чтобы превзойти возможности OLAP-инструментов, так как главным фактором использования технологии разработки данных является способность создавать *прогнозируемые*, а не *ретроспективные* модели.

16.3. Информационные потоки в хранилище данных

В технологии хранилищ данных основное внимание уделяется управлению пятью информационными потоками: входным, восходящим, нисходящим, выходным и метапоток (Hachathorn, 1995). Место этих потоков в структуре хранилища данных схематически показано на рис. 16.2. С каждым из этих потоков связаны определенные процессы:

- входной поток – извлечение, очистка и загрузка исходных данных;
- восходящий поток – повышение ценности сохраняемых в хранилище данных путем обобщения, упаковки и распределения исходных данных;
- нисходящий поток – архивирование и резервное копирование информации в хранилище;
- выходной поток – предоставление данных пользователям;
- метапоток – управление метаданными.

Определение. Входной поток – это процессы, связанные с извлечением, очисткой и загрузкой информации из источников данных в хранилище данных.

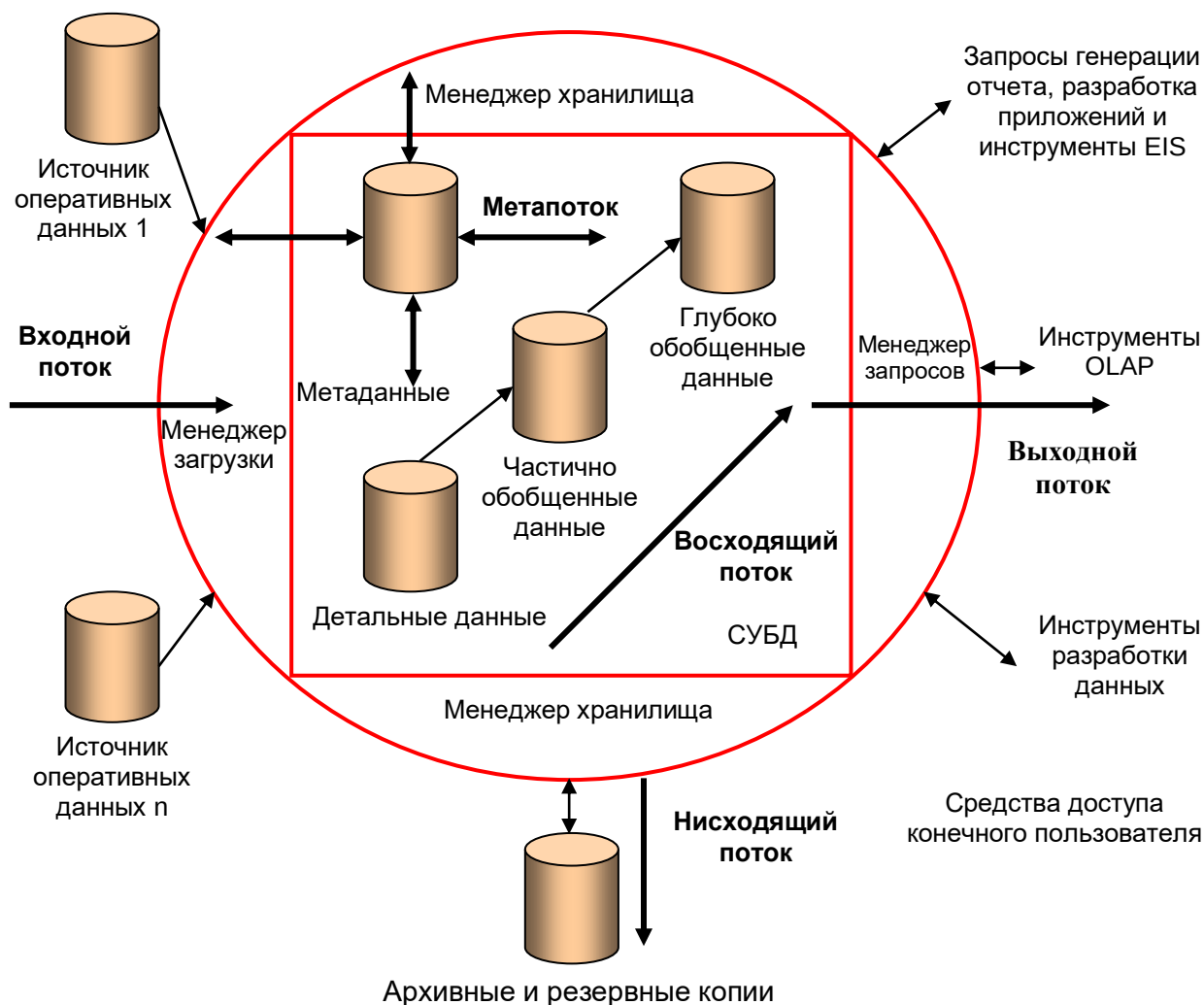


Рис. 16.2. Информационные потоки в хранилище данных

Входной поток связан с выборкой информации из источников данных с целью их последующей загрузки в хранилище данных. Поскольку исходные данные генерируются преимущественно OLAP-системами, эти данные должны быть перестроены в соответствии с требованиями хранилища данных. Перестройка данных включает такие операции, как:

- очистка данных;
- преобразование данных в соответствии с требованиями хранилища данных, включая добавление и/или удаление полей и денормализацию данных;
- проверка внутренней структуры непротиворечивости данных и их непротиворечивости по отношению к данным, уже загруженным в хранилище.

Для эффективного управления входным потоком необходимо подобрать механизм определения момента начала извлечения данных с последующим выполнением требуемых преобразований и проверкой непротиворечивости данных. Для создания единого непротиворечивого представления корпоративных данных очень важно в процессе извлечения информации из источников убедиться в том, что она находится в согласованном состоянии. Сложность процесса извлечения информации зависит от степени взаимной согласованности между различными источниками данных.

Непосредственно после извлечения из источника данные обычно загружаются во временное хранилище с целью выполнения очистки и проверки непротиворечивости. Поскольку этот процесс достаточно сложен, важно, чтобы он был полностью автоматизирован и сопровождался выдачей сообщений о любых возникающих проблемах или сбоях. Для обслуживания входного потока на рынке предлагаются специальные коммерческие инструменты. Однако если требуемая процедура не является тривиальной, то в случае использования коммерческих инструментов потребуется выполнить их предварительную настройку.

Определение. Восходящий поток – это процессы, связанные с повышением ценности сохраняемых в хранилище данных посредством обобщения, упаковки и распределения исходных данных.

Обслуживание восходящего потока включает выполнение следующих действий:

- *обобщение данных* посредством операций выборки, проекции, соединения и группирования связанных данных, выполняемое при получении более удобных и полезных для пользователей представлений информации. Обобщение может включать выполнение не только простых реляционных операций, но и проведение сложного статистического анализа, включая вычисление трендов, кластеризацию и подбор типичных значений;
- *упаковка данных* с преобразованием подробных исходных или обобщенных данных в более удобный формат представления, например, в виде электронных таблиц, текстовых документов, диаграмм, других графических представлений, закрытых баз данных и анимированных материалов;
- *распределение исходных данных* на соответствующие группы для повышения их подготовленности к использованию и доступности.

Параллельно с планированием повышения ценности данных следует учитывать и требования увеличения общей производительности хранилища, а также снижения текущих расходов на его сопровождение. Все эти требования противоречат друг другу, что вынуждает разработчиков либо повышать производительность обработки запросов, либо сокращать расходы на сопровождение.

Определение. Нисходящий поток – это процессы, связанные с архивированием и резервным копированием информации в хранилище данных.

Архивирование устаревших данных играет важную роль при обеспечении высокой эффективности и производительности хранилища данных за счет переноса устаревших данных с ограниченной ценностью на архивный носитель (например, на магнитную ленту или оптические диски). Однако если схема секционирования базы данных выполнена правильно, то

общее количество оперативных данных не должно влиять на производительность хранилища данных. Нисходящий поток информации включает процедуры, обеспечивающие возможность восстановления текущего состояния хранилища в случае потери данных из-за сбоев в программном или аппаратном обеспечении. Архивные данные следует хранить таким образом, чтобы в случае необходимости они снова могли быть восстановлены в хранилище данных.

Определение Выходной поток – это процессы, связанные с предоставлением данных пользователям.

Именно благодаря выходному потоку информации у сотрудников организации создается представление об истинной ценности хранилища данных. Полученные данные могут потребовать перестройки всех бизнес-процессов организации с целью повышения ее конкурентоспособности (Nackathorn, 1995). Существуют следующие основные действия, связанные с выходным потоком:

- *доступ к данным* означает удовлетворение запросов конечных пользователей в отношении нужных им данных. Главное здесь заключается в создании такой среды, в которой пользователи смогли бы эффективно использовать инструменты создания запросов для получения доступа к наиболее подходящему источнику данных. Частота выполнения отдельных запросов пользователей может варьироваться от однократно выполняемого произвольного запроса до регулярно выполняемых запросов или даже запросов, выполняемых по установленному графику. При разработке графика выполнения запросов пользователей очень важно обеспечить использование системных ресурсов максимально эффективным образом;

- *доставка* означает предварительную доставку информации на рабочие станции конечных пользователей. Это относительно новая область обработки информации в хранилищах данных, связанная с процессами типа публикации/подписка. Хранилище данных публикует различные бизнес-объекты, которые периодически подвергаются ревизии в соответствии с установленными шаблонами использования. Пользователи могут подписаться на такой набор бизнес-объектов, который наилучшим образом отвечает их требованиям.

Важной особенностью управления выходным потоком является активная популяризация хранилища данных среди пользователей, что может оказать существенное влияние на функционирование всей организации. В числе дополнительных действий по обслуживанию выходного потока следует также назвать направление запросов к соответствующим целевым таблицам и обработку профилей запросов конкретных групп пользователей с целью выяснения, какие именно обобщения данных могут оказаться полезными. Хранилища данных, содержащие обобщенные данные, потенциально предоставляют пользователям большое количество различных источников данных, способных дать ответ на их конкретные запросы. Однако производительность запроса может в значительной степени варьироваться в зависимости от характера обрабатываемых данных. Наиболее очевидной характеристикой производительности является объем считываемой информации. В состав действий по обслуживанию выходного потока входит и поиск системой наиболее эффективного способа выполнения запроса.

Определение. Метапоток – это процессы, связанные с управлением метаданными.

Предыдущие потоки характеризуют управление хранилищем данных в отношении перемещения данных в хранилище и из него. Метапоток – это процесс, связанный с перемещением метаданных, т. е. данных о других потоках: что содержится в потоке, откуда он поступает, какие операции выполнялись во время очистки, как осуществлялись интеграция и обобщение. Метапоток должен постоянно обновляться в соответствии с происходящими изменениями традиционных источников данных и бизнес-среды.

16.4. Инструменты и технологии хранилищ данных

Создание полностью интегрированного хранилища данных является чрезвычайно сложной задачей, поскольку нет такого разработчика, который смог бы предложить весь необходимый набор инструментов. Поэтому создавать хранилища данных обычно приходится

с использованием нескольких продуктов разных поставщиков. Полная интеграция и организация совместной работы используемых инструментов является чрезвычайно важной и сложной задачей.

Инструменты извлечения, очистки и преобразования данных

Выбор оптимальных инструментов извлечения, очистки и преобразования данных очень важен для успешного создания хранилища данных. Количество фирм-разработчиков, основное внимание которых обращено на удовлетворение требований, предъявляемых к хранилищам данных, постоянно растет. Предлагаемые ими инструменты могут решать гораздо более сложные задачи, чем простое перемещение данных между разными аппаратными платформами. При этом задачи извлечения данных из источника, их очистки и преобразования с последующей загрузкой в конкретную систему могут быть выполнены либо с помощью нескольких разных программных продуктов, либо посредством применения единого интегрированного подхода. Подобные интегрированные решения делятся на следующие категории:

- **генераторы кода.** Создают настраиваемые преобразующие программы на языках третьего или четвертого поколения исходя из предоставленных им определений форматов входных и выходных данных. Основной проблемой этого подхода является необходимость управления большим количеством программ, которые потребуются для поддержки сложного корпоративного хранилища данных. Фирмы-разработчики признают наличие указанной проблемы, поэтому некоторые из них, применив такие методы, как конвейерная обработка и автоматическая разработка графиков, создали специальные управляющие компоненты;

- **инструменты репликации информации базы данных.** Данные инструменты используют триггеры баз данных или журналы регистрации транзакций для обнаружения изменений отдельных элементов данных в одной системе и переноса этих изменений в копию исходных данных, размещенную в другой системе. Большинство инструментов репликации не поддерживает обнаружения изменений в нереляционных файлах и базах данных и зачастую не имеет инструментов для значительного преобразования и очистки данных. Эти инструменты могут быть использованы для восстановления базы данных после сбоя или создания базы данных для магазина данных, но при условии, что количество источников данных невелико, а требуемое преобразование данных имеет относительно простой характер;

- **механизмы динамического преобразования.** Исходя из установленных правил механизмы динамического преобразования извлекают данные из источника через определенные пользователем интервалы времени, преобразуют их, а затем отсылают и загружают обработанную информацию в указанное место назначения. На сегодняшний день большинство программных продуктов поддерживает только реляционные источники данных, однако в последнее время стали появляться продукты, способные работать и с нереляционными исходными файлами и базами данных. Примерами инструментов преобразования данных являются пакеты Enterprise/Access фирмы Apertus Corporation, Warehouse Manager фирмы Prisms Solutions, PASSPORT и Metacenter фирмы Carleton Corporation, ETI-EXTRACT фирмы Evolutionary Technologies Inc., Powermart Suite фирмы Informatica.

СУБД для хранилища данных

СУБД для хранилища данных очень редко бывает источником проблем интеграции. Благодаря относительной зрелости таких программных продуктов, большинство реляционных баз данных интегрируются с другими типами программного обеспечения вполне предсказуемым образом. Однако потенциальным источником проблем может послужить большой размер базы данных хранилища. При работе с подобной базой данных становится особенно важным обеспечение параллельности, а также таких традиционно важных параметров, как высокая производительность, масштабируемость, готовность и управляемость, что обязательно следует принимать во внимание при выборе СУБД.

Специализированные требования к реляционной СУБД (РСУБД), предназначенной для хранилища данных, были опубликованы в документе White Paper (Red Brick Systems, 1996). Они таковы:

- *высокая производительность загрузки данных.* В хранилище данных требуется периодически выполнять загрузку порций новых данных, причем, в ограниченных временных рамках. Производительность процесса загрузки в подобных случаях должна измеряться в сотнях миллионов строк или гигабайтах данных в час. Со стороны бизнес-задач не существует никаких ограничений в отношении максимально допустимого уровня производительности;

- *возможность обработки данных во время загрузки.* При загрузке в хранилище новых или обновленных данных обычно требуется выполнение нескольких последовательных этапов, включающих преобразование данных, фильтрование, переформатирование, проверку целостности, физическое сохранение, индексирование и обновление метаданных. На практике каждый такой этап может выполняться по отдельности, однако в общем процесс загрузки должен выглядеть как единая неразрывная процедура;

- *наличие средств управления качеством данных.* Для перехода к управлению на основе фактической информации требуются данные высочайшего качества. В хранилище данных должна гарантироваться локальная непротиворечивость данных, глобальная непротиворечивость данных, а также целостность данных на уровне ссылок, даже несмотря на использование «грязных» источников данных и громадные размеры базы данных. Хотя загрузка и подготовка данных – необходимые шаги, они все же не являются достаточными. Лишь способность дать ответы на запросы конечных пользователей является действительной мерой успешности создания хранилища данных. Анализ утверждает, что, чем больше ответов было успешно предоставлено пользователям, тем сложнее и изощреннее становятся очередные вводимые ими запросы;

- *высокая производительность запросов.* Управление на основе фактической информации и производительный анализ не должны замедляться из-за низкой производительности обработки запросов со стороны СУБД хранилища данных. Большие комплексные запросы для ключевых бизнес-операций должны завершаться за приемлемое время;

- *широкая масштабируемость по размеру (до терабайт).* Размеры хранилищ данных возрастают с огромной скоростью и достигают величин от сотен гигабайт до терабайт и даже петабайт. Используемая РСУБД не должна иметь никаких архитектурных ограничений на размер базы данных и должна поддерживать модульное и параллельное управление. Случае сбоя РСУБД должна сохранять готовность к работе и предоставлять механизм восстановления до исходного состояния. РСУБД должна поддерживать работу с устройствами массовой памяти (оптические диски или иерархические устройства хранения). Производительность выполнения запроса должна зависеть не от размера базы данных, а от сложности самого запроса;

- *масштабируемость по количеству пользователей.* В настоящее время считается, что доступ к хранилищу данных будет ограничен только относительно небольшим кругом менеджеров. Однако маловероятно, что такая тенденция сохранится и при возрастании значения хранилищ данных. По некоторым оценкам, в недалеком будущем РСУБД для хранилищ данных должны будут поддерживать работу сотен и даже тысяч параллельно работающих пользователей, обеспечивая при этом приемлемую производительность выполнения их запросов;

- *возможность организации сети хранилищ данных.* Хранилище данных должно обладать способностью работать в большой сети, состоящей из многих хранилищ данных. Хранилище данных должно включать инструменты, которые координировали бы перемещение подмножеств данных между отдельными хранилищами. На своей рабочей станции пользователи должны иметь возможность просматривать и работать с содержимым нескольких хранилищ данных;

- *наличие средств администрирования хранилища.* Исключительно большой размер и циклическая природа хранилищ данных требует наличия простых и в то же время гибких ин-

струментов администрирования. РСУБД должна предоставлять средства управления для ограничения ресурсов, подсчета накладных расходов для всех пользователей, а также систему установки приоритетов выполнения запросов для удовлетворения потребностей различных категорий пользователей и видов деятельности. РСУБД должна также иметь средства для отслеживания и настройки режимов рабочей нагрузки, необходимых для оптимизации производительности и пропускной способности системы;

– *поддержка интегрированного многомерного анализа*. Ценность многомерных представлений – общепризнанный факт, а потому поддержка работы с ними непременно должна быть предусмотрена в РСУБД, используемой для организации хранилища данных, поскольку это является условием для обеспечения максимальной производительности реляционных OLAP-инструментов. РСУБД должна поддерживать быстрое и простое создание предварительно подготовленных итоговых значений для больших хранилищ данных, а также предоставлять инструменты для автоматизации процесса создания этих предварительно вычисленных обобщений. Динамическое вычисление обобщенных значений должно быть согласовано с требованиями обеспечения необходимого уровня производительности интерактивной работы пользователей;

– *расширенный набор функциональных средств запросов*. Конечным пользователям необходимо иметь возможность выполнять аналитические вычисления, последовательный и сравнительный анализ, согласованный доступ к детальными обобщенным данным. Использование языка SQL в клиент/серверной среде создания запросов по типу «выбери и щелкни» иногда может оказаться непрактичным и даже просто невозможным из-за высокой сложности пользовательских запросов. В РСУБД должен быть предусмотрен полный набор всех необходимых аналитических инструментов.

При работе с хранилищем данных обычно требуется обработать огромное количество данных, а технология параллельной работы с базами данных предлагает эффективное решение для обеспечения необходимой производительности. Успешность работы параллельных СУБД зависит от эффективного управления многими ресурсами, например, такими как процессоры, память, диски и сетевые подключения. По мере возрастания популярности хранилищ данных фирмы-разработчики создают большие СУБД, использующие технологию организации параллельных вычислений. Основная цель состоит в решении поставленных пользователем задач с использованием нескольких узлов, параллельно работающих над одной и той же проблемой. Важнейшими характеристиками параллельных СУБД являются масштабируемость, оперативность и готовность. Параллельные СУБД могут одновременно выполнять сразу несколько операций с базой данных, разбивая отдельные задачи на малые части так, чтобы они могли быть распределены между несколькими процессорами. Параллельные СУБД должны уметь выполнять параллельные запросы. Иными словами, они должны уметь разбивать большие сложные запросы на подзапросы, параллельно запускать отдельные подзапросы на выполнение, а затем собирать вместе полученные результаты. Такие СУБД должны уметь выполнять в параллельном режиме загрузку данных, сканирование таблицы, а также архивирование и резервное копирование данных.

Существуют два основных типа архитектуры аппаратного обеспечения для выполнения параллельных вычислений, которые могут использоваться в качестве платформы для сервера базы данных в хранилищах данных:

– симметричная мультипроцессорная обработка (Symmetric Multi-Processing – SMP) – это группа тесно связанных процессоров, которые совместно используют оперативную и дисковую память;

– массовая мультипроцессорная обработка (Massively Multi-Processing – MMP) – это группа слабо связанных процессоров, каждый из которых использует свою собственную оперативную и дисковую память.

Метаданные хранилища данных

Управление метаданными в хранилище данных представляет собой чрезвычайно сложную и многогранную задачу. Метаданные используются в самых разных целях, а управление метаданными является важнейшим вопросом при создании полностью интегрированного хранилища данных. Основным назначением метаданных является сохранение информации о происхождении данных с той целью, чтобы администраторы хранилища могли знать историю любого элемента данных, помещенного в хранилище. Однако проблема состоит в том, что метаданные в пределах хранилища данных обладают несколькими функциями, связанными с преобразованием и загрузкой данных, обслуживанием хранилища данных и генерацией запросов.

Метаданные, связанные с преобразованием и загрузкой данных, должны описывать источник данных и любые изменения, внесенные в эти данные. Например, для каждого исходного поля должны храниться такие сведения, как уникальный идентификатор, исходное имя поля, тип источника данных, исходное расположение, включая системное и объектное имя, а также тип преобразованных данных и имя таблицы назначения. Если поле подвергается какому-то изменению, начиная с простого изменения его типа и вплоть до выполнения над ним сложного набора процедур и функций, все эти изменения также должны быть записаны.

Метаданные, связанные с управлением данными, описывают способ хранения данных в хранилище. Каждый объект базы данных должен быть описан, включая данные, содержащиеся во всех таблицах, индексах и представлениях. Кроме того, должны быть описаны и любые связанные с этими объектами ограничения. Данная информация хранится в системном каталоге СУБД, но с учетом некоторых дополнительных ограничений, присущих хранилищам данных. Например, метаданные должны описывать любые поля, связанные с обобщенными данными, включая способ обобщения. Кроме того, должно быть описано секционирование таблиц, в том числе ключ секционирования, а также диапазон данных, связанных с каждой секцией.

Метаданные требуются также менеджеру запросов для генерации соответствующих запросов. Менеджер запросов генерирует дополнительные метаданные о выполняемых запросах, которые могут быть использованы для генерации исторических данных обо всех выполненных запросах и их профилях для каждого пользователя, группы пользователей или хранилища данных в целом.

Важнейшим вопросом интеграции является синхронизация метаданных разного типа, используемых в пределах всего хранилища данных. Поскольку разные инструменты хранилища данных создают и используют свои собственные метаданные, для достижения полной интеграции требуется сделать так, чтобы эти инструменты могли пользоваться метаданными совместно. Задача в этом случае заключается в синхронизации метаданных между разными программными продуктами различных фирм-разработчиков, использующих разные хранилища метаданных. Например, необходимо идентифицировать нужный элемент метаданных на соответствующем уровне детализации одного программного продукта, потом установить его соответствие другому элементу метаданных на соответствующем уровне детализации другого программного продукта, а затем отрегулировать любые существующие между ними различия в способе кодирования. Эту операцию следует повторить для всех других метаданных, общих для двух программных продуктов. Более того, любые изменения метаданных или даже «мета-метаданных» в одном программном продукте должны быть переданы другому программному продукту. Задача синхронизации двух программных продуктов очень сложна, а потому повторение этого процесса для шести или более продуктов, которые образуют программное обеспечение хранилища данных, может потребовать очень большого расхода ресурсов. Тем не менее синхронизация метаданных обязательно должна быть выполнена. Решить данную проблему можно двумя способами:

- с помощью механизмов автоматической передачи метаданных между хранилищами метаданных разных инструментов;
- путем создания репозитория метаданных.

В настоящее время ведутся исследования возможных механизмов передачи метаданных. Ранние продукты, построенные на передаче метаданных, имели очень ограниченный успех из-за отсутствия стандарта обмена метаданными. В 1995 году был образован комитет Meta-data Coalition Committee, который разработал спецификацию Meta-data Interchange Format (MIF), предназначенную для стандартизации процесса передачи метаданных. В настоящее время она используется фирмами-разработчиками для создания инструментов обмена метаданными. Несмотря на полезность этих продуктов, все же лучше для этой цели использовать репозиторий метаданных.

Репозиторий метаданных объединяет различные типы метаданных в единое хранилище и может синхронизировать и реплицировать метаданные, распределенные по всему хранилищу данных. Таким образом, репозиторий метаданных представляет собой источник метаданных для других инструментов хранилища данных. В качестве примеров репозитория метаданных приведем PLATINUM Repository фирмы Platinum Technologies, ROCHADE Repository фирмы R & O, Directory Manager фирмы Prisms Solutions и Universal Directory фирмы Logic Works.

Инструменты управления и администрирования

Хранилище данных обязательно должно включать инструменты администрирования, предназначенные для управления столь сложной средой. Эти инструменты относительно ограничены в своих возможностях, особенно те, которые хорошо интегрированы с различными типами метаданных и с типичными операциями в хранилищах данных. Инструменты управления и администрирования хранилищ данных должны позволять выполнять такие операции, как:

- мониторинг загрузки данных из нескольких источников;
- проверка качества и целостности данных;
- управление и обновление метаданных;
- мониторинг текущей производительности базы данных с целью обеспечения быстрой реакции на запрос и эффективного использования ресурсов;
- аудит процессов использования хранилища данных с целью сбора информации о стоимости выполнения каждой пользовательской работы;
- репликация, разбиение и распределение данных;
- поддержка эффективного управления хранилищем данных;
- очистка данных;
- архивирование и резервное копирование данных;
- средства восстановления после сбоя;
- управление средствами защиты.

В качестве примеров инструментов управления и администрирования можно назвать пакеты HP Intelligent Warehouse фирмы Hewlett-Packard, FlowMark and Data Hub фирмы IBM, Site Analyzer фирмы Information Builders, Inspect/SQL фирмы Precise Software, Prism Warehouse Manager фирмы Prisms Solutions, Enterprise Control and Coordination фирмы Red Brick Systems, SAS/CPE фирмы SAS Institute и SourcePoint фирмы Software AG.

16.5. Магазины данных

Вслед за появлением и быстрым развитием понятия хранилища данных появилась и близкая ему концепция магазинов данных или витрина данных (data mart).

Определение. Магазин данных – это подмножество хранилища данных, которое поддерживает требования отдельного подразделения или деловой сферы организации.

Магазин данных содержит некоторое подмножество данных хранилища данных, которое обычно представлено в виде обобщенной информации, связанной с некоторым подразделением или деловой сферой предприятия. Магазин данных может быть независимым или определенным образом связанным с централизованным хранилищем данных.

По мере увеличения размеров хранилища данных для удовлетворения различных потребностей организации потребуется принимать те или иные компромиссные решения. Популярность магазинов данных основана на том очевидном факте, что корпоративные хранилища данных создавать и использовать сложнее. На рис. 30.3 показана типичная архитектура хранилища данных и связанных с ним магазинов данных. Перечислим основные отличительные черты магазина данных от хранилища данных:

- магазин данных отвечает требованиям пользователей только одного из подразделений организации или некоторой ее деловой сферы;
- магазин данных обычно не содержит детальных оперативных сведений (в отличие от хранилища данных);
- поскольку магазин данных содержит меньше информации, чем хранилище, структура информации магазина данных более понятна и проста в управлении.

Существует несколько подходов к созданию магазинов данных. Один из них заключается в создании корпоративного хранилища данных, который может непосредственно использоваться пользователями, а также поставлять сведения для магазинов данных. Другой подход заключается в создании нескольких магазинов данных с возможностью их интеграции в виде единого хранилища данных. Наконец, третий подход основан на создании инфраструктуры корпоративного хранилища данных с одновременным созданием одного или нескольких магазинов данных, предназначенных для удовлетворения насущных бизнес-потребностей организации.

Для магазина данных можно выбрать двухуровневую или трехуровневую архитектуру. При этом хранилище данных (если это хранилище данных поставляет данные для магазина данных) образует первый уровень (в случае необходимости), магазин данных – второй уровень, а рабочая станция пользователя – третий уровень. При этом данные распределены между всеми этими тремя уровнями.

Существует несколько перечисленных ниже причин, по которым следует создавать магазины данных:

- для предоставления пользователям доступа к данным, которые приходится анализировать чаще других;
- для предоставления данных группе пользователей некоторого отдела или деловой сферы в форме, которая соответствует их коллективному представлению о данных;
- для сокращения времени ответа на запрос (за счет сокращения объема обрабатываемых данных);
- для предоставления данных, структурированных в соответствии с требованиями, предъявляемыми используемыми инструментами доступа к данным. Для многих инструментов доступа к данным (особенно для специальных инструментов разработки данных или многомерного анализа) может потребоваться создать отдельную внутреннюю структуру базы данных. На практике подобные инструменты часто создают свои собственные магазины данных, предназначенные исключительно для поддержки их собственных специфических функций;
- магазины данных обычно содержат меньшее количество данных, а потому такие задачи, как очистка, загрузка, преобразование и интеграция данных, выполняются проще. Следовательно, реализация и настройка магазина данных выполняется проще, чем разработка и реализация корпоративного хранилища данных;
- стоимость реализации магазина данных обычно существенно ниже, чем стоимость создания хранилища данных;
- круг потенциальных пользователей магазина данных более четко определен, поэтому учесть их требования и организовать необходимую поддержку проще, чем в случае корпоративного хранилища данных.

Процесс создания и управления магазином данных можно оценить с помощью некоторых основных характеристик, перечисленных ниже:

– *функциональность*. Возможности магазина данных возрастают вместе с ростом их популярности. В отличие от небольших и легкодоступных баз данных, некоторые магазины данных должны быть масштабируемы вплоть до гигабайтных размеров и предоставлять средства для выполнения сложного анализа с использованием инструментов разработки данных. Сложность и размер некоторых магазинов данных порой соответствует сложности и размерам корпоративных хранилищ данных небольшого масштаба;

– *размер базы данных*. Пользователям магазина данных требуется обеспечить меньшее время реакции системы по сравнению с временем ответа при обработке запросов в хранилище данных. Однако производительность выполнения запросов падает по мере возрастания размера базы данных магазина. Некоторые фирмы-разработчики магазинов данных исследуют способы сокращения размеров баз данных магазинов с целью повышения их производительности. Например, в программном продукте Pilot Decision Support Suite фирмы Pilot Software поддерживаются динамически регулируемые размерности и иерархии с целью сокращения размера базы данных и времени консолидации данных. Динамические размерности позволяют создавать обобщения по требованию без их предварительного расчета и хранения в кубе многомерной базы данных;

– *производительность загрузки данных*. В магазине данных должен поддерживаться баланс между двумя критически важными параметрами: временем реакции системы и производительностью загрузки данных. Магазин данных, созданный для быстрого получения ответа на запросы пользователя, будет иметь большое количество сводных таблиц и обобщенных показателей. К сожалению, создание таких таблиц и вычисление этих показателей существенно увеличивает время загрузки данных. Фирмы-разработчики внимательно исследуют этот вопрос и пытаются повысить производительность загрузки. Например, фирма Red Brick разработала технологию Continually Adaptive Indexing TARGETindex, предназначенную для индексирования, которое автоматически и непрерывно адаптируется к постоянно обновляемым данным. В некоторых многомерных базах данных, например, в базе данных Arbor Software Essbase, поддерживается частично-последовательное обновление данных, так что структуру многомерной базы данных не потребуется изменять при каждом обновлении, что обычно приходилось делать раньше. Дело в том, что в новом варианте обновляются только подверженные изменениям ячейки;

– *доступ пользователей к данным в нескольких магазинах данных*. Одно из решений этой задачи заключается в репликации данных между различными магазинами данных или, иными словами, в создании *виртуальных магазинов данных*. Виртуальные магазины данных являются представлениями нескольких физических магазинов данных или корпоративных хранилищ данных, настроенных согласно требованиям особых групп пользователей. В настоящее время некоторые фирмы-разработчики предлагают специализированные программные продукты для управления виртуальными магазинами данных, например, DSS Administrator фирмы MicroStrategy;

– *доступ к магазинам данных через Internet/Intranet*. Технологии Internet/Intranet предлагают пользователям дешевые средства доступа к магазинам и хранилищам данных с помощью таких Web-браузеров, как Netscape Navigator и Microsoft Internet Explorer. Инструменты доступа к магазинам данных через Internet/Intranet обычно располагаются между Web-браузером и инструментами анализа данных. Фирмы-разработчики предлагают подобные решения с расширенными возможностями работы в Web-среде, например, Information Advantage фирмы MicroStrategy и PilotSoftware фирмы Arbor Software. Эти программные продукты включают компоненты Java и ActiveX;

– *администрирование*. Поскольку количество магазинов данных в организации может быстро возрасти, то возникает потребность в централизованном управлении и координации деятельности всех магазинов данных. Сразу после копирования в магазины данные могут стать противоречивыми, поскольку каждый пользователь может изменять их в своем магазине данных с целью выполнения различных видов анализа. Администрирование нескольких магазинов данных в пределах одной организации – достаточно сложная задача, поскольку

для ее решения приходится выполнять поддержку версий, контролировать непротиворечивость и целостность информации, обеспечивать корпоративную безопасность, а также выполнять настройку производительности. Среди подходящих для этой цели программных продуктов следует упомянуть DSS Administrator фирмы MicroStrategy и Site Analyzer фирмы IBI;

– *установка*. Процесс создания магазина данных со временем все больше усложняется. Фирмы-разработчики предлагают программные продукты, называемые «полностью готовыми магазинами данных», которые на самом деле представляют собой недорогой набор инструментов для работы с магазинами данных. К таким продуктам относятся SmartMart фирмы IBI, Visual Warehouse фирмы IBM и PowerMart фирмы Informatica.